Android平台开发技术

李莹
liying@buaa.edu.cn

课程网站

课程中心

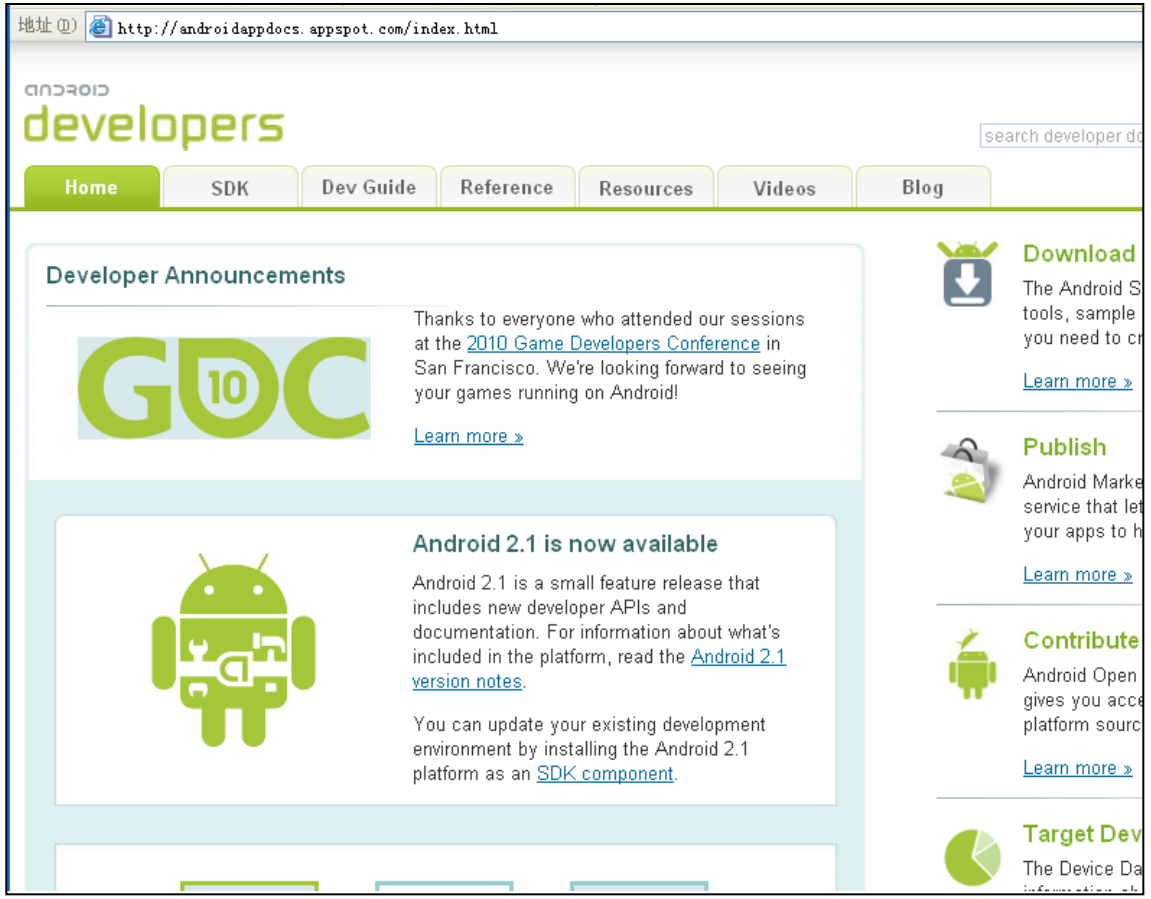
- https://course-buaa-edu-cn.vpn.buaa.edu.cn:8118/portal/site/f711ef16-cc7b-4f6e-8039-09ae87c50c75

课程内容

- 1 Android平台综述
- 2 活动
- 3 事件处理机制
- 4 服务
- 5 广播机制
- 6 数据持久化
- 7 内容容器
- 8 机器学习实践应用（华为）

获取Android资料的途径

- http://developer.android.com （翻墙）
- http://androidappdocs.appspot.com
  - 下载SDK等相关软件
  - 查看文档
  - 在线帮助
  - 开发社区
- http://www.android.com/
- http://www.android123.com.cn/
- http://www.androidcn.org/index.php
- https://sites.google.com/site/androidappcourse/home

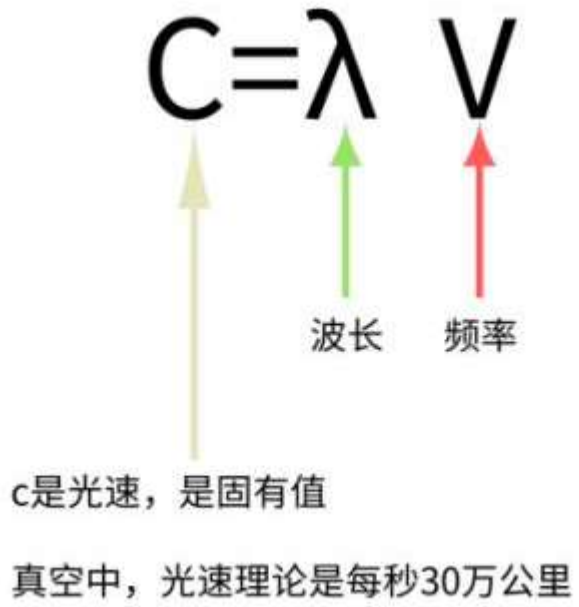


学习方法

- 建立开发环境
- 阅读SDK 文档
- 背景知识
  - Java
  - 面向对象
  - 设计模式
  - J2ME等其它手机编程(选)

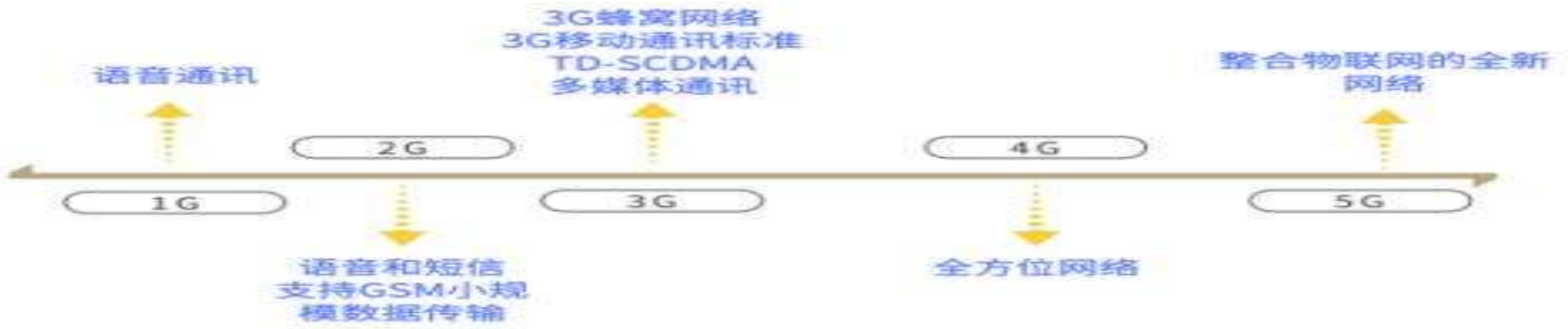
无线通讯技术发展

- 从1G到5G
  - 无线通讯：传输电磁波
  - 频率越来越高，波长原来越小
  - 基站越来越密



无线通讯技术发展

- 第一代：1G(1985)
  - 模拟信号
  - 只能通话
- 第二代：2G(1990)
  - 数字信号(32Kbps)
  - 上网、发送短信
- 第三代：3G(2000)
  - 较高频宽(2Mbps)
  - 视频通讯
- 第四代：4G(2010)
  - 高频宽(100Mbps)
  - 移动通讯
- 第五代：5G(2017)
  - 超高频宽(2.5Gbps)
  - 物联网



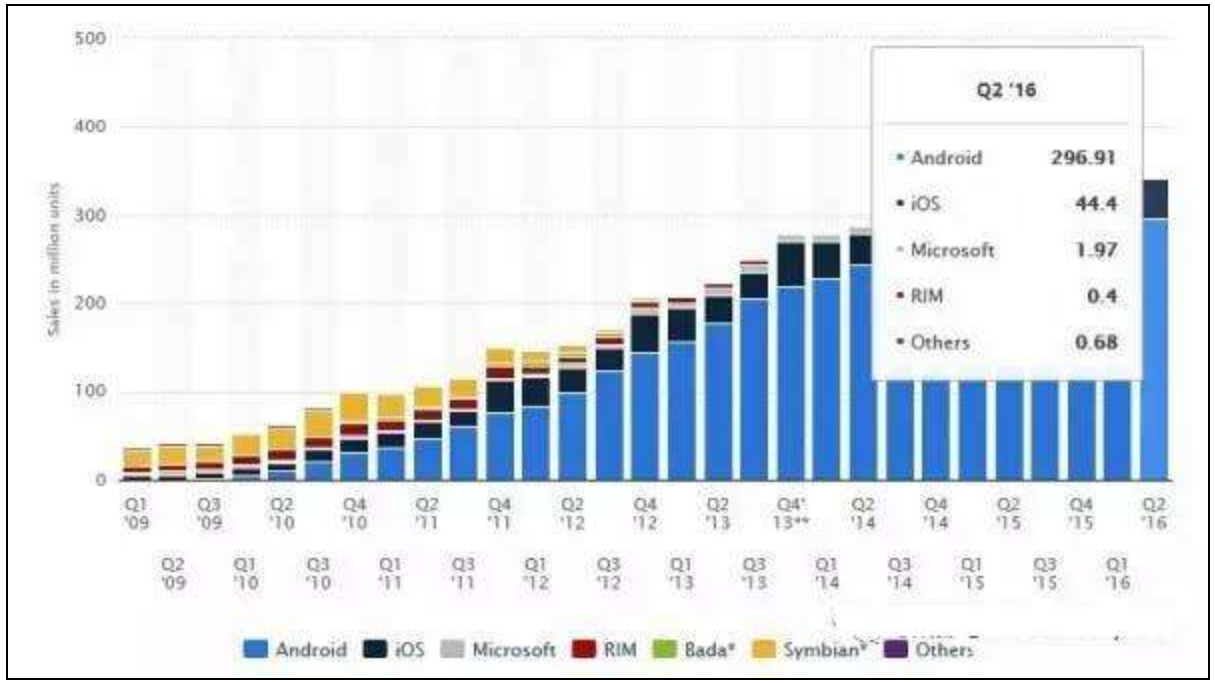
代际	关键特征	峰值速率	核心新业务	物理层关键技术	网络架构
1G (1985)	模拟信号	~2.4 Kbps	无线通话	FM（调频）	模拟蜂窝
2G (1990)	数字信号	32 Kbps	短信、WAP网页	TDMA（时分）	数字蜂窝
3G (2000)	移动宽带	2 Mbps	视频通话、彩信	CDMA（码分）	分组交换
4G (2010)	高速宽带	100 Mbps	高清视频、手游	OFDMA（频分并行）	全IP化
5G (2017)	万物互联	2.5 Gbps	自动驾驶、AR/VR	毫米波 + 波束成形	云化核心网

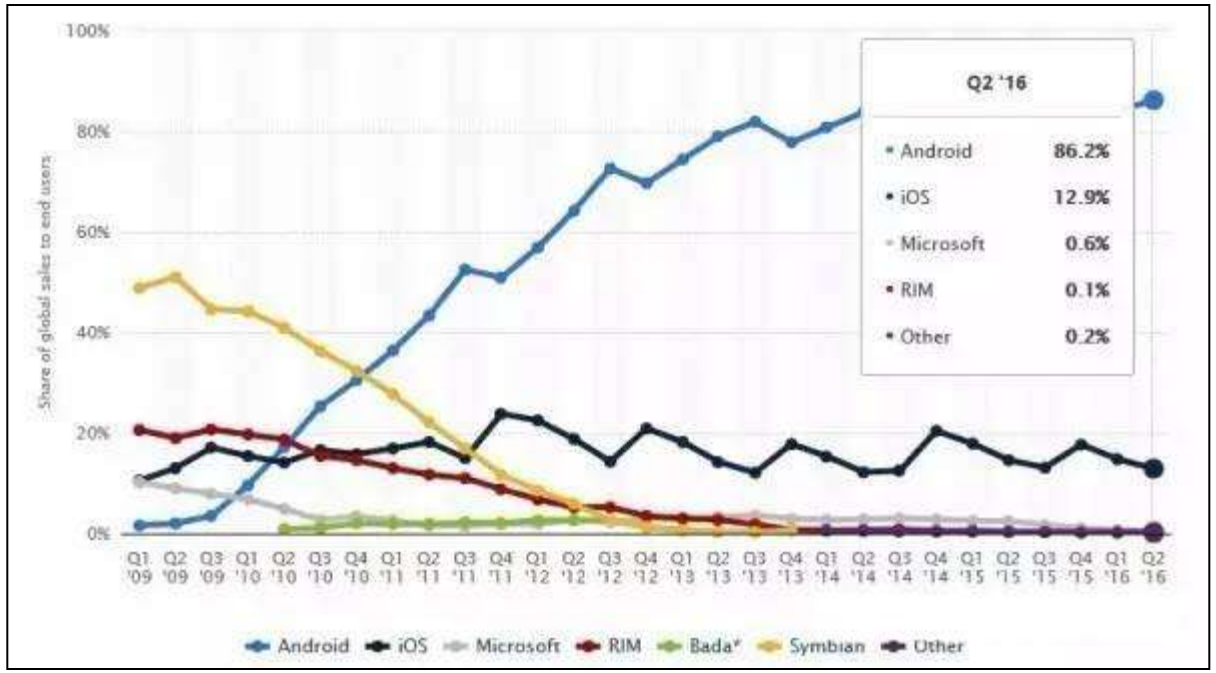
移动操作系统

- 支持移动设备APP开发

  Android、iOS、Windows Phone 7、Symbian和 BlackBerry OS

  - 应用软件互补兼容





Android和IOS

安卓系统和苹果系统

类型	开发平台不同	开发理念不同	运行机制不同	硬件使用效率不同	程序优化机制不同	系统安全性差异
安卓	Linux	免费、开源	无法强制限制内存资源使用	硬件设计和软件开发相对独立	应用软件缺乏监管，优化不到位	开源缺乏管控，极易被破解和破坏
苹果	Unix	收费、闭源	严格控制内存资源使用（墓碑机制、沙漏机制）	软件适配硬件资源	有严格审核机制，利于版本优化	系统封闭，漏洞极少，很难被攻击

开放手机联盟OHA



OHA（Open Handset Alliance）

开放手机联盟是美国Google公司2007年11月5日宣布组建的一个全球性联盟组织。

- 加速智能移动设备的发展、提供用户更多更好更便宜的服务
- 支持Google发布的Android手机操作系统或者应用软件，共同开发名为Android的开放源代码移动系统
- 开放手机联盟包括手机制造商、手机芯片厂商和移动运营商几类。目前，联盟成员数量已经达到了84 家
- 由OHA开发的Android™，是第一个完整的、开放的、免费的智能移动开发平台


Handset Manufacturers
MOTOROLA
htc
smart mobility
SAMSUNG
LG Electronics
Software
Ascender Corporation
ebay
esmertec
Google
livewire
Living Image
NUANCE
pv
SkyPoP
SONiVOX
sound that rocks.
Mobile Operators
中国移动通信
CHINA MOBILE
Designing The Future
KDDI
NTT DoCoMo
Sprint
TELECOM
Telefónica
T-Mobile
open handset alliance
Semiconductor
Audience
BROADCOM.
intel
MARVELL
nVIDIA.
QUALCOMM
SiRF
Synaptics
Texas Instruments
Commercialization
Aplix Corporation
noser
tat
WIND RIVER

Android历史（1）



- 你了解Android吗?
  - 名字由来
    - 外表像人类的机器人——《未来夏娃》，1886
    - Droid字体——Ascender设计的Logo，2010
    - 体现开源、直接面向开发者精神
  - 版本发展
    - 阿童木（Android Beta），2007年11月
    - 发条机器人（Android 1.0）
    - Android第一版Android 1.1，2008年9月

Android历史（2）

- 你了解Android吗？（续）
  - 甜点作为版本号：纸杯蛋糕（Cupcake）、甜甜圈（Donut）、闪电泡芙（Éclair）、冻酸奶（Froyo）、姜饼（Gingerbread）、蜂巢（Honeycomb）、冰淇淋三明治（Ice Cream Sandwich）、果冻豆（Jelly Bean）、奇巧（KitKat）、棒棒糖（Lollipop）、棉花糖（Marshmallow）、牛轧糖（Nougat）、奥利奥（Oreo）、馅饼（Pie）。



猜测下一代Android命名？

Android历史（2）

版本名称	版本号	版本发布时间	对应API
(no code name)	1.0	2008年9月23日	API level 1
(no code name)	1.1	2009年2月2日	API level 2
Cupcake	1.5	2009年4月17日	API level 3,NDK 1
Donut	1.6	2009年9月15日	API level 4,NDK 2
Eclair	2.0.1	2009年12月3日	API level 6
Eclair	2.1	2010年1月12日	API level 7,NDK3
Froyo	2.2.x	2010年1月12日	API level 8,NDK 4
Gingerbread	2.3 – 2.3.2	2011年1月1日	API level 9,NDK5
Gingerbread	2.3.3 – 2.3.7	2011年9月2日	API level 10
Honeycomb	3.0	2011年2月24日	API level 11
Honeycomb	3.1	2011年5月10日	API level 12,NDK 6
Honeycomb	3.2.x	2011年7月15日	API level 13
Ice Cream Sandwich	4.0.1 – 4.0.2	2011年10月19日	API level 14,NDK 7
Ice Cream Sandwich	4.0.3 – 4.0.4	2012年2月6日	API level 15,NDK 8
Jelly Bean	4.1	2012年6月28日	API level 16
Jelly Bean	4.1.1	2012年6月28日	API level 16
Jelly Bean	4.2-4.2.2	2012年11月	API level 17
Jelly Bean	4.3	2013年7月	API level 18
KitKat	4.4	2013年7月24日	API level 19
Kitkat Watch	4.4W	2014年6月	API level 20
Lollipop(Android L)	5.0/5.1	2014年6月25日	API level 21/API level 22
Marshmallow(Android M)	6.0	2015年5月28日	API level 23

三大优势：

- 开放性
- 定制化
- 免费用

开发趋势：

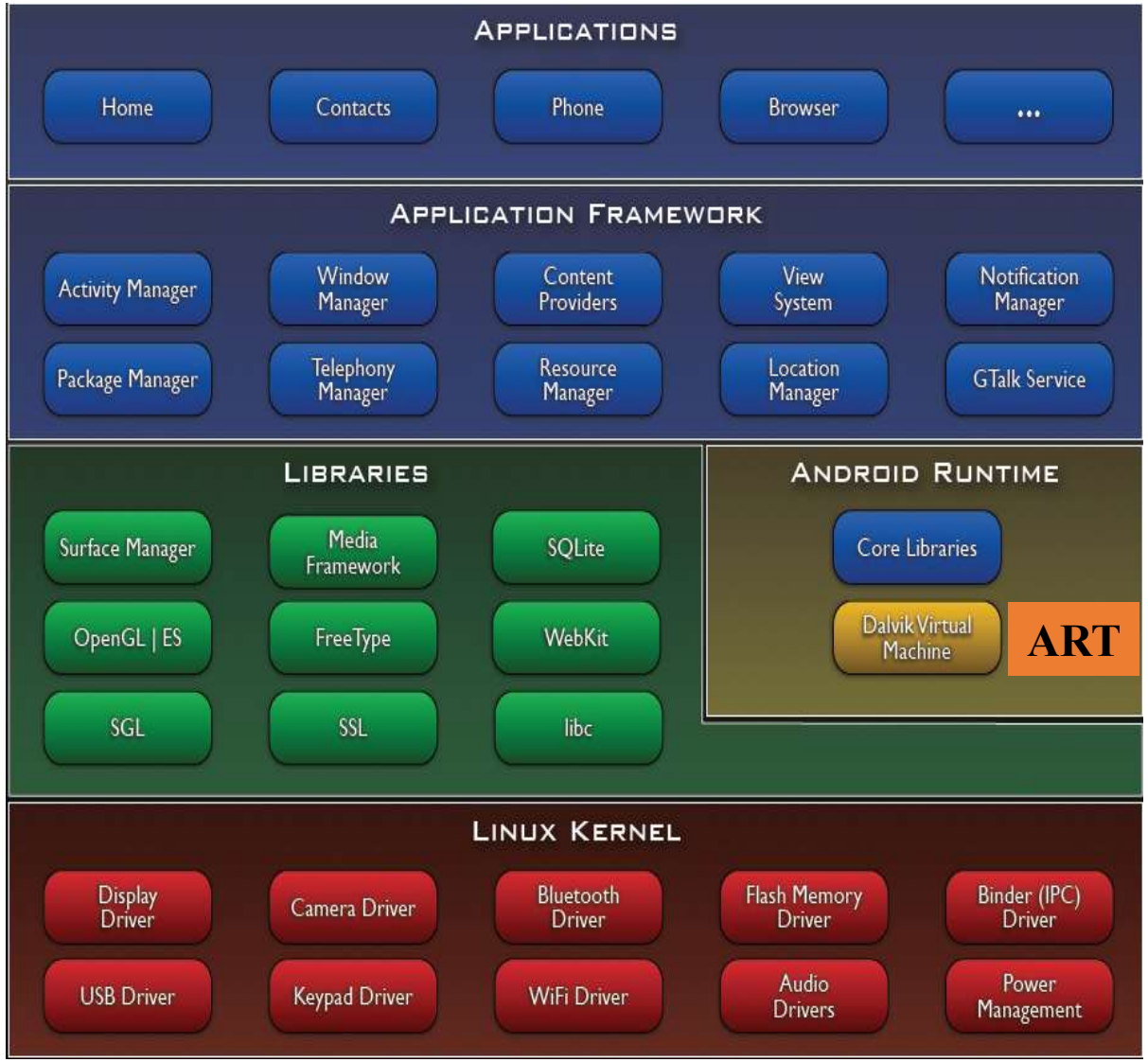
- 系统碎片化：Rom定制
- 屏幕碎片化：适配尺寸

Android系统架构（1）

- 系统架构设计方法

  软件层叠方式（Software Stack）

  - 层内高内聚、层间低耦合
  - 分工协作、易于维护



Android系统架构（2）

- Linux内核层

  提供核心服务

  - 硬件驱动
  - 进程管理
  - 内存管理
  - 网络连接
  - 安全防护



Android系统架构（2）

- 系统运行库层

  提供主要特性支持

  - 数据库
  - 3D绘图
  - 浏览器
  - JAVA编程
  - 虚拟机

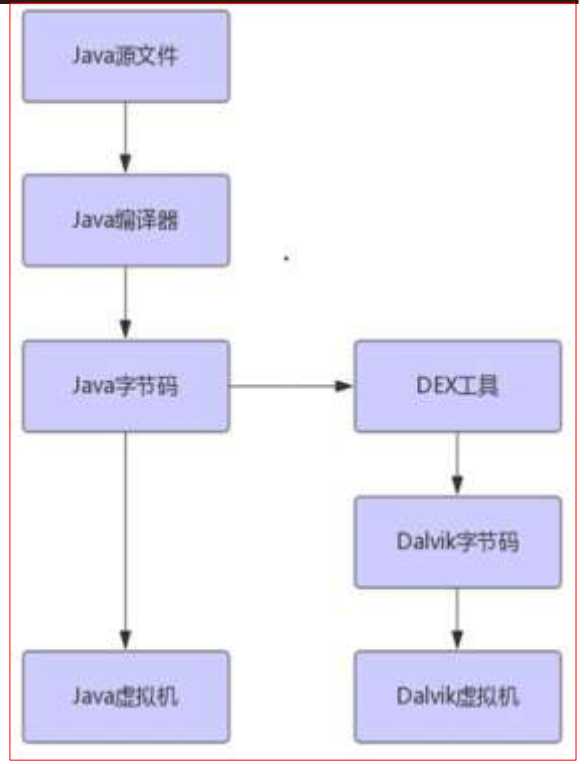


DVM (5.0后升级为ART)

- Google
- Dalvik excutable (*.dex)

JVM

- Oracle (Sun)
- Java bytecode (*.class)



Android系统架构（2）

- 应用框架层

  提供各种API

  - 供系统程序调用
  - 供开发程序调用



- 应用层

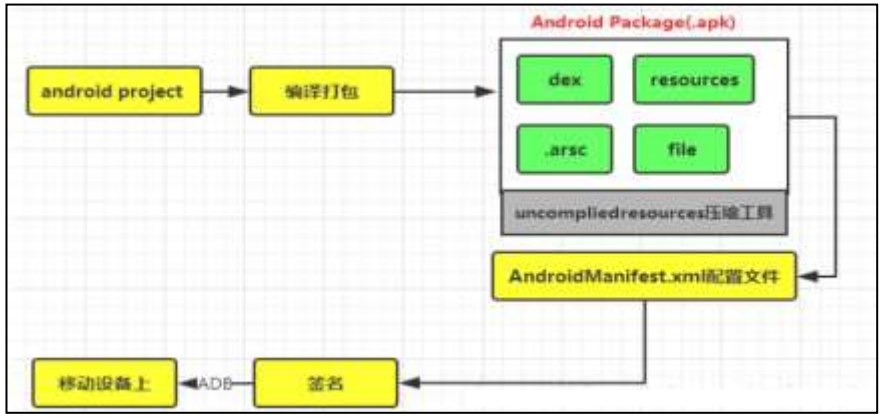
  提供各种应用程序

  - 邮件
  - 日历
  - 地图
  - ……



Android开发环境

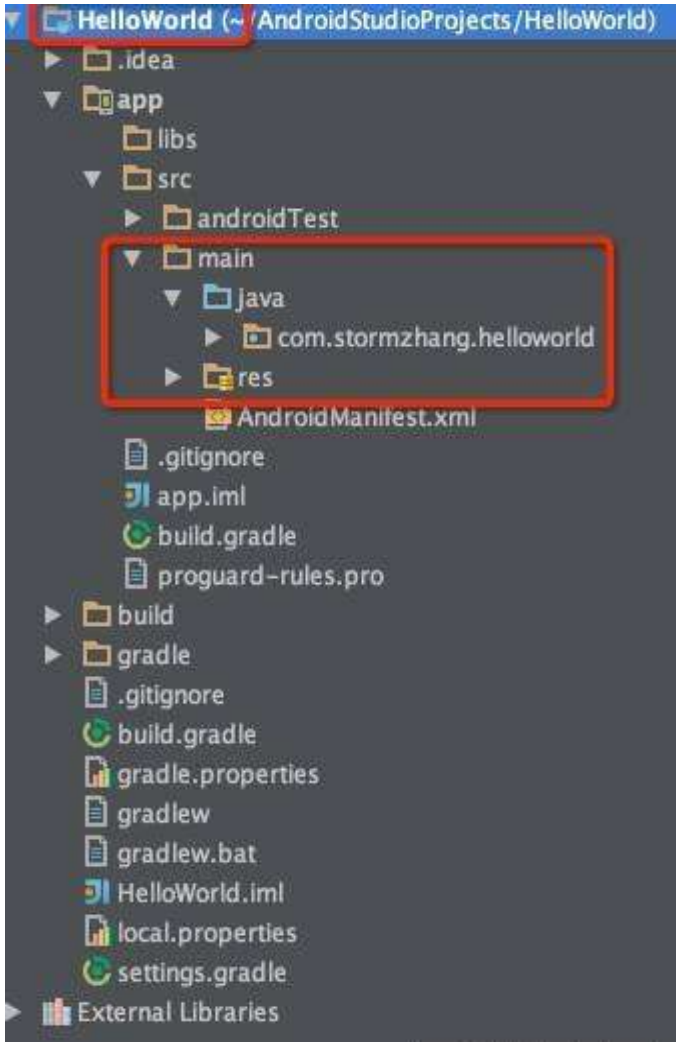
- Android主流开发环境（3种）
  - Eclipse + ADT + SDK（被取代）
  - Android Studio + SDK（建议用）
  - IntelliJ IDEA + SDK（轻量级）



序	名称	说明
1	Dalvik	安卓特有虚拟机
2	AVD	Android virtual device，安卓模拟器
3	ADT	Android development tools，安卓开发工具
4	SDK	Software development kit，软件开发工具包
5	DDMS	Dalvik debug monitor service，安卓调式工具
6	adb	安卓调试桥，在sdk的platform-tools目录下，功能很多,命令行必备
7	DX工具	将.class转换成.dex文件
8	AAPT	android asset packing tool，安卓资源打包工具
9	R.java文件	由AAPT工具根据App中的资源文件自动生成,可以理解为资源字典
10	AndroidManifest.xml	app包名 + 组件声明 + 程序兼容的最低版本 + 所需权限等程序的配置文件

Android Studio开发APP

- 项目结构
  - build：编译时自动生成的文件
  - libs：使用的第三方jar包
  - AndroidTest：编写测试用例
  - java：编写java代码
  - res：使用的资源（图片、布局、字符串）
  - AndroidManifest.xml：配置文件
  - test：编写Unit测试用例
  - . gitignore：将指定目录或文件排除在版本控制外
  - app.iml：IntelliJ IDEA 项目自动生成的文件
  - build.gradle：指定很多项目构建相关的配置
  - proguard-rules.pro：指定项目代码的混淆规则



注意：资源文件只能以小写字母和下划线做首字母，随后的名字中只能出现 [a-z0-9_.]

Android四大组件

- 四大组件
  - 活动：Activity，界面上显示的内容都要放在活动里
  - 服务：Service，后台运行，不可见
  - 广播接收器：Broadcast Receiver，接收和发送广播消息
  - 内容提供器：Content Provider，跨应用程序间共享数据的唯一方法

Android四大组件——活动（1）

- 活动组件（Activity）

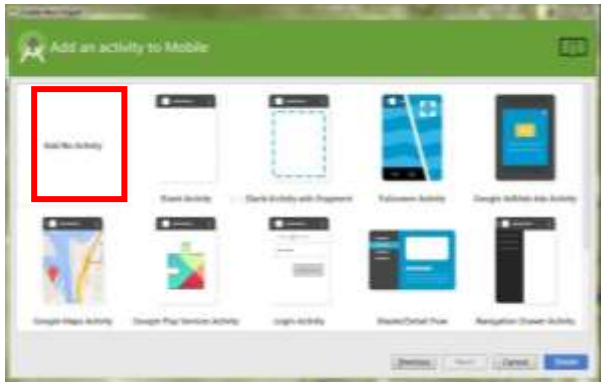
  一个APP可以包含0个或多个活动

  - 当做屏幕、显示界面、用于交互
  - 用栈存储（任务栈）

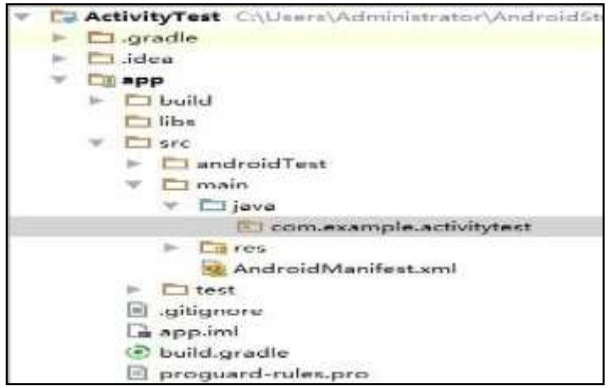
- 新建活动

  逻辑和视图分离，一个活动对应一个布局

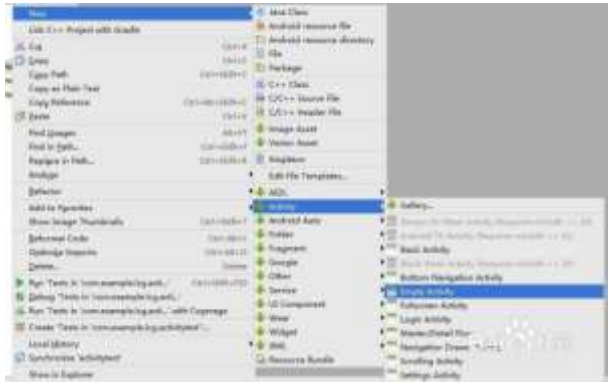
  - 新建活动：自动\手动，在src中新建类，继承Activity类
  - 新建布局：显示界面内容，在res/layout中新建xml布局文件
  - 加载布局：重写活动onCreate()方法，关联布局和活动
  - 注册活动：在AndroidManifest.xml中注册活动(自动完成)
  - 设主活动：在AndroidManifest.xml中设置主活动



(1)选择【空活动】



(2)右击”src->java->项目名称”



(3)选择” New→Activity→Empty Activity”

Android四大组件——活动（2）

- 新建活动（续）

  重写活动的onCreate()方法

```
public class FirstActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
    }

}
```

```
public class FirstActivity extends Activity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.first_layout);
    }

}
```

- 调用setContentView()方法加载布局，参数为布局文件的id
- 任何资源都会在R文件中自动生成一个资源id
- R和Android.R不同
- res文件夹下的所有文件不要使用大写字母命名
- 不要试图修改R类中的内容，Android开发工具会自动生成会维护该静态类

Android四大组件——活动（3）

- 新建活动（续）

在AndroidManifest.xml中注册活动

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.activitytest"
    android:versionCode="1"
    android:versionName="1.0" >
    <uses-sdk
        android:minSdkVersion="14"
        android:targetSdkVersion="19" />
    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name=".FirstActivity"
            android:label="This is FirstActivity" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

- 在<application>标签注册活动
- 在<activity>标签完成活动注册
- 在<intent-filter>标签设置主活动和将该活动显示到界面应用列表

1. 你写代码：FirstActivity.java + first_layout.xml
↓
2. 编译阶段：aapt 编译 first_layout.xml → 生成二进制布局 + 更新 R.java
↓
3. DEX编译：javac + dx 把 FirstActivity.java 编译成 .dex 字节码
↓
4. 打包成 APK（签名、对齐）
↓
5. 用户点击桌面图标启动App
↓
6. 系统（ActivityManagerService）创建 FirstActivity 实例
↓
7. 系统调用 onCreate() → 你写了 super.onCreate()（初始化窗口）
↓
8. 你调用 setContentView(R.layout.first_layout)
↓
9. 系统通过 Resources 类加载 R.layout.first_layout 对应的二进制布局文件
↓
10. 系统解析XML，把 Button、TextView 实例化成 View 对象
↓
11. 系统把这些View添加到 DecorView（顶层容器）
↓
12. 屏幕显示你的界面！

Android四大组件——活动（4）

新建活动 class MyActivity extends AppCompatActivity{…}

新建布局 在res中创建布局文件xml，设计活动要显示的界面

重写onCreate setContentView(R.layout.first_layout);

注册活动 在AndroidManifest.xml的<application>标签内<activity>标签注册活动：<activity android:name=".活动名"></activity>

设主活动 在AndroidManifest.xml的<application>标签内<activity>的<intent-filter>设置主活动

启动活动 startActivity(it);//it是一个Intent对象

关闭活动 finish();或Back键

Android四大组件——活动（5）

- 活动的生命周期

  活动是可以堆叠的（隐含时间序）

  - 栈：存储一组活动
    - 入栈：启动活动，startActivity(it)；
    - 出栈：按Back键或者销毁活动，finish()；
  - 任务：管理一组活动，一个任务就是个栈（活动的集合）

Android四大组件——活动（6）

- 活动的生命周期（续）

  活动的生命周期包含4种状态

  - 运行：活动处于栈顶
  - 暂停：活动不在栈顶，但仍可见
  - 停止：活动不在栈顶，且完全不可见
  - 销毁：活动从栈移除

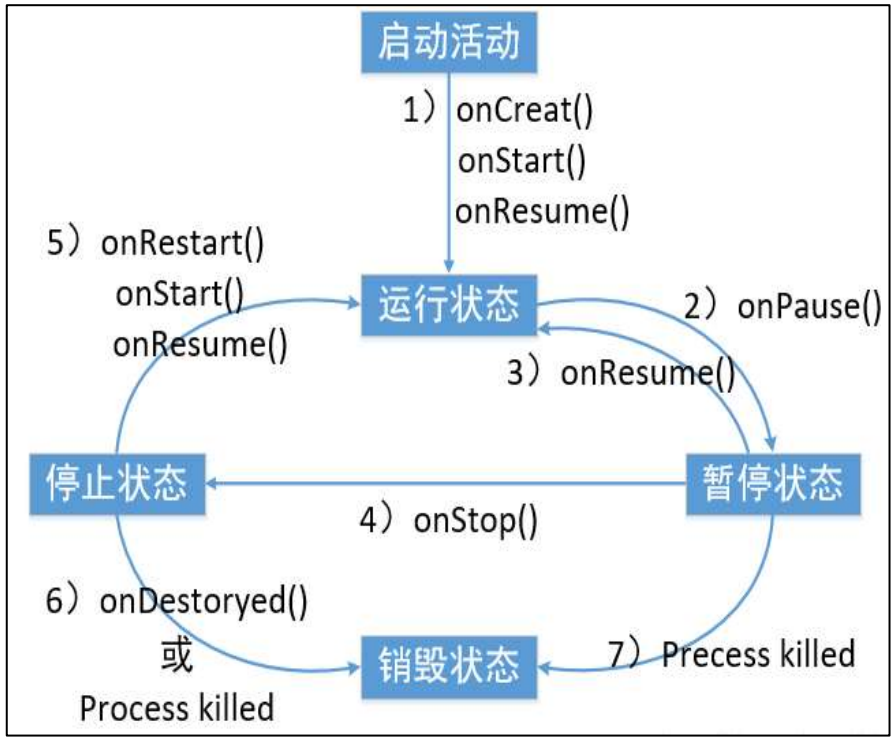
生命周期	被系统回收优先级
运行	1，最不可能被回收
暂停	2，内存极低时被回收
停止	3，内存不足时被回收
销毁	4，最先被回收

Android四大组件——活动（7）

- 活动的生命周期（续）

Activity类定义7个回调方法，覆盖活动生命周期4种状态，回调方法是在事件发生时被自动调用，具体触发事件如下：

- onCreate()：活动创建时
- onStart()：活动由不可见变为可见时
- onResume()：准备好和用户进行交互时
- onPause()：系统准备去启动或恢复另一个活动时
- onStop()：活动完全不可见时被停止时
- onRestart()：活动由停止状态变为运行状态之前
- onDestroy()：活动被销毁之前

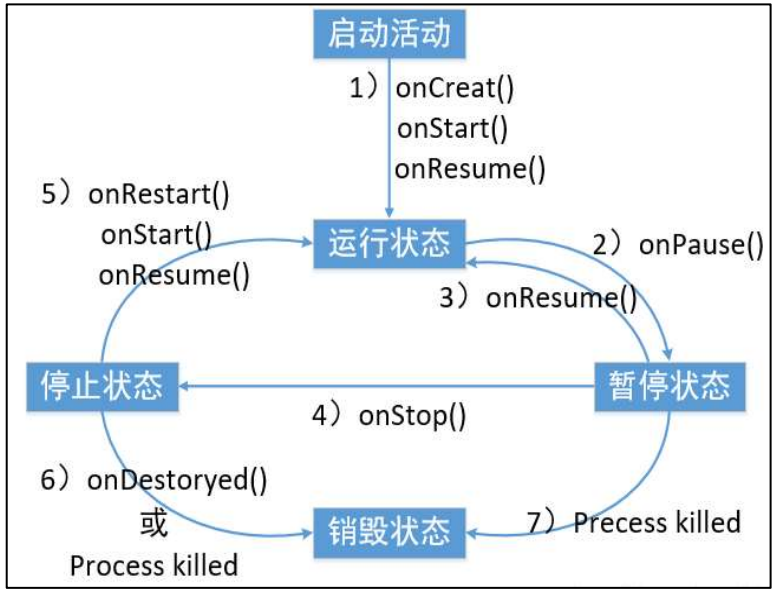


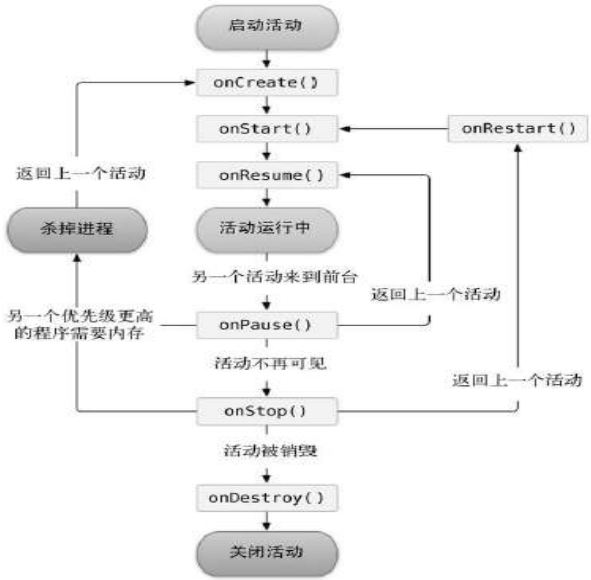
Android四大组件——活动（8）

- 活动的生命周期（续）

活动的3种生存期

- 完整生存期：活动在onCreate()方法和onDestory()方法之间的过程
- 可见生存期：活动在onStart()方法和onStop()方法之间的过程（活动对用户可见，但不一定能交互）
- 前台生存期：活动在onResume()方法和onPause()方法之间的过程（活动既可见，又能与用户交互）





Android四大组件——活动（9）

- 活动的生命周期（续）

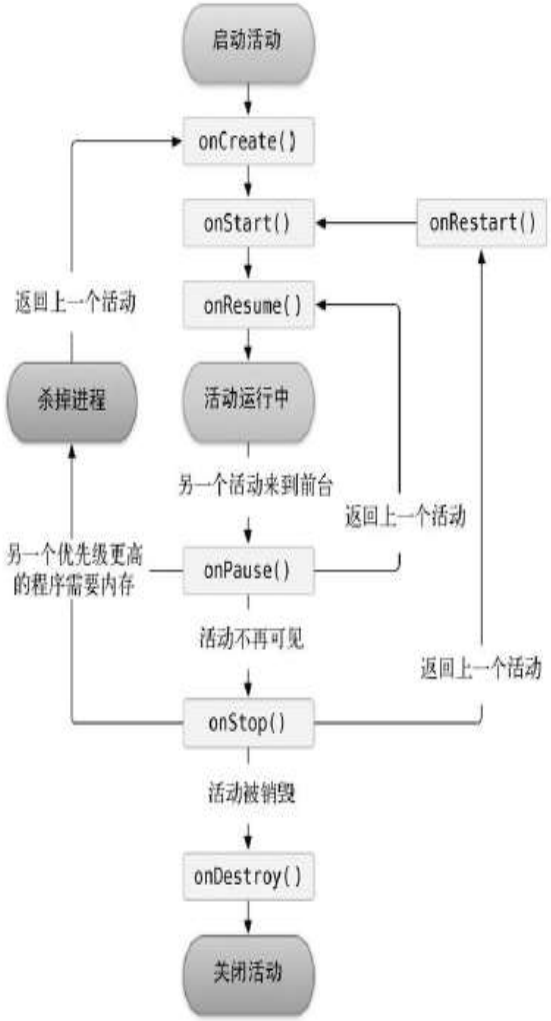
活动的几种场景（类比舞台）

- 创建活动：onCreate()在后台生成，不可见
- 启动活动：onStart()到等候区，可见但不表演
- 恢复活动：onResume()到台中央，可见且开始表演
- 暂停活动：onPause()到等候区，可见但不表演
- 停止活动：onStop()到台下，不可见
- 重启活动：onRestart()从台下重新返回后台，不可见
- 销毁活动：onDestory()活动消亡

➢ 活动要跟用户交互，一定处于栈顶

➢ 每次只能转换一个状态，状态间不能跳转





Android四大组件——活动（10）

- 活动生命周期的场景
  - 场景1：主活动启动子活动1（普通界面）



- 第一次创建主活动时：
  - 依次执行onCreate() 、 onStart() 和onResume()
- 启动活动1时，子活动1会覆盖主活动：
  - 依次执行主活动的onPause()和onStop()
- 在子活动1中按Back键返回主活动
  - 依次执行主活动的onRestart()、 onStart()和onResume()，但不会执行onCreate()

Android四大组件——活动（11）

- 活动生命周期的场景
  - 场景2：主活动启动子活动2（对话框）

```
<activity android:name=".NormalActivity">
</activity>
<activity android:name=".DialogActivity"
    android:theme="@android:style/Theme.Dialog">
</activity>
```



- 第一次创建主活动时：
  - 依次执行onCreate()、onStart()和onResume()
- 启动活动2时，子活动2不会覆盖主活动：
  - 只执行主活动的onPause()，但不执行onStop()
- 在子活动2中按Back键返回主活动
  - 只执行主活动的onResume()

- android:theme=”@android:style/Theme.Dialog” // 设置主题类型
- android:theme=”Theme.Black” // 设置主题属性

Android四大组件——活动（12）

- 活动生命周期的场景
  - 场景3：主活动按Back键退出



- 依次执行主活动的onPause()、onStop()和onDestroy()
- 最终销毁MainActivity

Android四大组件——活动（13）

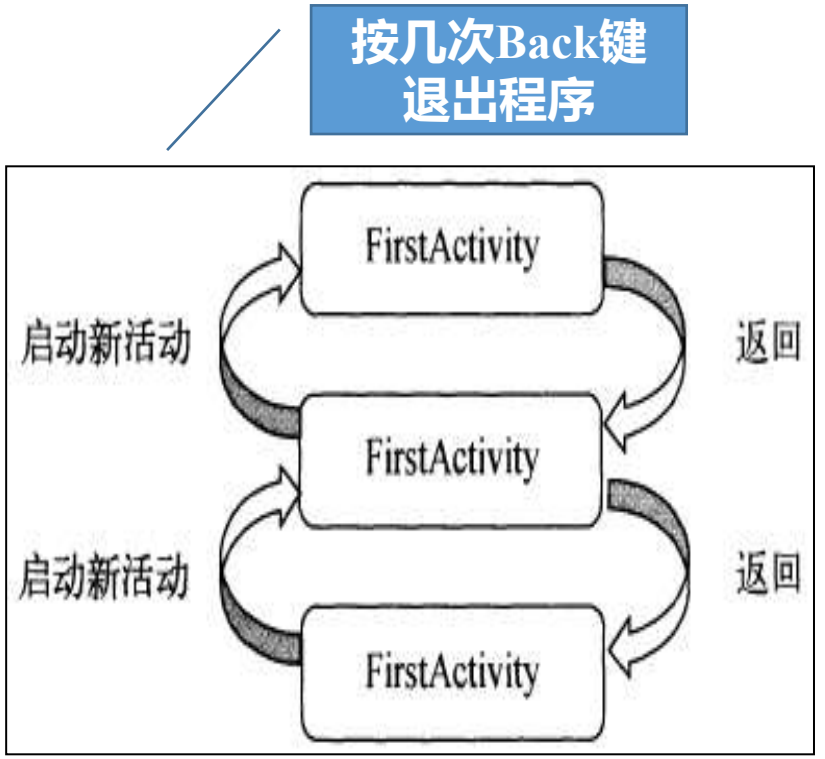
- 活动的启动模式
  - 四种启动模式：standard、singleTop、singleTask、singleInstance
    - standard：默认，每次启动创建该活动一个新实例，无论活动是否已经在栈中
    - singleTop：如果栈顶有该活动就直接使用，则不再创建新实例
    - singleTask：如果栈中有该活动，则将该活动置于栈顶（之上的所有活动全部出栈）
    - singleInstance：用一个新栈管理该活动，实现活动共享
  - 设置启动模式：在AndroidManifest.xml中给<activity>指定android:launchMode属性

```
<activity
    android:name=".FirstActivity"
    android:launchMode="singleTask"
    android:label="This is FirstActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

Android四大组件——活动（14）

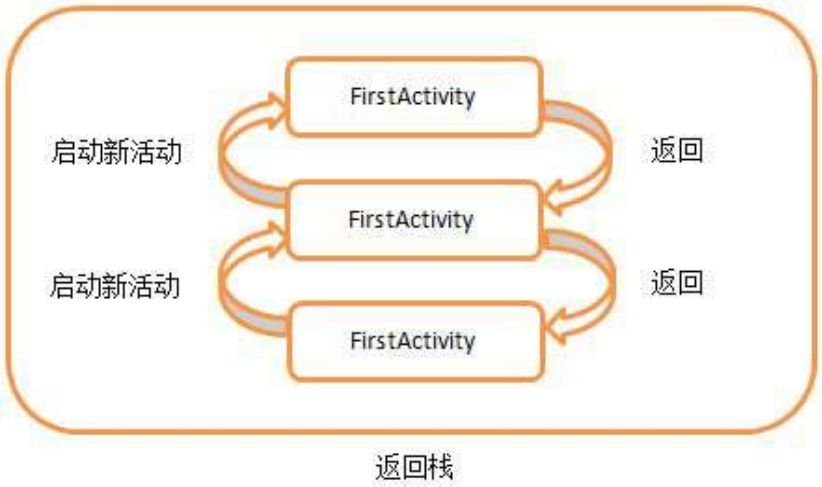
- 活动的启动模式——standard
  - 场景：当前活动自己启动自己
  - 操作：连续点击两次按钮
  - onCreate()调用次数：点击一次，调用一次

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Log.d("FirstActivity", this.toString());
    setContentView(R.layout.first_layout);
    Button button1 = (Button) findViewById(R.id.button_1);
    button1.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            Intent intent = new Intent(FirstActivity.this, FirstActivity.class);
            startActivity(intent);
        }
    });
}
```



Android四大组件——活动（15）

- 活动的启动模式（续）
- standard场景：撰写10封邮件，希望打开10个新界面，需要启动10个独立的新activity



standard生命周期流程

- 第一次进入：onCreate onStart onResume
- 在栈顶再次进入： onCreate onStart onResume
- 不在栈顶再次进入：onCreate onStart onResume

Android四大组件——活动（16）

- 活动的启动模式（续）
- singleTop场景：搜索栏，在搜索结果界面可以直接继续新的搜索，返回一次就能到主界面



情况一：栈顶已经存在



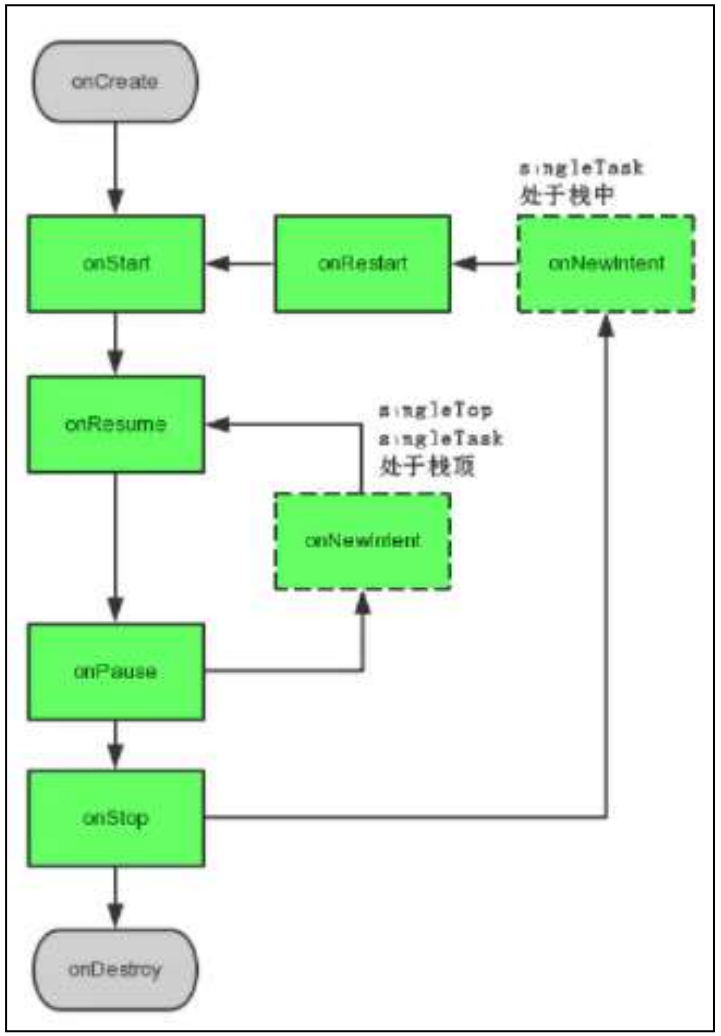
情况二：栈顶不存在

singleTop生命周期流程

- 第一次进入：onCreate onStart onResume
- 在栈顶再次进入：onNewIntent
- 不在栈顶再次进入：onCreate onStart onResume

课外补充

- 理解onNewIntent()
  - 存在问题：通过Intent启动一个活动时，如果栈中已经存在该活动，系统仍会创建一个新的活动实例并显示
  - 解决方法：避免活动被多次实例化，将活动的加载模式设置为单任务模式
  - 到达效果：再次启动活动时，不会调用onCreate()产生新实例，而是调用onNewIntent()并重用栈中该活动的实例
  - 具体实现：在AndroidManifest.xml中将活动launchMode属性设置为singleTask或SingleTop



Android四大组件——活动（17）

- 活动的启动模式（续）
- singleTask场景：微信主页面，只要打开过，就只显示这一个。

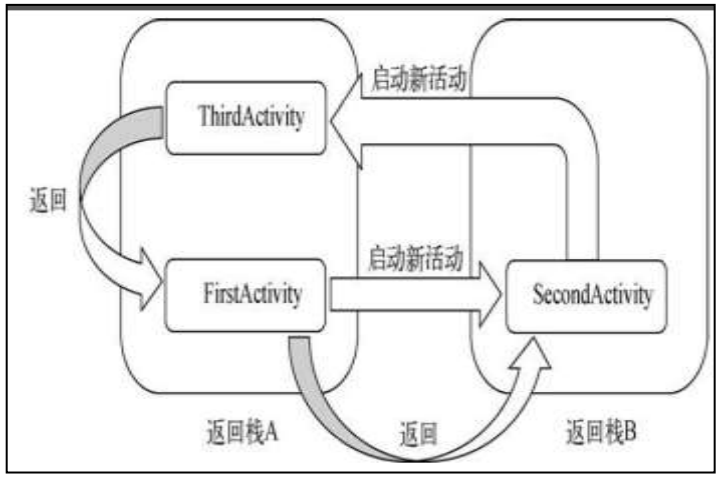


singleTask生命周期流程

- 第一次进入：onCreate onStart onResume
- 在栈顶再次进入：onNewIntent
- 不在栈顶再次进入：onNewIntent onRestart onStart onResume

Android四大组件——活动（18）

- 活动的启动模式（续）
- singleInstance场景：闹铃界面进入设置闹铃界面，按Home返回闹铃界面而非主界面



singleInstanc生命周期流程

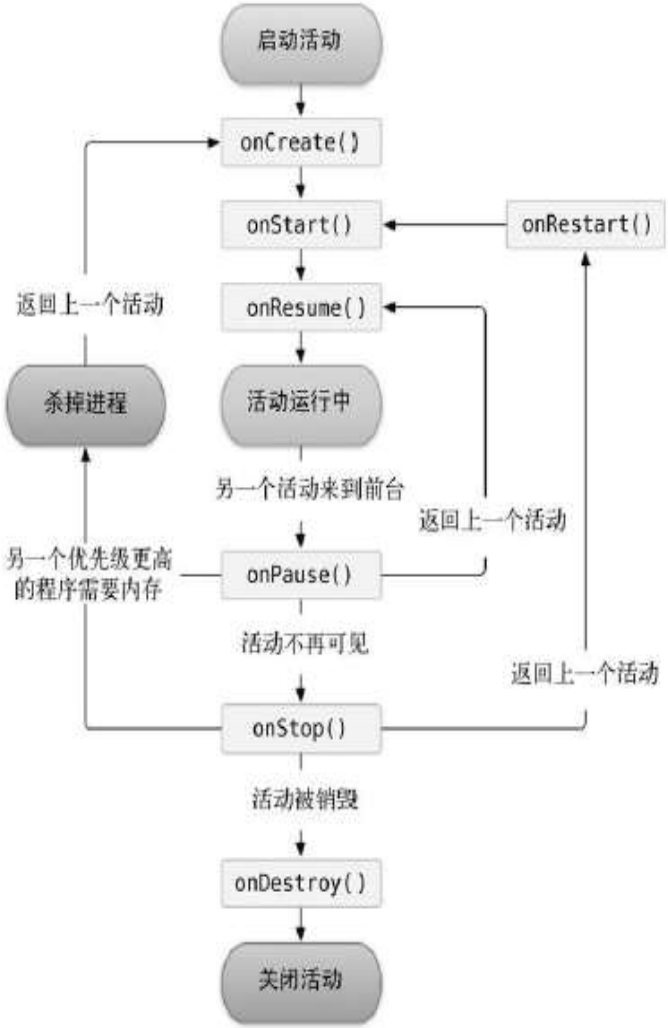
- 活动1启动活动2：1) 先创建栈B，再创建活动2的activity，依次调用onCreate onStart onResume
  2) 创建Intent，实现活动1到活动2跳转
- 活动2启动活动3：1) 在栈A顶部，创建活动3的activity，依次调用onCreate onStart onResume
  2) 创建Intent，实现活动2到活动3跳转
- 动作1：活动3按Back键->活动3被销毁，活动1处于栈A顶部
  活动1按Back键->活动1被销毁，栈A为空，返回栈B
- 动作2：活动2按Back键->活动2被销毁，栈B为空，返回栈A

Android四大组件——活动（19）

- 活动被回收问题

  活动处于停止状态，可能会被系统回收

  - 场景：活动A启动活动B，然后按Back再返回活动A
  - 现象：活动A正常显示，但数据全部丢失
  - 原因：执行onCreate()方法而不是onRestart()方法
  - 解决：使用onSaveInstanceState()回调方法



Android四大组件——活动（20）

- 活动被回收问题（续）
  - 活动回收前保存数据：活动回收前会自动调用onSaveInstanceState()方法
  - 活动回收后恢复数据：系统回收方法前会有通过onSaveInstanceState()方法将数据保存在Bundle参数中

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    String tempData = "Something you just typed";
    outState.putString("data_key", tempData);
}
```

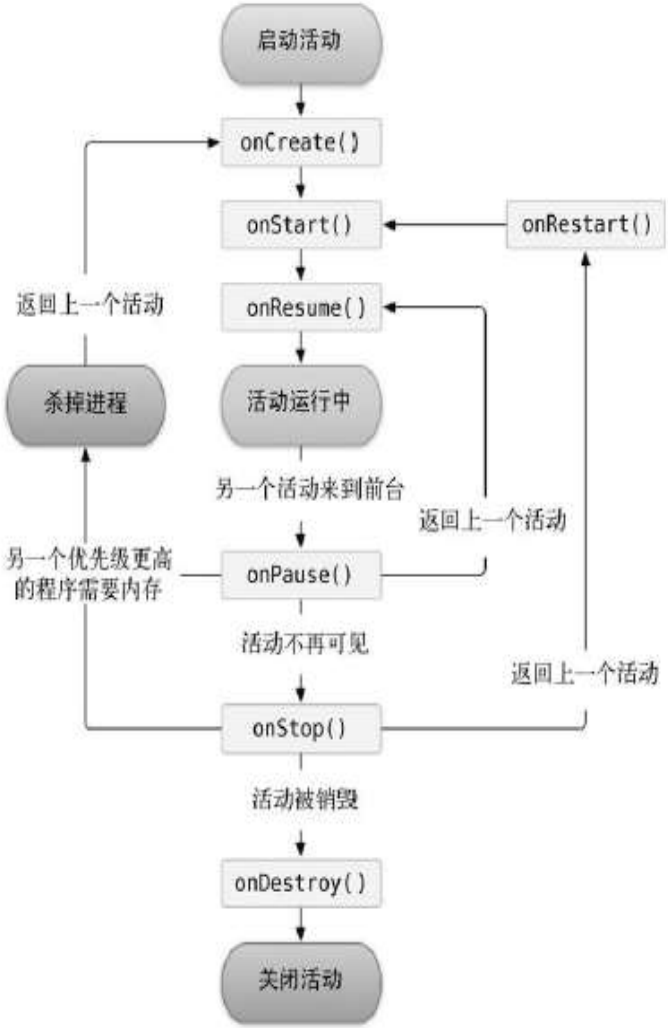
```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Log.d(TAG, "onCreate");
    setContentView(R.layout.activity_main);
    if (savedInstanceState != null) {
        String tempData = savedInstanceState.getString("data_key");
        Log.d(TAG, tempData);
    }
    ...
}
```

Android四大组件——活动（21）

- 活动被回收问题

  活动处于停止状态，可能会被系统回收

  - 场景：活动A启动活动B，然后按Back再返回活动A
  - 现象：活动A正常显示，但数据全部丢失
  - 原因：执行onCreate()方法而不是onRestart()方法
  - 解决：使用onSaveInstanceState()回调方法



Android四大组件——活动（22）

- 活动被回收问题（续）
  - 活动回收前保存数据：活动回收前会自动调用onSaveInstanceState()方法
  - 活动回收后恢复数据：系统回收方法前会有通过onSaveInstanceState()方法将数据保存在Bundle参数中

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    String tempData = "Something you just typed";
    outState.putString("data_key", tempData);
}
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    Log.d(TAG, "onCreate");
    setContentView(R.layout.activity_main);
    if (savedInstanceState != null) {
        String tempData = savedInstanceState.getString("data_key");
        Log.d(TAG, tempData);
    }
    ...
}
```

对比维度	`onSaveInstanceState()`	`onPause()`
触发时机	系统**准备回收**Activity时（或屏幕旋转等配置变更时）	Activity**失去焦点**时（每次都会调用）
是否一定调用	❌ **不保证**（用户主动按返回键退出时不调用）	✅ **一定调用**
适合保存什么	**UI状态**（输入框内容、滚动位置、勾选状态）	**持久化数据**（业务数据、用户信息）
数据存储位置	临时存放在 `Bundle` 中（内存）	写入数据库 / SharedPreferences / 文件（持久存储）
恢复方式	通过 `onCreate()` 或 `onRestoreInstanceState()` 的 `savedInstanceState`	手动从持久存储中读取

Android四大组件——活动（23）

- 思考

1、活动A启动活动B，活动B再启动活动A，分别对standard、singleTop、singleTask

2、活动A启动活动B，活动B再启动活动C，活动B是singleInstance，活动A和C是默认

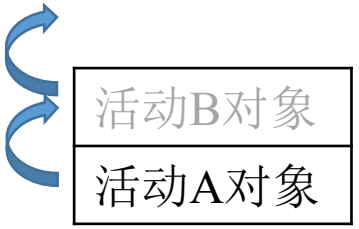
- 栈变化情况
- 退出程序需要按几次Back键



standard栈

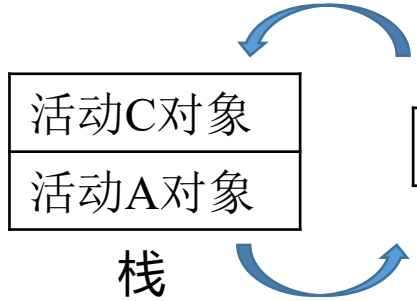


singleTop栈
判断栈顶是否存在需要启动的活动



活动B出栈
重新启动活动A

singleTask栈
判断栈中是否存在需要启动的活动





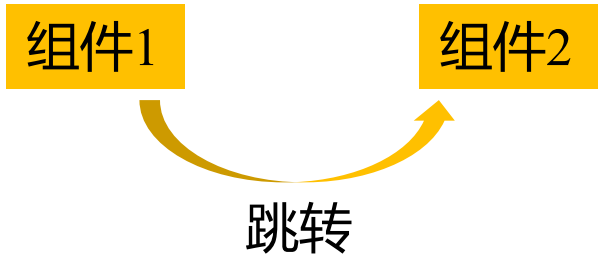
栈

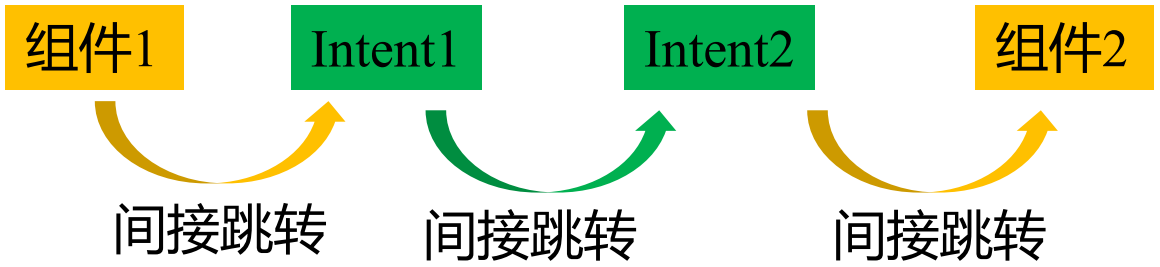
singleInstance栈

问题

- 如何实现两个activity组件间的调转?

  直接跳转 or 间接跳转





Intent作用

- 提供一致的编程模型
- 降低编码耦合度

Android通信桥梁——Intent（1）

- Intent（意图）

  Intent负责Android各组件间跳转和通信

  - 原理：Android的一种消息机制，运行时绑定请求者与实现者。
  - 作用： 在程序内部或者程序之间调用不同的组件，启动活动、服务和发送广播等组件
  - 方式：显式Intent和隐式Intent
    - 显式Intent：意图明显，带两个参数，第二个参数明确要启动的活动
    - 隐式Intent：意图含蓄，可带一个参数或不带参数，由系统分析并确定合适的活动

Intent(Context packageContext, Class **)//显式Intent

- 第一个参数Context：提供一个启动活动的上下文
- 第二个参数Class：指定想要启动的目标活动类

Intent()//无参隐式Intent

Intent(action)//带参隐式Intent，action表示被启动活动具有的动作信息

Android通信桥梁——Intent（2）

- 显式Intent和隐式Intent

```
button1.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(FirstActivity.this, SecondActivity.class);
        startActivity(intent);
    }
});
```

显式Intent

```
button1.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent("com.example.activitytest.ACTION_START");
        intent.addCategory("com.example.activitytest.MY_CATEGORY");
        startActivity(intent);
    }
});
```

隐式Intent

```
<activity android:name=".SecondActivity" >
    <intent-filter>
        <action android:name="com.example.activitytest.ACTION_START" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

隐式Intent

- 在AndroidManifest.xml中通过在<activity> 标签下配置<intent-filter> 内容，可以指定当前活动能够响应的action和category
- 在创建活动java文件中通过Intent构造函数设置指定的action 和category
- 只有action和category都匹配，活动才会响应该Intent

Android通信桥梁——Intent（3）

- 显式Intent和隐式Intent（续）

```
button1.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent("com.example.activitytest.ACTION_START");
        startActivity(intent);
    }
});
```

```
<activity android:name=".SecondActivity" >
    <intent-filter>
        <action android:name="com.example.activitytest.ACTION_START" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

```
<activity android:name=".SecondActivity" >
    <intent-filter>
        <action android:name="com.example.activitytest.ACTION_START" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="com.example.activitytest.MY_CATEGORY"/>
    </intent-filter>
</activity>
```

category.DEFAULT

- 是一种默认category类型
- 不需要手动添加到Intent中
- 调用startActivity方法时自动将它 添加到Intent中

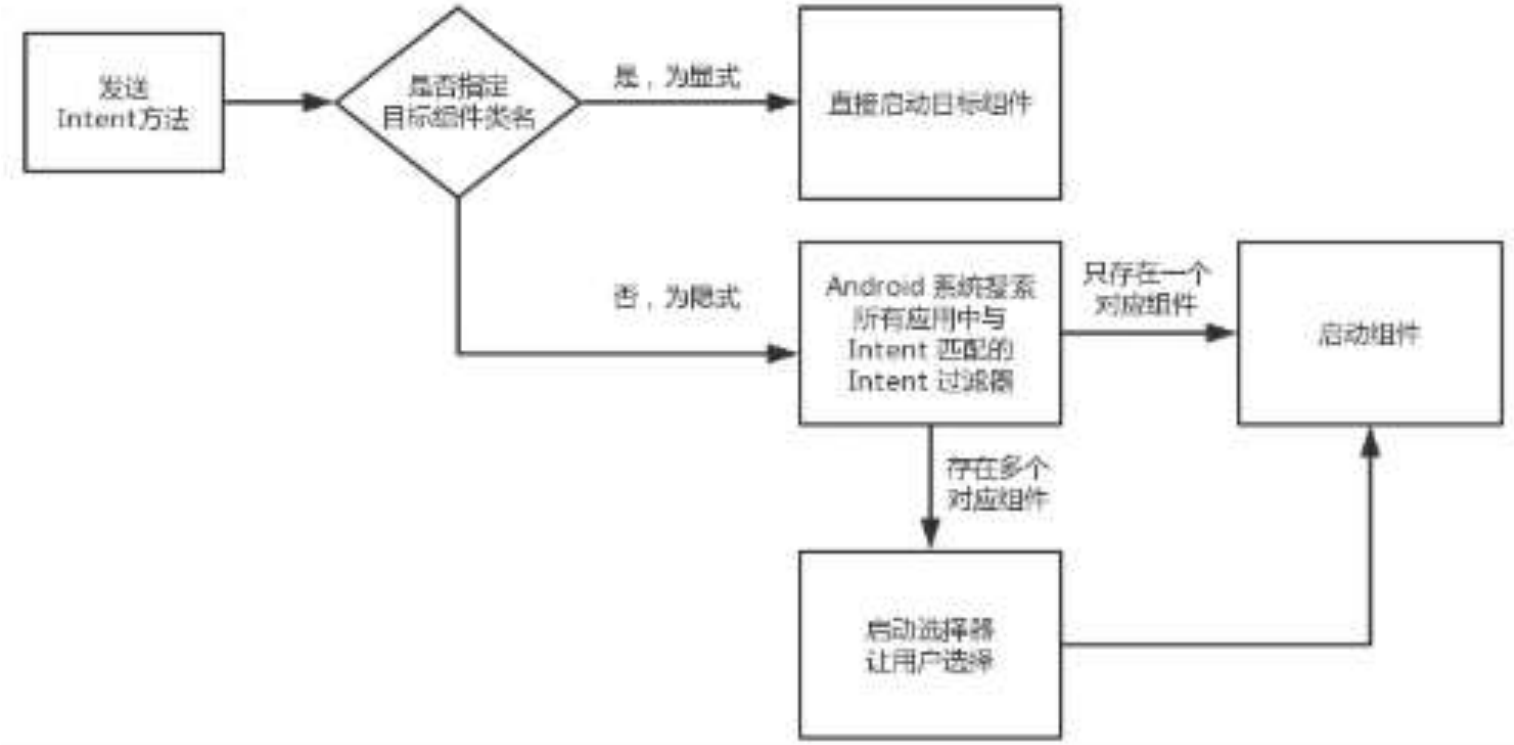
注意：

- 一个活动可以添加多个action
- 一个活动可以添加多个category
- 一个intent只能设置一个action
- 一个intent可以设置多个category
- 隐式Intent需要在AndroidManifest.xml要有与该intent匹配的action和category，否则报错

规则	含义	研发视角
一个 Activity 可以有多个 Action	同一个 Activity 可以响应多种动作	比如一个 Activity 既能查看图片，也能编辑图片
一个 Activity 可以有多个 Category	同一个 Activity 可以属于多个类别	比如既是 `DEFAULT`，也是 `BROWSABLE`
一个 Intent 只能设置一个 Action	Intent 中只能有一个主要动作	可以用 `setAction()` 覆盖之前的 Action
一个 Intent 可以设置多个 Category	Intent 中可以包含多个 Category	用 `addCategory()` 添加，用 `removeCategory()` 移除
隐式 Intent 必须有匹配的 Action 和 Category	否则抛 `ActivityNotFoundException`	所有 Category 必须完全匹配

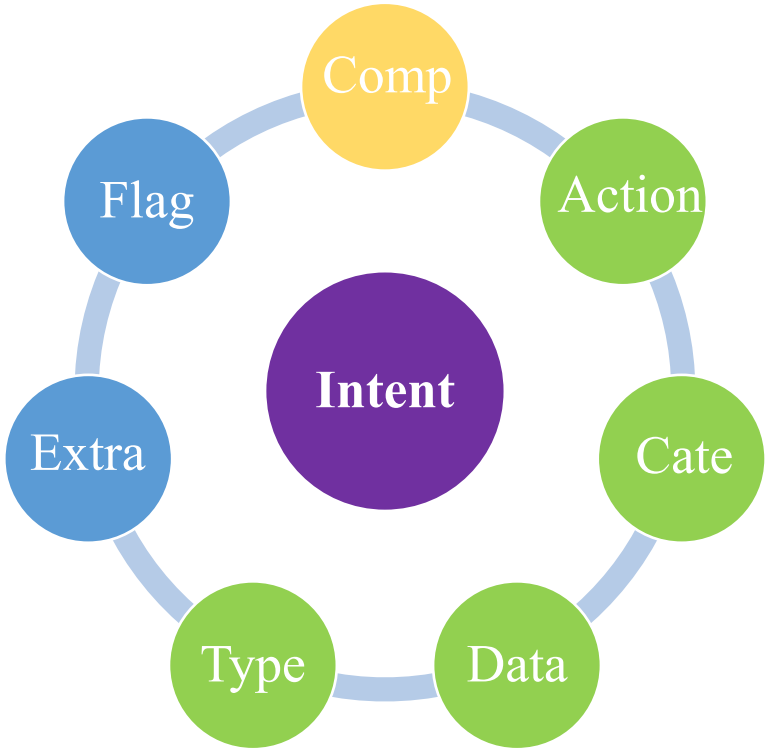
Android通信桥梁——Intent（4）

- Intent工作流程



Android通信桥梁——Intent（5）

- Intent（续）
  - 属性：7种，分别是Component，Action，Category，Data，Type，Extra，Flag



Intent包含的信息

- 显式intent:：必须通过Component指定打开的activity
- 隐式intent：主要通过Action，Category，Data，Type筛选打开的activity
- 附加信息：Extra、Flag

Android通信桥梁——Intent（6）

- Intent的属性
  - Component：目标组件名称，使intent仅传给指定的组件
  - Action：使Intent启动能够响应指定“抽象动作”的组件
  - Category：为Action增加额外的附加类别信息
  - Data：为Action提供待操作数据的路径，即URI
  - Type：为Data所指定的Uri提供对应的数据类型，即MIME
  - Extra：携带的附加数据，实现组件间传递信息
  - Flag：为Intent添加额外的控制旗标，指定Android系统如何启动Activity

Android通信桥梁——Intent（7）

【例】Intent构造示例

```
//显式Intent构造示例
Intent intent = new Intent(this,TagerActivity.class);
```

```
//隐式Intent构造示例
Intent intent=new Intent(Intent.ACTION_VIEW);//设定action为展示内容
intent.setData(Uri.parse("http://www.baidu.com"));//设置data的uri为一个网址
intent.addCategory(Intent.CATEGORY_LAUNCHER);//目标组件为某个应用的首页面
intent.setFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP);//目标Activity的启动方式为single top
intent.putExtra("extra_my","Tom");//附加String数据
```

Android通信桥梁——Intent（8）

【例】Intent隐式应用

```
Intent intent=new Intent(Intent.ACTION_VIEW);
等价
//Intent intent=new Intent();
//intent.setAction ("android.intent.action.View"));
startActivity(intent)
```



Android系统筛选出所有满足Intent过滤器的组件

Android通信桥梁——Intent（9）

【例】点击按钮，返回系统Home桌面

分析：用intent启动满足条件的activity。

```
@Override
public void onClick(View v)
{
      // 创建 Intent 对象
      Intent intent = new Intent();
      // 为 Intent 设置 Action、Category 属性
      intent.setAction(Intent.ACTION_MAIN);
      intent.addCategory(Intent.CATEGORY_HOME);
      startActivity(intent);
}
```

【例】点击按钮，打开指定的网页

```
@Override
public void onClick(View v)
{
      // 创建 Intent
      Intent intent = new Intent();
      String data = "http://www.crazyit.org";
      // 根据指定字符串解析出 Uri 对象
      Uri uri = Uri.parse(data);
      // 为 Intent 设置 Action 属性
      intent.setAction(Intent.ACTION_VIEW);
      // 设置 Data 属性
      intent.setData(uri);
      startActivity(intent);
}
```

Android通信桥梁——Intent（10）

【例】点击按钮，拨打指定电话号码

```
@Override
public void onClick(View v)
{
      // 创建 Intent
      Intent intent = new Intent();
      // 为 Intent 设置 Action 属性（动作为：拨号）
      intent.setAction(Intent.ACTION_DIAL);
      String data = "tel:13800138000";
      // 根据指定字符串解析出 Uri 对象
      Uri uri = Uri.parse(data);
      // 设置 Data 属性
      intent.setData(uri);
      startActivity(intent);
}
```

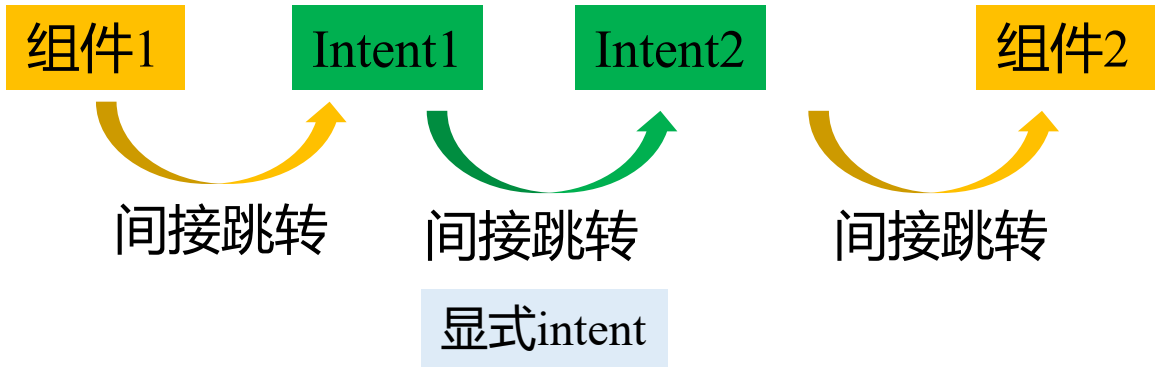
Android通信桥梁——Intent（11）

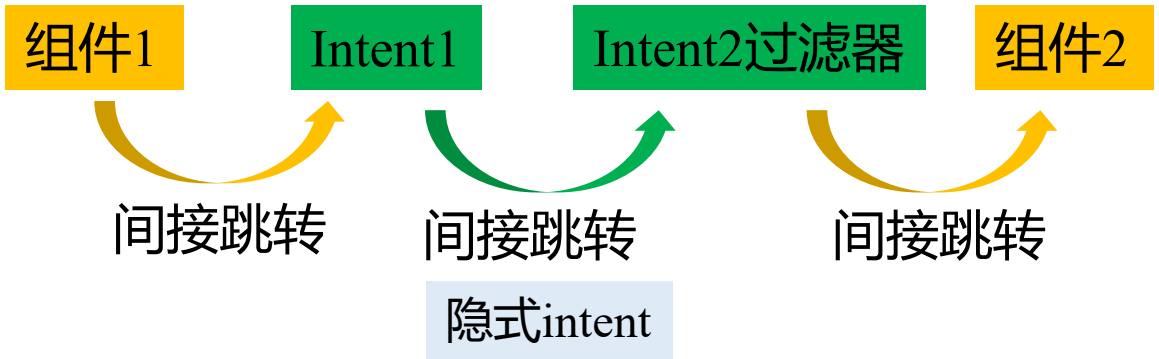
Action 常量	对应字符串	简单说明
ACTION_MAIN	android.intent.action.MAIN	应用程序入口
ACTION_VIEW	android.intent.action.VIEW	显示指定数据
ACTION_ATTACH_DATA	android.intent.action.ATTACH_DATA	指定某块数据将被附加到其他地方
ACTION_EDIT	android.intent.action.EDIT	编辑指定数据
ACTION_PICK	android.intent.action.PICK	从列表中选择某项，并返回所选的数据
ACTION_CHOOSER	android.intent.action.CHOOSER	显示一个 Activity 选择器
ACTION_GET_CONTENT	android.intent.action.GET_CONTENT	让用户选择数据，并返回所选数据
ACTION_DIAL	android.intent.action.DIAL	显示拨号面板
ACTION_CALL	android.intent.action.CALL	直接向指定用户打电话
ACTION_SEND	android.intent.action.SEND	向其他人发送数据
ACTION_SENDTO	android.intent.action.SENDTO	向其他人发送消息
ACTION_ANSWER	android.intent.action.ANSWER	应答电话
ACTION_INSERT	android.intent.action.INSERT	插入数据
ACTION_DELETE	android.intent.action.DELETE	删除数据
ACTION_RUN	android.intent.action.RUN	运行数据
ACTION_SYNC	android.intent.action.SYNC	执行数据同步
ACTION_PICK_ACTIVITY	android.intent.action.PICK_ACTIVITY	用于选择 Activity
ACTION_SEARCH	android.intent.action.SEARCH	执行搜索
ACTION_WEB_SEARCH	android.intent.action.WEB_SEARCH	执行 Web 搜索
ACTION_FACTORY_TEST	android.intent.action.FACTORY_TEST	工厂测试的入口点

Category 常量	对应字符串	简单说明
CATEGORY_DEFAULT	android.intent.category.DEFAULT	默认的 Category
CATEGORY_BROWSABLE	android.intent.category.BROWSABLE	指定该 Activity 能被浏览器安全调用
CATEGORY_TAB	android.intent.category.TAB	指定 Activity 作为 TabActivity 的 Tab 页
CATEGORY_LAUNCHER	android.intent.category. LAUNCHER	Activity 显示顶级程序列表中
CATEGORY_INFO	android.intent.category. INFO	用于提供包信息
CATEGORY_HOME	android.intent.category. HOME	设置该 Activity 随系统启动而运行
CATEGORY_PREFERENCE	android.intent.category.PREFERENCE	该 Activity 是参数面板
CATEGORY_TEST	android.intent.category.TEST	该 Activity 是一个测试
CATEGORY_CAR_DOCK	android.intent.category.CAR_DOCK	指定手机被插入汽车底座（硬件）时运行该 Activity
CATEGORY_DESK_DOCK	android.intent.category.DESK_DOCK	指定手机被插入桌面底座（硬件）时运行该 Activity
CATEGORY_CAR_MODE	android.intent.category.CAR_MODE	设置该 Activity 可在车载环境下使用

Android通信桥梁——Intent（12）

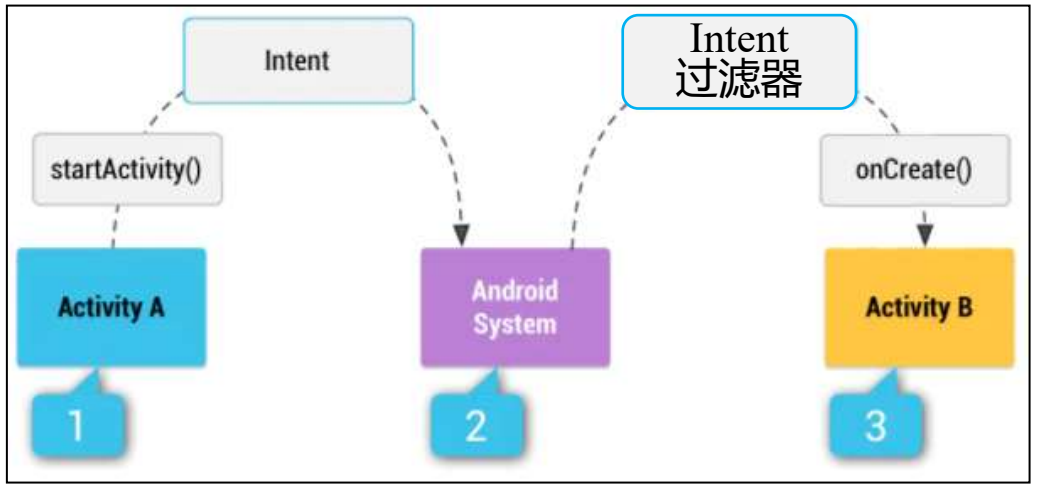
- Intent机制理解
  - 充当中转媒介
  - 调用者只关注提意图
  - 实现调用者和响应者的解耦
  - 统一接口





Android通信桥梁——Intent（13）

- Intent和Intent过滤器
  - Intent针对请求者，用于描述“我的需求”
  - Intent过滤器针对实现者，用于描述“让系统筛选谁能满足我的需求”
  - 两者匹配，则启动相应的组件（准确地说是响应相应的Intent）



隐式Intent通过系统传递以启动其他Activity流程：
步骤1： 请求方A创建包含条件描述Intent，并将其传递给 startActivity()；
步骤2：Android 系统搜索所有应用中与 请求方A中Intent匹配的Intent过滤器；
步骤3：找到匹配项之后，系统通过调用匹配 的实现方（Activity B）的 onCreate() 方法，并将其传递给 Intent，以此启动匹配 Activity。

Android通信桥梁——Intent（14）

- Activity组件响应隐式 Intent需要注意

1、必须在AndroidManifest.xml中的Intent过滤器<intent-filter>…</intent-filter>中添加action和category属性（category.DEFAULT不可缺省）

2、Intent中一定要有action和category属性（category.DEFAULT可缺省）

思考

- 对于组件：
  一个组件可以建立几个Intent
- 对于组件的Intent：
  一个Intent可以设置一个action，多个category、多个data…
- 对于组件的Intent过滤器：
  一个Intent过滤器可以匹配多个action、多个category、多个data

Android通信桥梁——Intent（15）

```
<activity android:name="MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity
```

```
<activity android:name="ShareActivity">
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <data android:mimeType="text/plain"/>
    </intent-filter>
    <intent-filter>
        <action android:name="android.intent.action.SEND"/>
        <action android:name="android.intent.action.SEND_MULTIPLE"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <data android:mimeType="application/vnd.google.panorama360+jpg"/>
        <data android:mimeType="image/*"/>
        <data android:mimeType="video/*"/>
    </intent-filter>
</activity>
```

Android通信桥梁——Intent（16）

- 隐式Intent匹配规则
  - 隐式 Intent会与所有设备所有应用组件的 Intent 过滤器进行比对
  - 一个组件可以设置多个Intent过滤器，某个Intent对象只需与其中任何一个匹配即可启动组件
  - 系统会根据Intent与Intent过滤器的action、category、data进行三次比对，全部通过才能匹配
  - 被调活动Intent过滤器中各元素必须是主调活动Intent对象中各个元素的子集
  - 主调活动Intent对象：一个action、多个category、多个data
  - 被调活动Intent过滤器：多个action，多个category、多个data

Android通信桥梁——Intent（17）

- Intent过滤器匹配规则
  - Action匹配规则
    - Intent 只能包含一项 action，而 Intent 过滤器可以包含多项 action
    - 只要 Intent 的 action 可以在 Intent 过滤器中找到对应项即可通过测试
    - 当 Intent 过滤器未声明任何 action 时，任何 Intent 都不能通过匹配
    - 当 Intent 未指定 Action 时，无法通过匹配
  - Category匹配规则
    - 只有Intent中的所有category是Activity中Intent过滤器中category的子集，才算匹配成功
    - 当目标组件为 Activity并且响应隐式 Intent时，必须添加 "android.intent.category.DEFAULT" 到 Intent 过滤器中
    - 当目标组件为广播时，Intent 和 Intent 过滤器都不设置 category，可通过匹配。

Android通信桥梁——Intent（18）

- Intent传递数据
  - 一次传单个数据：putExtra(“键”，值)
  - 一次传一组数据：先存入Bundle 对象，再putExtra(Bundle对象)



Android通信桥梁——Intent（19）

```
//添加 Extra 示例
Intent intent = new Intent(this,TagerActivity.class);
intent.putExtra("extra_int",1000);
int[] ints={1,2,3};
intent.putExtra("extra_int_array",ints);
intent.putExtra("extra_string","teger Activity");
startActivity(intent);
```

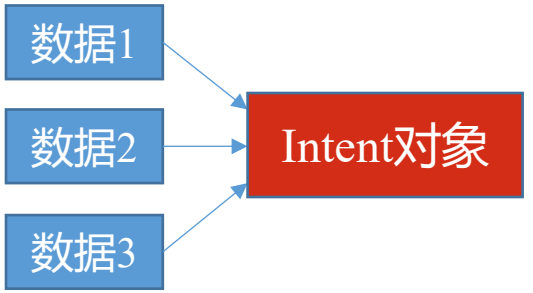
```
//对应取出数据示例
Intent intent = getIntent();
int i=intent.getIntExtra("extra_int",0);
int[] ints=intent.getIntArrayExtra("extra_int_array");
String a = intent.getStringExtra("extra_string");
```

Android通信桥梁——Intent（20）

- Intent一次传递单个数据动作分解

  上一个活动（主）向下一个活动（子）传递数据

  - 一次传递一个数据：将数据直接嵌入Intent对象中发送
    - 主活动启动子活动：new Intent()
    - 主活动存储一个数据：Intent的putExtra()方法
    - 主活动发送Intent对象：startActivity()
    - 子活动获取Intent对象：getIntent()
    - 子活动接收一个数据：Intent的getStringExtra()方法（或getIntExtra() 或getBooleanExtra()）



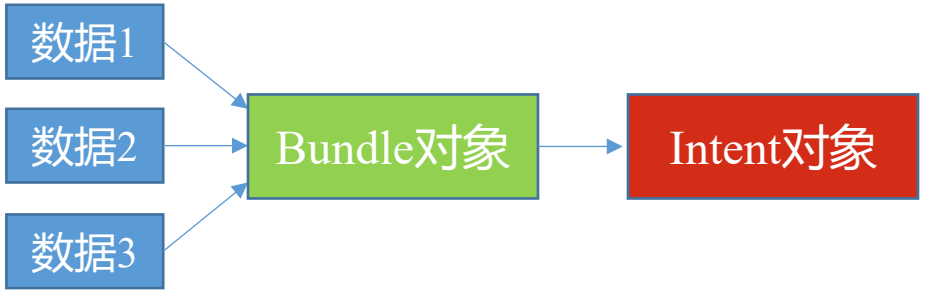

```
button1.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        String data = "Hello SecondActivity";
        Intent intent = new Intent(FirstActivity.this, SecondActivity.class);
        intent.putExtra("extra_data", data);
        startActivity(intent);
    }
});
```

```
public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.second_layout);
        Intent intent = getIntent();
        String data = intent.getStringExtra("extra_data");
        Log.d("SecondActivity", data);
    }

}
```

Android通信桥梁——Intent（21）



- Intent传递数据（续）

  上一个活动（主）向下一个活动（子）传递数据

  - 一次传递多个数据：将数据间接存入Bundle对象，再用Intent发送Bundle对象
    - 主活动启动子活动：new Intent()
    - 主活动存储多个数据：Bundle的putInt()方法（或putString()等）
    - 主活动嵌入Bundle对象：Intent的putExtras()方法
    - 主活动发送Bundle对象：startActivity()
    - 子活动获取Intent对象：getIntent()
    - 子活动获取Bundle对象：Intent的getExtras()方法
    - 子活动接收多个数据：Bundle的getInt()方法（或getString()等）

```
Intent it1 =  new Intent (A.this, B.class);
Bundle bd = new Bundle();
bd.putInte("num",1);
bd.putSting("detail","hello");
it1.putEntras(bd);
startActivity(it1);
```

```
Intent it2 = getIntent();
Bundle bd = it2.getExtras();
int n = bd.getInt("num");
String d = bd.getString("detail");
```

注意：使用Bundle传递数据的大小<1MB，否则会报TransactionTooLargeException异常

Android通信桥梁——Intent（22）

- Intent回传数据（续）

  下一个活动（子）向上一个活动（主）传递数据

  - 主活动启动子活动：new Intent()
  - 主活动发动带唯一请求码的Intent对象：startActivityForResult()回调方法
  - 子活动存储一个数据：Intent的putExtra()方法
  - 子活动发送带结果状态的Intent对象：setResult()方法
  - 主活动接收带结果状态和请求码的Intent对象：onActivityResult()方法

子活动向主活动传递数据

- 主活动不发送数据，只发送请求码，用于接收时判断数据源
- 主活动使用startActivityForResult()可以使在子活动销毁时回调主活动的onActivityResult()方法
- onActivityResult()方法接收子活动返回三个参数：请求码、结果状态、传递数据

```
button1.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent intent = new Intent(FirstActivity.this, SecondActivity.class);
        startActivityForResult(intent, 1);
    }
});
```

主活动

```
public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.second_layout);
        Button button2 = (Button) findViewById(R.id.button_2);
        button2.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent();
                intent.putExtra("data_return", "Hello FirstActivity");
                setResult(RESULT_OK, intent);
                finish();
            }
        });
    }
}
```

子活动

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    switch (requestCode) {
        case 1:
            if (resultCode == RESULT_OK) {
                String returnedData = data.getStringExtra("data_return");
                Log.d("FirstActivity", returnedData);
            }
            break;
        default:
    }
}
```

主活动

Android通信桥梁——Intent（23）

```
//原Activity中启动新Activity并请求返回数据
Intent intent = new Intent(this,TagerActivity.class);
startActivityForResult(intent,1);
```

```
//新Activity中设定返回Intent并销毁，销毁后会回调到原 Activity 的 onActivityResult()方法
Intent intent=new Intent();
intent.putExtra("extra_boolean",true);
setResult(RESULT_OK,intent); finish();
```

```
//原 Activity中重写onActivityResult()方法
@Override protected void onActivityResult(int requestCode, int resultCode, Intent data){
    switch (requestCode){
        case 1:
            if (resultCode==RESULT_OK){
                boolean b=data.getBooleanExtra("extra_boolean",false);
            }
            break;
       default:
       }
}
```

Android通信桥梁——Intent（24）

- Intent将多个数据传递给多个页面

```
//页面A发送多个数据给页面B
Intent intent=new Intent(MainActivity.this,BActivity.class);
Intent.putExtra("String","MainActivity中的值");
intent.putExtra("int",11); startActivity(intent);
```

```
//页面B接收多个数据
Intent intent = getIntent();
string = intent.getStringExtra("String");
key = intent.getIntExtra("int",0);
```

```
//页面B继续将多个数据发送给页面C
Intent intent=new Intent(BActivity.this,CActivity.class);
intent.putExtra("String1",string);
intent.putExtra("int1",key);
intent.putExtra("boolean",true);
startActivity(intent);
```

Android通信桥梁——Intent（25）

- Bindle和Intent将多个数据传递给多个页面

```
//页面A发送多个数据给页面B
 Intent intent = new Intent(MainActivity.this, BActivity.class);
Bundle bundle = new Bundle();
bundle.putString("String","MainActivity中的值");
bundle.putInt("int",11);
intent.putExtra("bundle",bundle);
startActivity(intent);
```

```
//页面B接收多个数据
Intent intent = getIntent();
bundle=intent.getBundleExtra("bundle");
```

```
//页面B继续将多个数据发送给页面C
 Intent intent=new Intent(BActivity.this,CActivity.class);
//可以传给CActivity额外的值
bundle.putBoolean("boolean",true);
intent.putExtra("bundle1",bundle);
startActivity(intent);
```

扩展内容

- Intent解析机制Intent解析机制主要是通过查找已注册在AndroidManifest.xml中的所有<intent-filter>及其中定义的Intent，通过PackageManager来查找能处理这个Intent的组件
- 解析中，Android是通过Intent的action、type、category等属性来进行判断，方法如下：
  - ✓ 如果Intent指明action，则目标组件IntentFilter的action列表中就必须包含有这个action，否则不能匹配
  - ✓ 如果Intent没有提供type，系统将从data中得到数据类型。和action一样，目标组件的数据类型列表中必须包含Intent的数据类型，否则不能匹配
  - ✓ 如果Intent中的数据不是content:类型的URI，而且Intent也没有明确指定type，将根据Intent中数据的scheme（比如 http:或者mailto:）进行匹配。同上，Intent 的scheme必须出现在目标组件scheme列表
  - ✓ 如果Intent指定了一个或多个category，这些类别必须全部出现在组建的类别列表中。比如Intent中包含了两个类别：LAUNCHER_CATEGORY和ALTERNATIVE_CATEGORY，解析得到的目标组件必须至少包含这两个类别。

Android——活动（12）

- 常见活动

利用通用资源标志符URI（Universal Resource Identifier）表示数据（图像、字符串、视频、网页等）

- 拨打电话

```
Uri uri = Uri.parse("tel:10086"); // 给移动客服10086拨打电话
Intent intent = new Intent(Intent.ACTION_DIAL, uri);
startActivity(intent);
```

- 发送短信

```
Uri uri = Uri.parse("smsto:10086"); // 给10086发送内容为 “Hello”的短信
Intent intent = new Intent(Intent.ACTION_SENDTO, uri);
intent.putExtra("sms_body", "Hello");
startActivity(intent);
```

Android四大组件——活动（13）

- 常见活动（续）

利用通用资源标志符Uri（Universal Resource Identifier）表示数据（图像、字符串、视频、网页等）

- 打开浏览器

```
Uri uri = Uri.parse(“http://www.baidu.com”); // 打开百度主页
Intent intent = new Intent(Intent.ACTION_VIEW, uri);
startActivity(intent);
```

- 多媒体播放

```
Intent intent = new Intent(Intent.ACTION_VIEW);
Uri uri = Uri.parse("file:///sdcard/foo.mp3");
intent.setDataAndType(uri, "audio/mp3");
startActivity(intent);
```

思考题

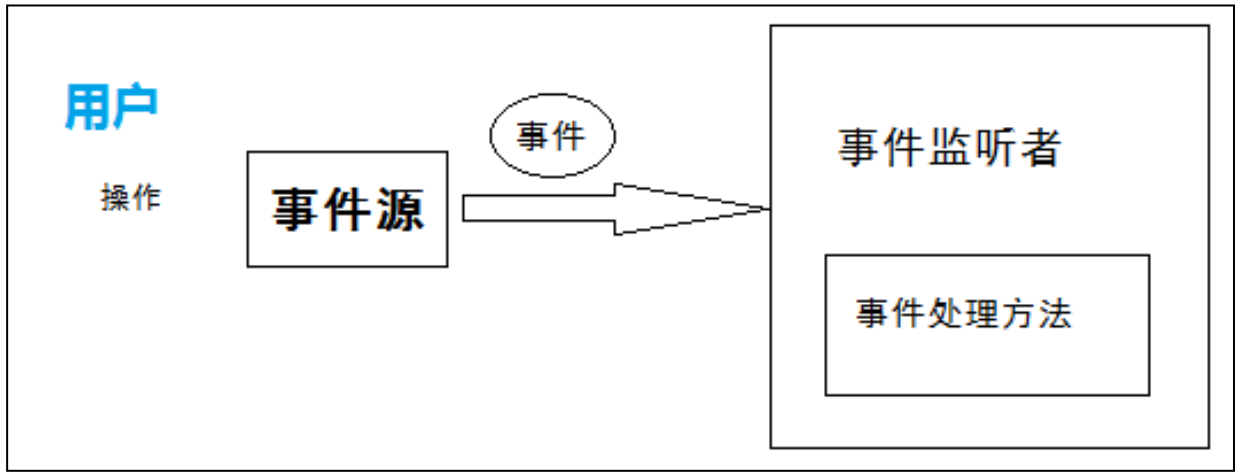
- **onStop() 的延迟调用**
- 主活动.onPause() → 子活动1.onCreate() → 子活动1.onStart() → 子活动1.onResume() → 主活动.onStop()
-

Android事件处理机制

- 事件处理机制

  事件处理机制包括事件源、事件和事件处理三个要素。

  - 事件源：发生事件的组件
  - 事件：封装界面组件发生的特定事件，即可以被识别的操作
  - 事件处理：响应并完成具体的事件



Android事件处理机制

- Android两种事件处理机制
  - 基于监听的事件处理机制：为组件绑定特定的事件监听器
  - 基于回调的事件处理机制：重写组件中特定的响应事件的回调方法

事件监听和事件回调
- 事件回调：处理通用性事件，如按下键盘、弹起键盘、按下组件
- 事件监听：处理特殊性事件，如控制光标四个方向移动

基于监听的事件处理机制

- 基于监听的事件处理机制

  事件监听处理机制包括事件源、事件、事件监听器三个要素

  - 事件源：发生事件的组件
  - 事件：封装界面组件发生的特定事件信息，即可以被识别的操作，由系统自动产生
  - 事件监听器：监听事件源发生的事件，调用相应的事件处理器
    - 事件处理器：响应并完成相应的事件，即事件监听器中的实现方法
    - 监听器用于对特定事件进行监听。一旦监听到特定事件，则由监听器截获该事件，并回调自身的特定方法对事件进行处理。

基于监听的事件类型

- View 组件将截获的事件分为 6 种，对应以下 6 种事件监听器接口
  - OnClickListener 接口此接口处理的是单击事件，例如，在View上进行单击动作，在 View 获得焦点的情况下单击“确定”按钮或者单击轨迹球都会触发该事件。当单击事件发生时，OnClickListener 接口会回调 public void onClick(View v) 方法对事件进行处理。其中参数 v 指的是发生单击事件的 View 组件。
  - OnLongClickListener 接口此接口处理的是长按事件，当长时间按住某个 View 组件时触发该事件。其对应的回调方法为 public boolean onLongClick(View v)，当返回 true 时，表示已经处理完此事件，若事件未处理完，则返回 false，该事件还可以继续被其他监听器捕获并处理。
  - OnFocusChangeListener 接口此接口用于处理 View 组件焦点改变事件。当 View 组件失去或获得焦点时会触发该事件。其对应的回调方法为 public void onFocusChange(View v, Boolean hasFocus)，其中参数 v 表示产生事件的事件源，hasFocus 表示事件源的状态，即是否获得焦点

基于监听的事件类型

- OnKeyListener 接口此接口用于对手机键盘事件进行监听，当View获得焦点并且键盘被敲击时会触发该事件。其对应的回调方法为 public boolean onKey(View v, int keyCode, KeyEvent event)。方法参数说明：keyCode 为键盘码。event 为键盘事件封装类的对象。
- OnTouchListener 接口此接口用来处理手机屏幕事件，当在 View 的范围内有触摸、按下、抬起、滑动等动作时都会触发该事件，并触发该接口中的回调方法。其对应的回调方法为 public boolean onTouch(View v, MotionEvent event)，对应的参数同上。
- OnCreateContextMenuListener 接口此接口用于处理上下文菜单被创建的事件。其对应的回调方法为 public void onCreateContextMenu(ContextMenu menu, View v, ContextMenuInfo info)，其中参数 menu 为事件的上下文菜单，参数 info 是该对象中封装了有关上下文菜单的其他信息。

基于监听的事件处理方法

- 五种响应事件监听的方法

btn = (Button) findViewById(R.id.btnshow);

- **第一种：内部类响应事件**

```
//内部类处理事件响应
btn_login.setOnClickListener(new MyListener());
```

```
//内部类作为事件监听器
class MyListener implements OnClickListener{
    @Overridepublic void onClick(View v){
        ****//响应事件
    }
}
```

基于监听的事件处理方法

- 五种响应事件监听的方法
  - **第二种：匿名类响应事件**

```
//匿名类同时作为事件监听器和处理事件响应
btn.setOnClickListener(new OnClickListener(){
    public void onClick(View v){
        ****//响应事件
}
```

基于监听的事件处理方法

- 五种响应事件监听的方法
  - **第三种：在布局文件中设置UI标签的android:onClick()方法**

```
//在标签上直接绑定事件处理方法
<Buttion
    android:layout_width = “wrap_content”
    android:layout_height = “wrap_content”
    android:onClick = “btnClick”
```

```
//在Activity代码中定义一个事件处理方法
public void btnClick(View view){
    *****//响应事件
}
```

直接将监听绑定到组件的说明

- Android标签基本都onClick方法和onLongClick方法
- 在代码中定义的监听事件处理方法名称必须和android:onClick设置的监听事件处理方法名称一致，并且必须接受一个View类型参数，该方法需要在Activity中实现

基于监听的事件处理方法

- 五种响应事件监听的方法
  - **第四种：外部类响应事件**

```
//预先定义外部类MyExternListener.java
public class MyExternListener implements OnClickListener{
    public MyExternListener(Activity activity, EditText username,EditText passwd{
        this.activity = activity;
        this.username = username;
        this.passwd = passwd;
}
    public void onClick(View v){
         *****//响应事件
}//通过事件监听器的成员变量及重写构造方法，事件源可以传递信息给事件监听器
```

外部类实现监听的优劣？

```
//在Activity代码中调用外部类的事件响应方法
import com.MyExternListener
btn.setOnClickListener(newMyExternListener(MainActivity.this,et_username,et_passwd));
```

基于监听的事件处理方法

- 五种响应事件监听的方法
  - **第五种：Activity自身类响应事件**

```
//Activity直接继承监听器接口
public class MyExternListener extends Activity implements OnClickListener{
    btn.setOnClickListener(this);
    @override
    public void onClick(View v){
        *****//响应事件
}
```

Activity自身响应事件的说明

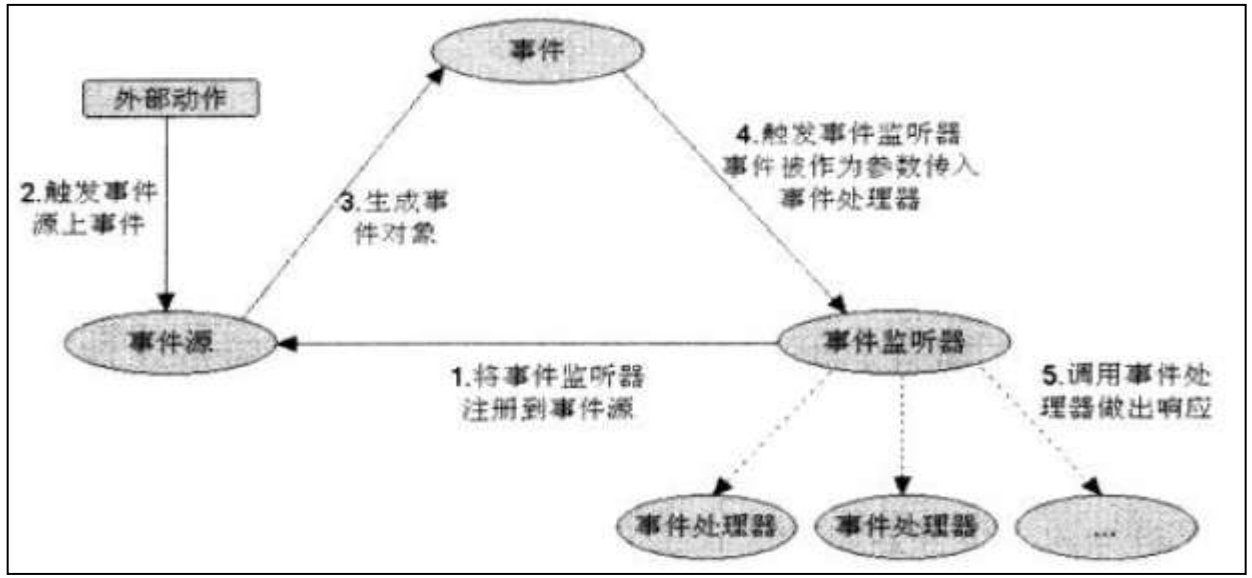
- 监听器设置为this代表的Activity自身类
- 自身类实现事件监听接口
- 自身类重写接口的方法

基于监听的事件处理方法

- 五种响应事件监听的方法
  - 通过设置UI组件的android:onClick属性，然后代码实现方法。这种方法业务逻辑和UI耦合性太高，实际业务一般不用
  - 匿名类：一般只应用特定组件的特定业务响应内部类（只用一次）
  - 外部类：可作为多个UI共同的事件处理，适用多个UI组件复用；当事件源要传递信息给事件监听器时则不太灵活，必须通过成员变量及构造方法
  - Activity自身类：处理比较灵活，实际中用的最多

基于监听的事件处理机制

- 基于监听的事件处理机制（续）



监听的两个顺序

- 一个事件源绑定多个监听器：按事件源(组件)响应的顺序决定监听器的执行顺序
- 一个监听器具有多个事件处理器：按事件中自子事件发生的逻辑顺序决定事件处理器的执行顺序

事件监听机制

- 委派式：事件源事先将整个事件处理委托给事件监听器
- 一个事件可以绑定多个事件监听器
- 一个事件监听器处理同一类型的事件
- 一个事件可以分解为多个子事件，为此一个事件监听器可以添加多个事件处理器响应时，同一事件监听器中各个事件处理器都会被触发，触发顺序（即子事件的执行顺序）是按照事件发生的逻辑顺序由系统确定的
- 事件处理器就是监听器中对某类事件处理的特定方法

基于监听的事件处理机制

- 事件监听机制实现方法

  方法一：一个监听器去处理一类事件（set**Listener）

  方法二：一组监听器去处理一类事件（add**Listener）

```
//一个监听器去处理一类事件
ViewPager.addOnPageChangeListener(new ViewPager.OnPageChangeListener(){
        @Override
        public void onPageScrolled(int position, float positionOffset, int positionOffsetPixels){
                Log.i(TAG, “onPageScrolled:显示时间”);
        }
        @Override
        public void onPageSelected(int position){
                Log.i(TAG, “onPageSelected:记录用户行为”);
        }
        @Override public void onPageScrollStateChanged(int state){
                Log.i(TAG, “onPageScrollStateChanged:显示当前访问人数”);
        }
})
```

- set**Listener只能设置一个监听，之后调用该方法会覆盖之前设置的监听
- add**Listener可设置多个监听，不会被覆盖

```
//一组监听器去处理一类事件
ViewPager.addOnPageChangeListener(new ViewPager.OnPageChangeListener(){
        @Override
        public void onPageScrolled(int position, float positionOffset, int positionOffsetPixels){
                Log.i(TAG, “onPageScrolled:显示时间”);
        }
        @Override
        public void onPageSelected(int position){
                Log.i(TAG, ”onPageSelected:记录用户行为”);
        }
        @Override public void onPageScrollStateChanged(int state){
                Log.i(TAG, “onPageScrollStateChanged:显示当前访问人数”);
        }
})
ViewPager.addOnPageChangeListener(new ViewPager.OnPageChangeListener(){
        @Override
        public void onPageScrolled(int position, float positionOffset, int positionOffsetPixels){
                Log.i(TAG, “onPageScrolled:显示排名”);
        }
        @Override
        public void onPageSelected(int position){
                Log.i(TAG, “onPageSelected:统计分数”);
        }
        @Override public void onPageScrollStateChanged(int state){
                Log.i(TAG, “onPageScrollStateChanged:显示用户名”);
        }
});
```

输出日志是什么?

【例】手指翻页事件的输出日志内容

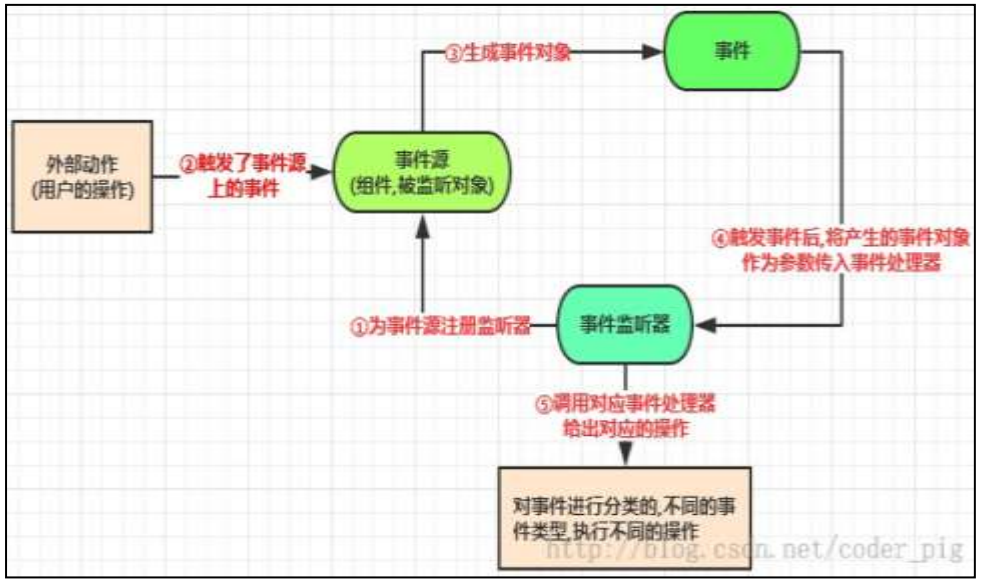
- onPageScrollStateChanged：手指操作屏幕使其发生变化时被调用
- onPageScrolled：屏幕滚动过程中不断被调用
- onPageSelected：手指滑动翻页且翻页距离足够大时被调用

```
//MainActivity.java日志输出
//开始翻页
onPageScrollStateChanged:显示当前访问人数
onPageScrollStateChanged:显示用户名
//连续滑动
onPageScrolled:显示时间
onPageScrolled:显示排名
onPageScrolled:显示时间
onPageScrolled:显示排名
onPageScrolled:显示时间
onPageScrolled:显示排名
//手指放开
onPageScrollStateChanged:显示当前访问人数
onPageScrollStateChanged:显示用户名
onPageSelected:记录用户行为
onPageSelected:统计分数
```

- 同时触发的不同监听器上同类事件处理器的执行顺序是按照组件上绑定监听器的顺序执行
- 同一时间监听器内多个事件处理器的触发顺序是按照事件的执行逻辑顺序

基于监听的事件处理机制

- 基于监听的事件处理流程
  - 第一步：为某个事件源(组件)绑定一个监听器，用于监听用户操作
  - 第二步：用户操作触发了事件源绑定的监听器
  - 第三步：生成对应的事件对象
  - 第四步：将事件源对象作为参数传给事件监听器
  - 第五步：事件监听器对事件源对象进行判断，执行对应的事件处理器(对应事件的处理方法)



基于监听的事件处理机制

```
public class MainActivity extends Activity {
        private Button btnshow;
        @Override
        protected void onCreate(Bundle savedInstanceState){
                super.onCreate(savedInstanceState);
                setContentView(R.layout.activity_main);
                btnshow = (Button) findViewById(R.id.btnshow);
                //直接new一个内部类对象作为参数
                btnshow.setOnClickListener(new BtnClickListener());
        }
        //定义一个内部类,实现View.OnClickListener接口,并重写onClick()方法
        class BtnClickListener implements View.OnClickListener{
            @Override
            public void onClick(View v) {
                Toast.makeText(getApplicationContext(), "按钮被点击了", Toast.LENGTH_
                SHORT).show();
                }
        }
}
```



事件参数去哪里了?

基于监听的事件处理机制

- Android中事件监听机制的优化

事件触发时，Android会将事件信息封装成**Event事件对象。根据实际需求确定是否将事件对象作为参数传递给事件处理器。

- 如果不传递事件对象也能满足设计需求，就不用封装事件对象
- 如果事件处理器中需要使用事件信息，就要封装事件对象并传递事件参数

```
planeView.setOnKeyListener(new OnKeyListener()
{
    @Override
    public boolean onKey(View source, int keyCode, KeyEvent event)
    {
        // 获取由哪个键触发的事件
        switch (event.getKeyCode())
        {
            // 控制飞机下移
            case KeyEvent.KEYCODE_S:
                planeView.currentY += speed;
                break;
            // 控制飞机上移
            case KeyEvent.KEYCODE_W:
                planeView.currentY -= speed;
                break;
            // 控制飞机左移
            case KeyEvent.KEYCODE_A:
                planeView.currentX -= speed;
                break;
            // 控制飞机右移
            case KeyEvent.KEYCODE_D:
                planeView.currentX += speed;
                break;
        }
        // 通知planeView组件重绘
        planeView.invalidate();
        return true;
    }
});
```

基于监听的事件处理机制

思考：

- 事件处理机制和JAVA中方法调用区别是什么
- 谁负责调用事件处理器

基于回调的事件处理

- 回调事件处理机制

  回调事件处理机制包括事件源和事件处理两个要素。

  - 事件源：界面上的组件
  - 事件处理：指定的事件处理方法，该方法继承GUI组件类，并重写类中的事件处理方法

GUI组件View的回调方法

①在该组件上触发屏幕事件: boolean onTouchEvent(MotionEvent event);
②在该组件上按下某个按钮时: boolean onKeyDown(int keyCode,KeyEvent event);
③松开组件上的某个按钮时: boolean onKeyUp(int keyCode,KeyEvent event);
④长按组件某个按钮时: boolean onKeyLongPress(int keyCode,KeyEvent event);
⑤键盘快捷键事件发生: boolean onKeyShortcut(int keyCode,KeyEvent event);
⑥在组件上触发轨迹球屏事件: boolean onTrackballEvent(MotionEvent event);

基于回调的事件处理

例：按钮的回调应用

```
//MyButton.java部分代码
public class MyButton extends Button{
    private static String TAG = "呵呵";
    public MyButton(Context context, AttributeSet attrs) {
        super(context, attrs);
    }
    //重写键盘按下触发的事件
    @Override
    public boolean onKeyDown(int keyCode, KeyEvent event) {
        super.onKeyDown(keyCode,event);
        Log.i(TAG, "onKeyDown方法被调用");
        return true;
    }//重写View的okKeyDown方法
}
```

为什么回调方法的返回值是boolean类型？

```
//布局文件部分代码
<RelativeLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
tools:context=".MyActivity">
<example.jay.com.mybutton.MyButton
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:text="按钮"/>
```

在布局文件中使用自定义View时必须用全限定类名

基于回调的事件处理

- 回调方法的返回值

  回调方法的返回值标识该方法是否能完全处理该事件

  - 返回true，表明该方法已经完全处理该事件，事件不会再传播出去
  - 返回false，表明该方法无法完全处理该事件，事件会被再传播出去
    - 传播方向：该组件所在Activity的回调方法

基于回调的事件处理

- 基于监听的事件处理vs基于回调的事件处理
  - 监听模型：事件源和事件监听器相互分离（托管机制），自定义事件
  - 回调模型：事件源和事件监听器融为一体（自管机制），通用型事件
- 监听和回调同时存在的事件处理顺序



基于回调的事件处理

例：按钮的回调事件传播

```
//MyButton.java部分代码
public class MyButton extends Button{
    private static String TAG = "呵呵";
    public MyButton(Context context, AttributeSet attrs){
        super(context, attrs);
    }
    //重写键盘按下触发的事件
    @Override
    public boolean onKeyDown(int keyCode, KeyEvent event){
        super.onKeyDown(keyCode,event);
        Log.i(TAG, "自定义按钮的onKeyDown方法被调用");
        return false;
    }
}
```

```java
//MainActivity.java部分代码
public class MyActivity extends ActionBarActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_my);
        Button btn = (Button)findViewById(R.id.btn_my);
        btn.setOnKeyListener(new View.OnKeyListener(){
            @Override
            public boolean onKey(View v, int keyCode, KeyEvent event){
                if(event.getAction() == KeyEvent.ACTION_DOWN){
                    Log.i("呵呵","监听器的onKeyDown方法被调用");
                }
                return false;
                }
        });
    }
    @Override
    public boolean onKeyDown(int keyCode, KeyEvent event){
        super.onKeyDown(keyCode, event);
        Log.i("呵呵","Activity的onKeyDown方法被调用");
        return false;
    }
}
```

```
//程序执行结果
"呵呵","监听器的onKeyDown方法被调用”
"自定义按钮的onKeyDown方法被调用“
"呵呵","Activity的onKeyDown方法被调用"
```

基于Handler的消息传递机制



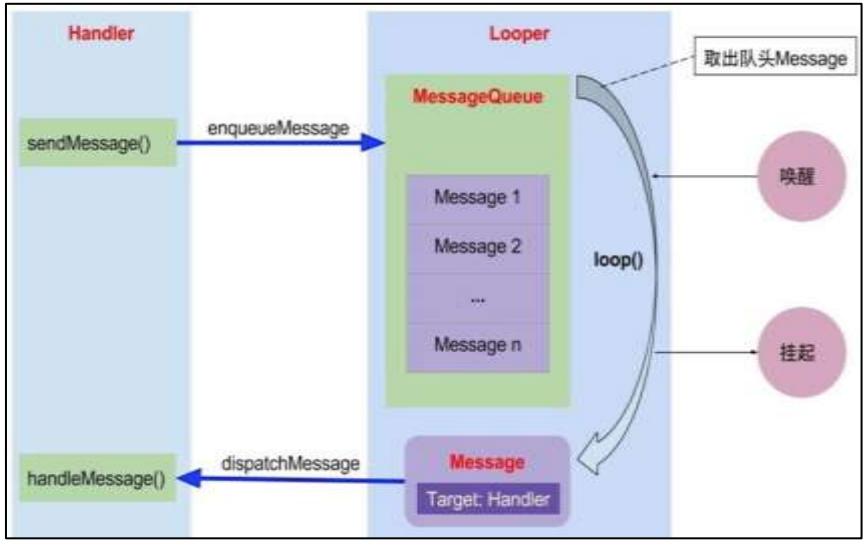
基于Handler的消息传递机制

- 线程间传递消息

Handler是Android一套消息处理机制，可以让子线程更新主线程UI界面，实现线程间通通信和消息处理。

- 核心概念：
  - ✓ UI 线程：主线程 ActivityThread，系统创建UI线程时，会初始化一个Looper对象、并创建一个与其关联的MessageQueue（一般一个应用程序有一个UI线程）。
  - ✓ Message：Handler 发送和处理的消息，由 MessageQueue 管理。
  - ✓ MessageQueue：消息队列，用来存放通过 Handler 发送的消息，按照先进先出执行，内部使用的是单链表的结构。
  - ✓ Handler：负责发送消息和处理消息。
  - ✓ Looper：负责消息循环，循环取出 MessageQueue 里面的 Message，并交给相应的 Handler 进行处理（一个线程有一个Looper）。

基于Handler的消息传递机制



- Handler工作原理
  - 四个对象：Handler, Message, MessageQueue, Looper
  - 两个状态：唤醒和挂起
  - 三个过程：子线程发送消息、消息队列取出消息、主线程接收和处理消息
    - 发送消息：子线程的Handler利用sendMessage方法发送一个消息，系统将该消息加入子线程自己的消息队列MessageQueue中
    - 取出消息：子线程消息队列的Looper遍历消息队列并将队列中的消息分发给主线程的Handler处理
    - 主线程Handler利用handleMessage方法处理该消息，完成了一个消息的发送和处理过程。

基于Handler的消息传递机制

- 在主活动中创建Handler

```
//MainActivity.java部分代码，主活动创建Handler
//引自《Android基础入门教程》第三章
public class MainActivity extends Activity {
    //定义切换的图片的数组id
    int imgids[] = new int[]{
        R.drawable.s_1, R.drawable.s_2,R.drawable.s_3,
        R.drawable.s_4,R.drawable.s_5,R.drawable.s_6,
        R.drawable.s_7,R.drawable.s_8
    };
    int imgstart = 0;
    final Handler myHandler = new Handler(){
        @Override//重写handleMessage方法,根据msg中what的值判断是否执行后续操作
        public void handleMessage(Message msg) {
            if(msg.what == 0x123){//message.what是用户定义的消息代码，以便收件人识别
                imgchange.setImageResource(imgids[imgstart++ % 8]);
            }
        }
    };
    @Override
    protected void onCreate(Bundle savedInstanceState) {
```

- 在子活动中创建Handler
  - S1：直接调用Looper.prepare()方法为当前子线程创建Looper对象，它的构造器会自动创建配套的MessageQueue;
  - S2：创建Handler对象，重写handleMessage( )方法，用于处理来自其他线程:信息
  - S3：调用Looper.loop()方法启动Looper

```
public class MyThread extends Thread {
    private Handler handler;
    @Override
    public void run() {
        super.run();
        // 创建Looper对象并关联到Thread
        Looper.prepare();
        handler = new Handler(Looper.myLooper(), new Handler.Callback() {
            @Override
            public boolean handleMessage(Message msg) {
                return false;
            }
        });
        Looper.loop();
    }
}
```

Looper常用方法

Looper.prepare(); // Looper初始化

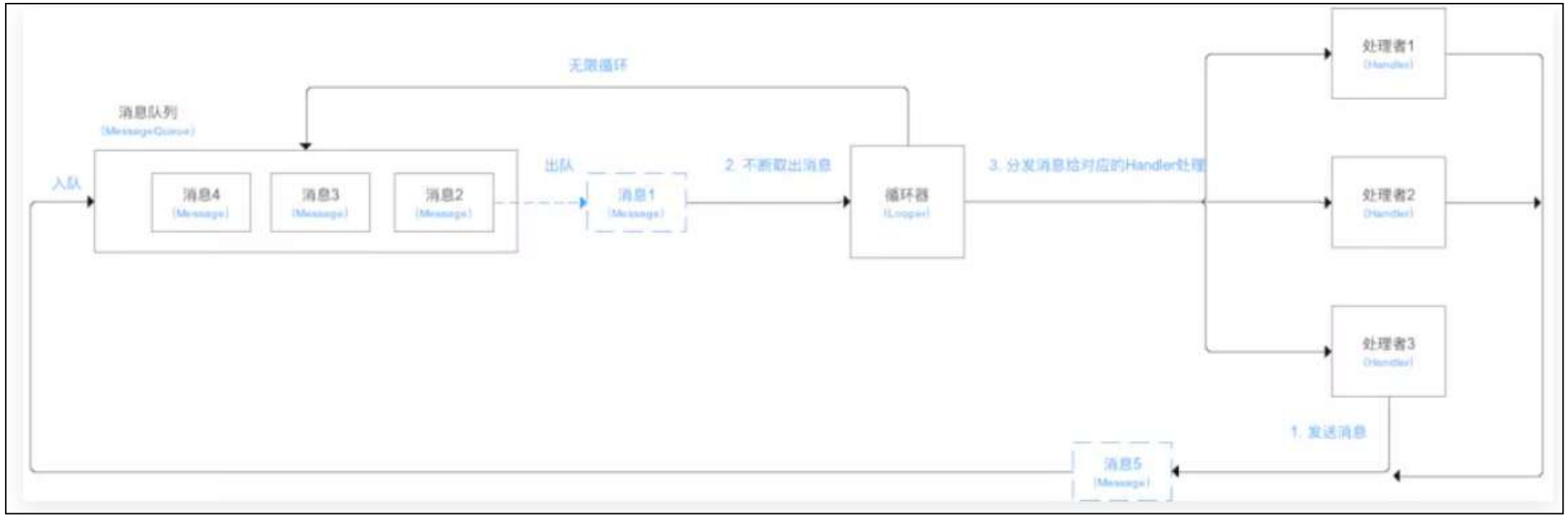
Looper.myLooper(); // 获取当前线程中ThreadLocal缓存的Looper对象

Looper.loop(); // 使当前线程进入循环，从而启动Handler机制

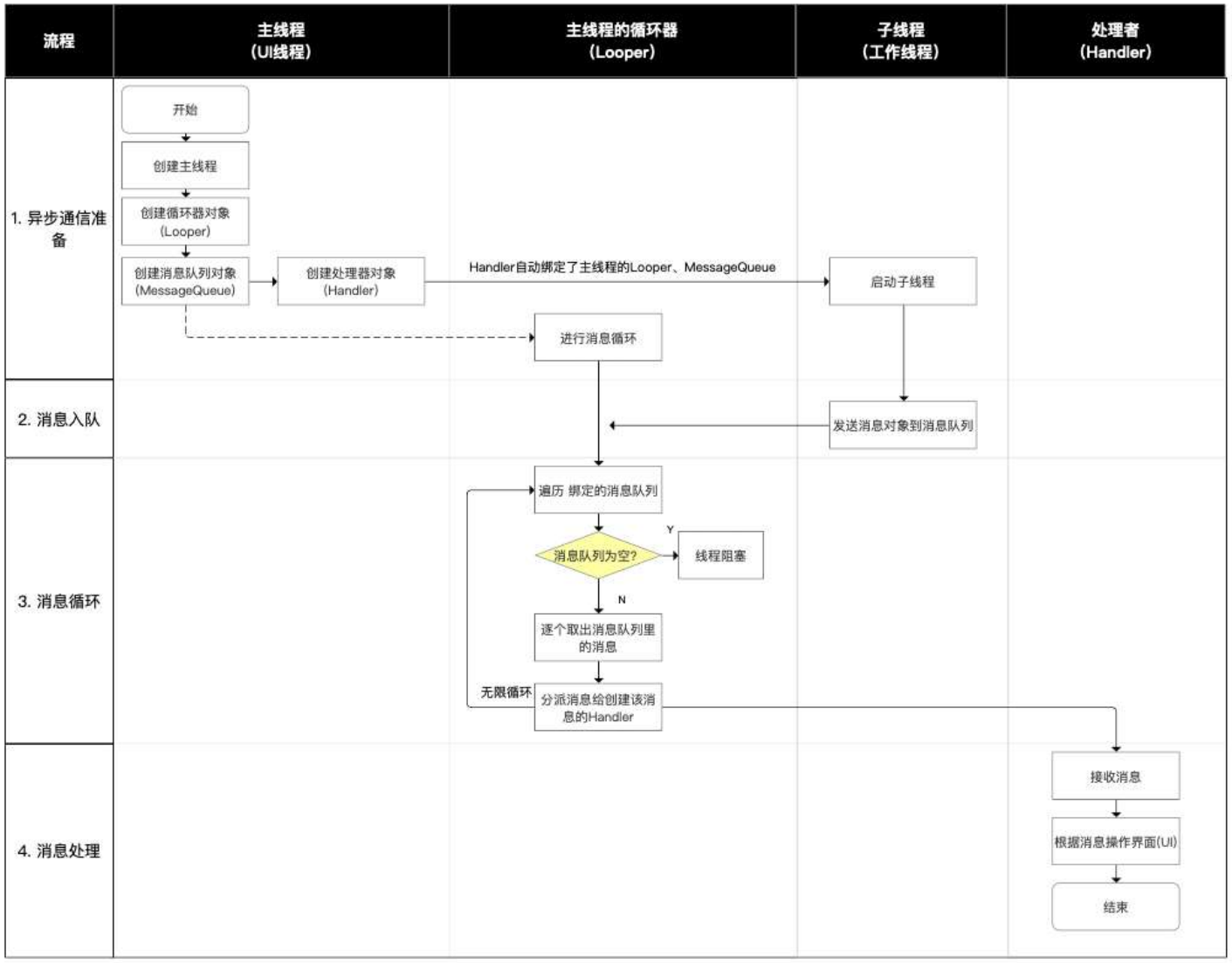
getLooper(); // 获取Handler缓存中的Looper对象

myHandler.getLooper().quit(); // 终止Looper.loop()的死循环

基于Handler的消息传递机制

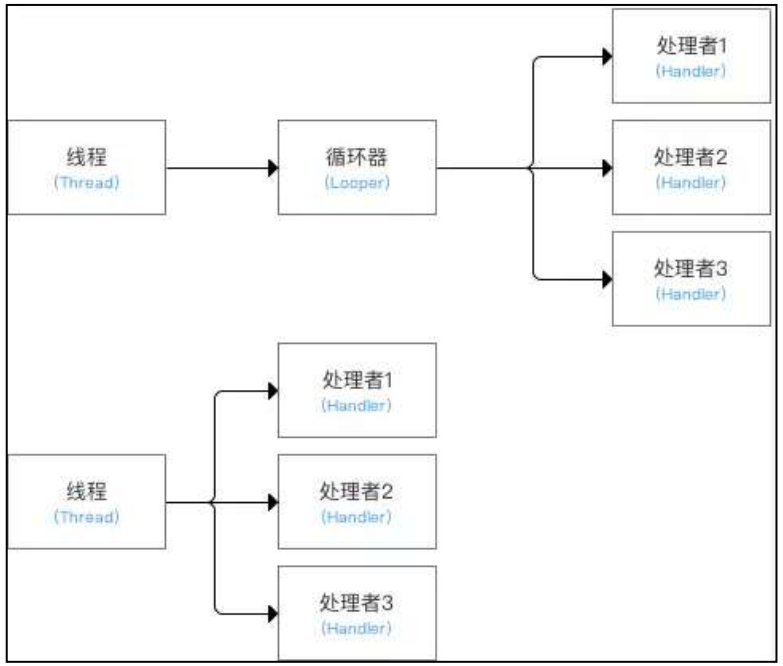


基于Handler的消息传递机制



基于Handler的消息传递机制

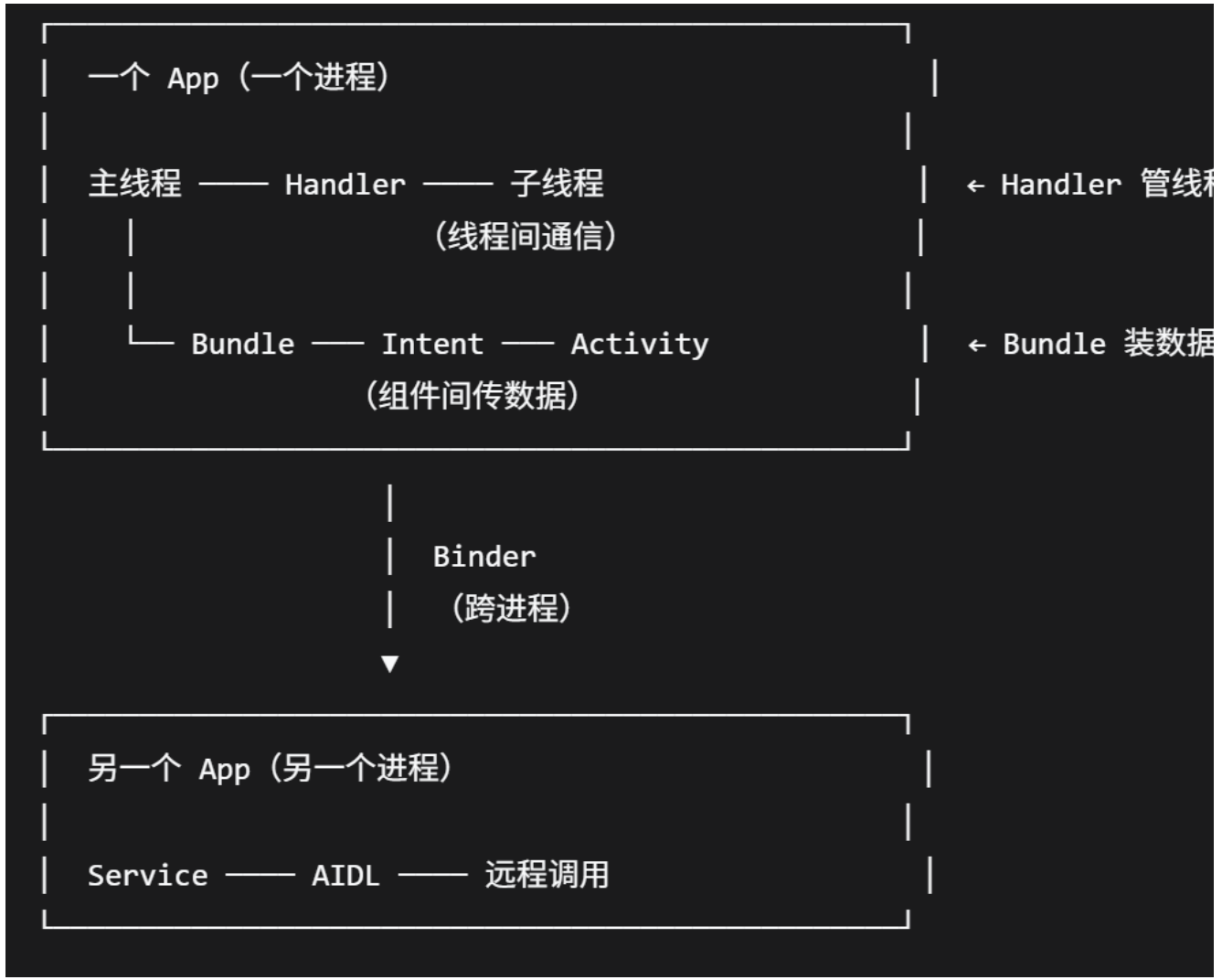
- 线程(Thread)、循环器(Looper)、处理者(Handler)三者关系
  - 1个线程(Thread)只能绑定1个循环器(Looper)，但可以有多个处理者(Handler)
  - 1个循环器(Looper)可绑定多个处理者(Handler)
  - 1个处理者(Handler)只能绑定1个循环器(Looper)



Handler 属于"线程"，不属于"消息 “

- 在主线程 new → 属于主线程
- 在子线程 new（已 prepare）→ 属于子线程
- **handleMessage：在 Handler 所属线程执行。**
- **sendMessage：任何线程都可以调用**，消息进入 Handler 所属线程的队列。
- **Handler 机制的精髓：一个线程可以往另一个线程的队列里塞消息，但消息的处理永远在"目标线程"上进行。**

```
Handler h1 = new Handler(); // 在哪个线程 new，就属于哪个线程
```


一个 App（一个进程）
主线程 —— Handler —— 子线程
（线程间通信）
← Handler 管线
└— Bundle —— Intent —— Activity
（组件间传数据）
← Bundle 装数据
Binder
（跨进程）
另一个 App（另一个进程）
Service —— AIDL —— 远程调用

Android四大组件——Service

- 服务组件（Service）

长期运行在后台、没有用户界面、甚至程序关闭仍可运行。

- Service不运行在独立进程，而是默认运行在主线程，即它依赖于创建它的应用程序进程
- 当应用程序进程被杀掉时，所有依赖于该进程的服务都会停止运行
- Service不能自动开启线程，需要在服务内部手动创建子线程，并在其中执行具体任务。否则，主线程会被阻塞
- Service分为两种形式：本地服务和远程服务
  - 本地服务：非独立进程，依附于主进程，服务和启动服务的组件在相同进程，节省资源
  - 远程服务：独立进程，不依附于主进程，服务和启动服务的组件在不同进程，为多进程提供服务

服务不能自己启动自己，需要借助其他组件

- 创建服务
  - 创建服务：右击项目，选择“New->Service-> Service”，新建*.java文件
  - 创建活动：在主活动.java文件中添加组件
  - 启动/停止服务：在组件中调用启动服务方法和停止服务方法
  - 注册服务：在manifest.xml文件添加<service>标签

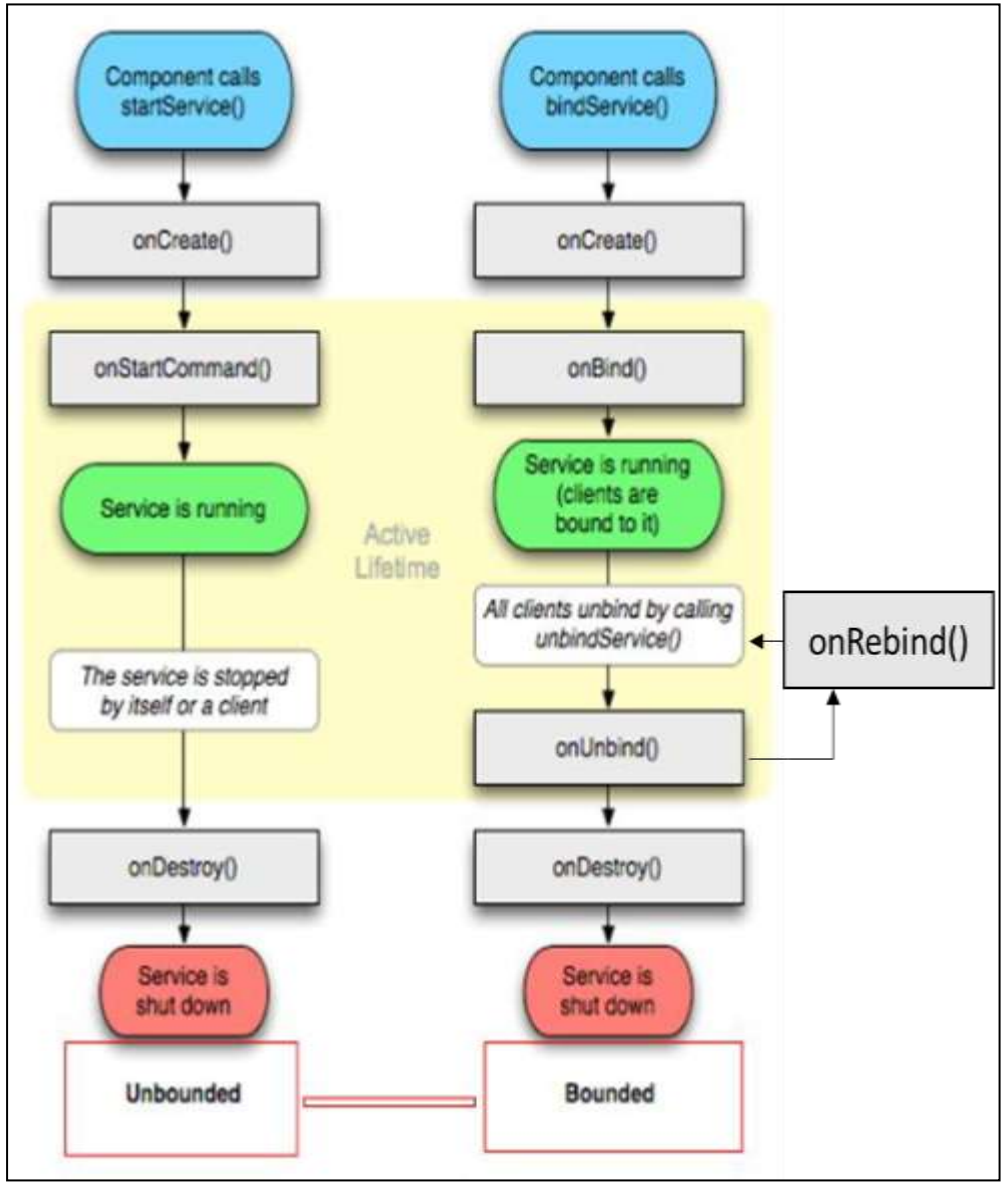
```
//创建服务
public class HelloService extends Service {
    /*当服务被创建时调用*/
    @Override
    public void onCreate(){
    }
    /*调用startService()启动服务时回调*/
    @Override public int onStartCommand(Intent intent,
int flags, int startId){
        return mStartMode;
    }
    /*通过bindService()绑定到服务的客户端*/
    @Override public IBinder onBind(Intent intent){
```

```
//注册服务
<application
    <service
        android:name="com.example.service.MyService"
        android:enabled="true"
        android:exported="true">
    </service>
</application>
```

- 启动服务（两种类型）
  - StartService()模式
    - 启动：调用StartService()方法
    - 停止：调用stopService ()方法
    - 状态：服务处于stated状态
    - 特点：当启动服务的组件被销毁，服务仍然运行

      服务和组件在一个进程
  - bindService()模式
    - 启动：绑定bindService()方法
    - 停止：解绑unbindService()方法
    - 状态：服务处于bound状态
    - 特点：绑定服务的组件解绑后，服务被销毁

      多个组件可以绑定一个服务



- 服务中的回调方法

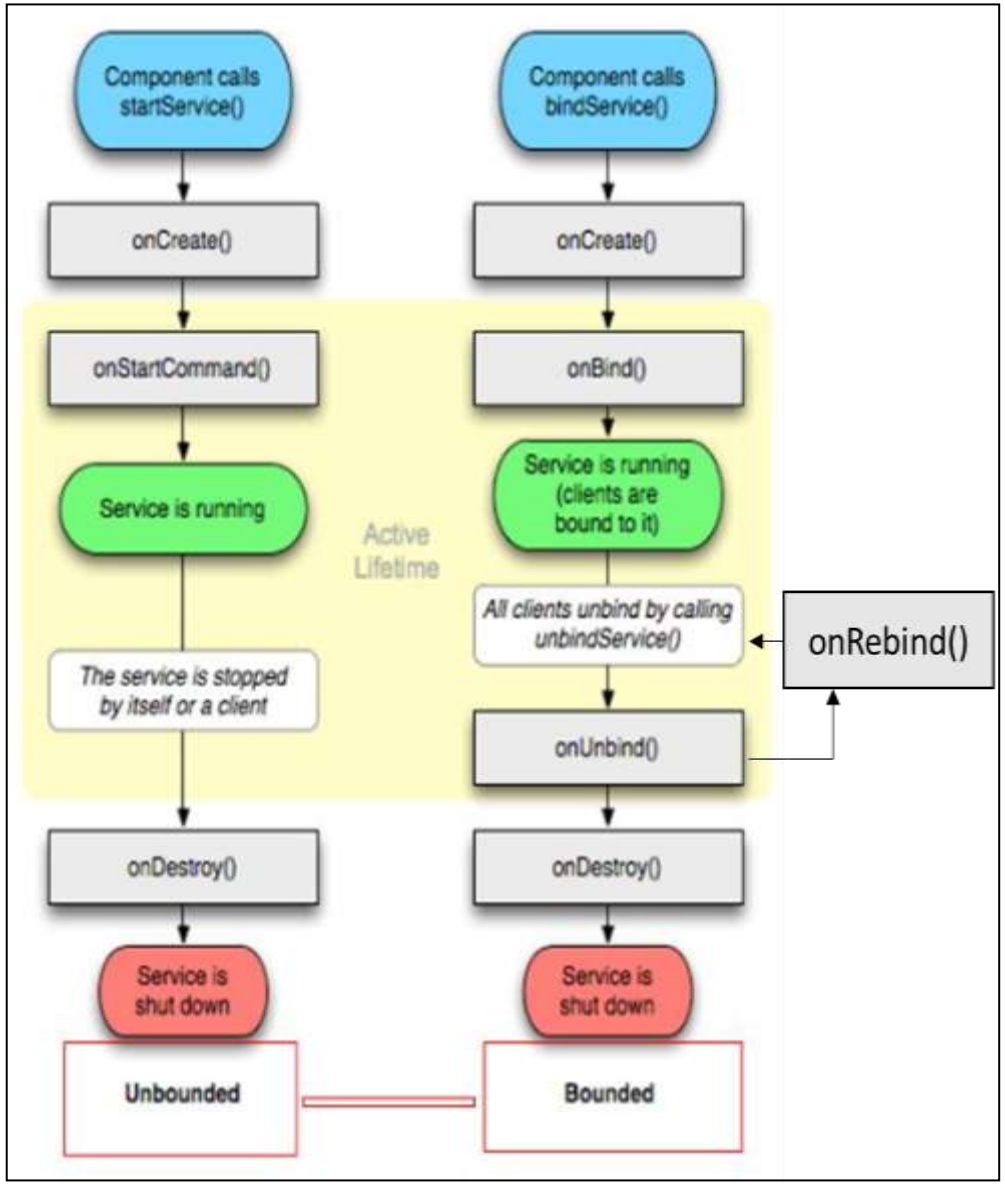
回调	描述
onStartCommand()	组件(如活动)通过调用startService()来请求启动服务时，系统回调该方法。一旦执行此方法，服务即会启动并可在后台无限期运行。如果采用该方法启动服务，必须通过stopSelf()或者stopService()方法来停止服务。
onBind()	当组件通过bindService()绑定服务时，系统回调该方法。如果采用该方法启动服务，需要通过返回IBinder对象提供一个接口，实现客户端与服务通信。无论是启动状态还是绑定状态，此方法必须重写，但在启动状态下直接返回 null。
onUnbind()	当客户中断所有服务发布的特殊接口时，系统回调该方法。
onRebind()	当新的客户端与服务连接，且此前它已经通过onUnbind(Intent)通知断开连接时，系统回调用该方法。
onCreate()	首次创建服务时，系统回调方法来执行一次性设置程序（在回调onStartCommand() 或onBind() 之前）。如果服务已在运行，则不会调用此方法，该方法只调用一次
onDestroy()	当服务不再有用或者被销毁时，系统回调该方法。需要销毁的服务通过实现该方法来清理任何资源，如线程、已注册的监听器、接收器等。（执行一次）

- 两种服务启动模式的区别
  - startService() 模式：
    - 一旦启动，服务即可在后台无限期运行
    - 即使启动服务的组件已被销毁也不受影响，除非手动调用才能停止服务
    - 已启动的服务通常是执行单一操作，而且不会将结果返回给调用方
  - bindService() 模式：
    - 被绑定服务提供了一个客户端-服务器接口，允许组件与服务进行交互、发送请求、获取结果
    - 利用进程间通信 (IPC，Inter-Process Communicate) 跨进程执行跨进程操作。 仅当与另一个应用组件（如活动）绑定时，绑定服务才会运行。 多个组件可以同时绑定到该服务，但全部取消绑定后，该服务即会被销毁。

- 服务的两个状态

状态	形式	含义
stated	本地服务	组件通过调用startService()启动了服务，则服务是Started状态。一旦启动，服务可以在后台无限期运行，即使启动它的组件已经被销毁。
bound	本地服务/远程服务	组件通过调用bindService()绑定了服务，则服务是Bound状态。Bound状态的服务提供了一个客户服务器接口来允许组件与服务进行交互，如发送请求，获取结果，甚至通过IPC来进行跨进程通信。

- 两种启动服务方式的执行过程
  - startService()方式：
    - 启动：调用startService()方法，依次回调–>onCreate()–>onStartCommand()
    - 停止：调用stopService()方法，依次回调–>onDestroy()
    - 多次启动：多次调用startService()方法，只执行执行一次onCreate()，自动执行多次onStartCommand()
  - bindService()方式
    - 启动：调用bindService()方法，依次回调–>onCreate() –>onBind()
    - 停止：调用unbindService()方法，依次回调–>onUnbind()–>onDestroy()
    - 多次启动：回调一次onCreate()，回调一次onBind()



【例】StartService()方法：连续单击三次开始服务，一次停止服务，logcat输出什么，依次调用了哪些方法



单击一次开始按钮
- 回调onCreate()方法
- 回调onStartCommand()方法

再连续单击两次开始按钮
- 回调onStartCommand()方法
- 回调onStartCommand()方法

单击一次停止按钮
- 回调onDestory()方法

```java
//自定义TestService.java
public class TestService extends Service {
    private final String TAG = "TestService";
    //必须要实现的方法
    @Override
    public IBinder onBind(Intent intent) {
        Log.i(TAG, "onBind方法被调用!");
        return null;
    }
    //Service被创建时调用
    @Override
    public void onCreate() {
        Log.i(TAG, "onCreate方法被调用!");
        super.onCreate();
    }
    //Service被启动时调用
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        Log.i(TAG, "onStartCommand方法被调用!");
        return super.onStartCommand(intent, flags, startId);
    }
    //Service被关闭之前回调
    @Override
    public void onDestroy() {
        Log.i(TAG, "onDestory方法被调用!");
        super.onDestroy();
    }
}
```

```xml
//AndroidManifest.xml完成Service注册
<!-- 配置Service组件,同时配置一个action -->
    <service android:name=".TestService1">
        <intent-filter>
            <action android:name=" com.jay.example.service.TEST_SERVICES"/
        </intent-filter>
    </service>
```

```
//mainActivity.java中启动服务
public class MainActivity extends Activity {
    private Button start;
    private Button stop;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        start = (Button) findViewById(R.id.btnstart);
        stop = (Button) findViewById(R.id.btnstop);
        //创建启动Service的Intent,以及Intent属性
        final Intent intent = new Intent();
        intent.setAction("com.jay.example.service.TEST_SERVICE1");
        //为两个按钮设置点击事件,分别是启动与停止service
        start.setOnClickListener(new OnClickListener() {
            @Override
            public void onClick(View v) {
                startService(intent);
                }
            });
            stop.setOnClickListener(new OnClickListener() {
                @Override
                public void onClick(View v) {
                    stopService(intent);
                }
             });
      }
}
```

intent负责将服务组件传递给activity组件

思考：

- 两个组件间如何传递数据？
- 为什么要用intent间接传递服务组件？

- StartService()方法详解
- 涉及方法：回调方法包括onCreate()、onStartCommand()、onDestory()
  普通方法包括startService()、stopService()
- 执行过程：1）首次调用StartService()方法，创建Service对象，依次回调onCreate()和onStartCommand()方法；
  2）后续再调用StartService()方法，不创建Service对象，只回调onStartCommand()方法
  3）建立的Service对象与调用组件无关，关闭调用组件Service仍运行
  4）调用StopService()方法，关闭Service对象，同时回调onDestory()方法

回调方法和普通方法的区别：系统自动调用和手动调用

- Service与Activity之间的通信
  - 通信类型：本地通信、远程通信
  - 通信前提：必须保证服务是以绑定的形式开启的，否则无法进行通信和数据交换
  - 通信载体：IBinder对象
  - 通信方法：调用bindService()方法和回调onServiceConnected()方法

bindService(Intent service, ServiceConnection conn, int flags)

- service：通过Intent指定要启动的服务
- coon：监听访问者与服务的连接情况，如果连接成功，则回调onServiceConnected()方法；如果连接断开，则回调onServiceDisconnected()方法
- flags：指定绑定时是否自动创建服务(前提是该服务还未创建)。如果是0，则不自动创建；如果是BIND_AUTO_CREATE，则自动创建

onServiceConnected(ComponentName name, IBinder service)

- ComponentName：已连接服务的具体组件名称
- IBinder：通信接口，实现与绑定服务间的通信

Binder和IBinder的关系

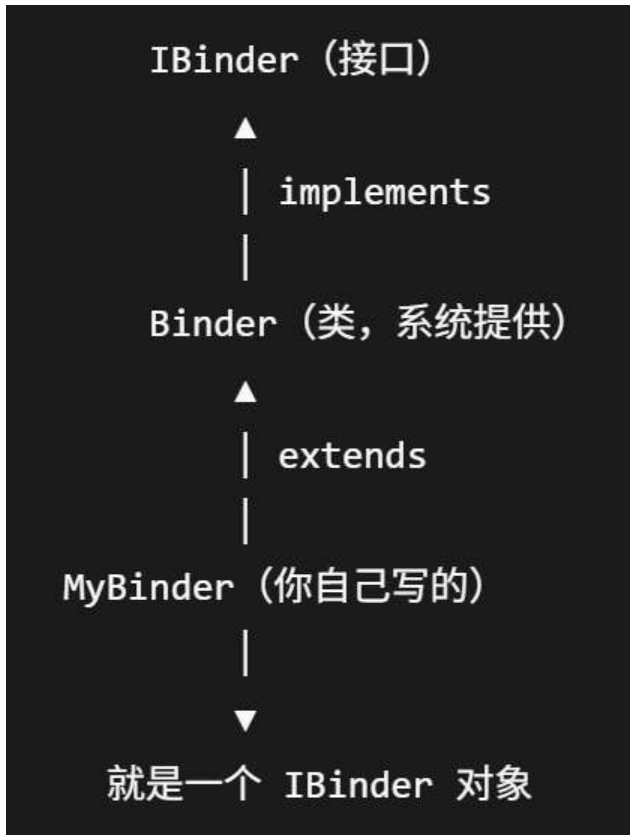
- IBinder是接口，必须自己写代码逻辑实现功能
- Binder是IBinder的具体实现类，继承Binder的类就是IBinder对象

```java
//在activity.java中实现
private BindService.MyBinder binder;
//回调ServiceConnection()方法
private ServiceConnection connection = new ServiceConnection(){
    @Override
    public void onServiceConnected(ComponentName name, IBinder service){
        System.out.printIn("连接服务")
        binder=(BindService.MyBinder)service;
        binder.startComm();//调用服务中的方法
    }
    @Override
    public void onServiceDisconnected(ComponentName name){
        System.out.printIn(“断开服务")
    }
};
//绑定服务
Intent bindIntent=new Intent(this,MyService.class);
bindService(bindIntent,connection,BIND_AUTO_CREATE);
//解绑服务
unbindService(connection);
```

连接成功，返回Ibinder对象

```java
//自定义BindService.java
public class BindService extends Service {
    private MyBinder mBinder = new MyBinder();
    private final String TAG = "TestService";
    //通过继承Binder类实现Ibinder对象
    public class MyBinder extends Binder{
        public void startComm(){
            Log.d(TAG, "startComm方法被调用");
        }
    }
    //必须要实现的方法
    @Override
    public IBinder onBind(Intent intent) {
        Log.i(TAG, "onBind方法被调用!");
        return mBinder;
    }
    //Service被创建时调用
    @Override
    public void onCreate() {
        Log.i(TAG, "onCreate方法被调用!");
        super.onCreate();
    }
    //Service被启动时调用
    @Override
    public int onStartCommand(Intent intent, in
        Log.i(TAG, "onStartCommand方法被调用!"
        return super.onStartCommand(intent, fla
```

```
① Activity: new Intent + bindService(intent, conn, BIND_AUTO_CREATE)
        ↓
② Service:  onCreate() 被调用          ← 日志：onCreate方法被调用!
        ↓
③ Service:  onBind() 被调用            ← 日志：onBind方法被调用!
        ↓
④ Service:  返回 mBinder
        ↓
⑤ Activity: onServiceConnected() 被调用  ← 打印：连接服务
        ↓
⑥ Activity: binder = (BindService.MyBinder) service;   ← 强转
        ↓
⑦ Activity: binder.startComm();        ← 打印：startComm方法被调用
        ↓
⑧ Activity: unbindService(connection)
        ↓
⑨ Service:  onUnbind()                 ← 如果重写了
        ↓
⑩ Service:  onDestroy() 被调用         ← 日志：onDestroy方法被调用!
```



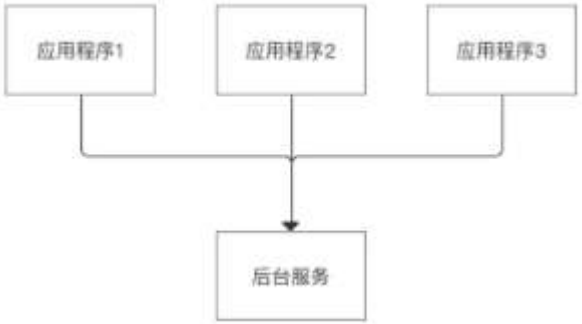
- BindService()方法实现服务通信
  - 本地服务通信：调用者通过Ibinder对象与service通信
    - 在Service的*.java文件中，自定义方法提供给访问者使用
    - 在Activity的*.java文件中，调用Service中自定义的各种方法
      - 创建Binder子类，让前端能够通过Binder类实现对service的调用
        - ✓ 调用bindService()方法，回调onCreate()方法和onBind()方法，返回IBinder对象
      - 在前端的onServiceConnected()方法中接收返回的Ibinder对象
        - ✓ 如果服务连接成功，则回调onServiceConnected()方法，获取服务返回的IBinder对象
      - 前端利用IBinder对象调用服务中自定义的各方法

- startService()方法与BindService()方法
  - 利用intent和service组件或其他组件间的传递
    - startService(intent)
    - bindService(intent, conn, Service.BIND_AUTO_CREATE);
  - startService()中的Service组件和其他组件无关
  - BindService()中的Service组件和其他组件相关，并且支持他们之间的通信

- 远程通信

远程服务通信时通过AIDL（Android Interface Definition Language）实现接口定义语言。

- 产生原因：Android系统中每个应用程序都运行在自己的进程中。如果想要完成不同进程（不同应用应用程序）间的通信，需要利用远程服务通信。
- 应用场景：多个应用程序共享同一个后台服务



- AIDL使用方法
  - 满足要求：
    - ✓ AIDL定义接口的源代码必须以.aidl结尾。
    - ✓ AIDL接口中用到的数据类型，除了基本数据类型及String、List、Map、CharSequence之外，其他类型全部都需要导入到包，即使它们在同一个包中。
    - ✓ Service端和客户端都需要使用Android SDK安装目录下的platform-tools子目录下的aidl.exe工具为AIDL接口提供实现（ADT工具会自动为AIDL接口生成实现）

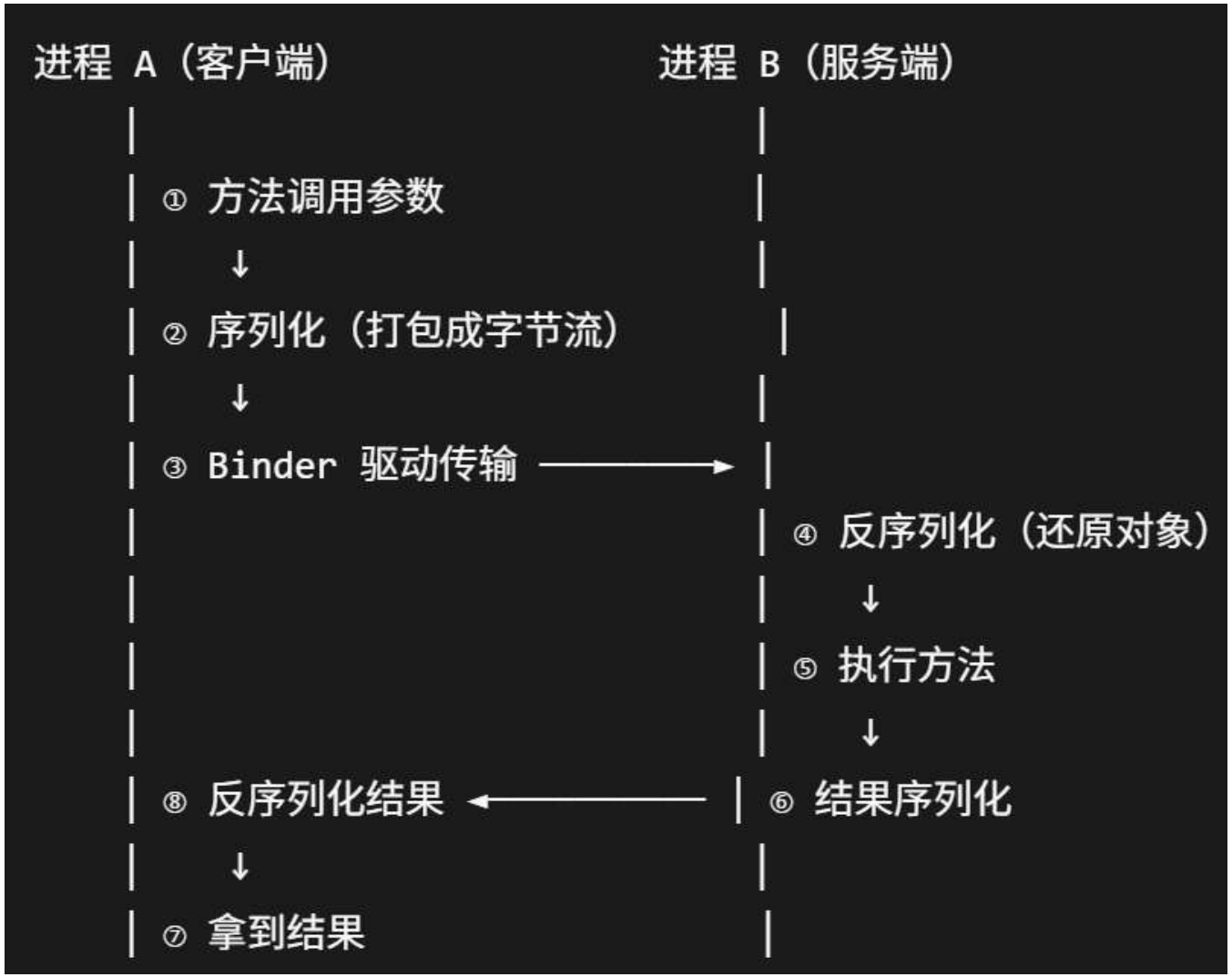
定义好上面的 AIDL 接口之后，ADT 工具会自动在 gen/org/crazyit/service 目录下生成一个 ICat.java 接口，在该接口里包含一个 Stub 内部类，该内部类实现了 IBinder、ICat 两个接口，这个 Stub 类将会作为远程 Service 的回调类——它实现了 IBinder 接口，因此可作为 Service 的 onBind()方法的返回值。

AIDL 的三步使用流程

第 1 步：定义 .aidl 文件

第 2 步：编译，自动生成 Java 接口

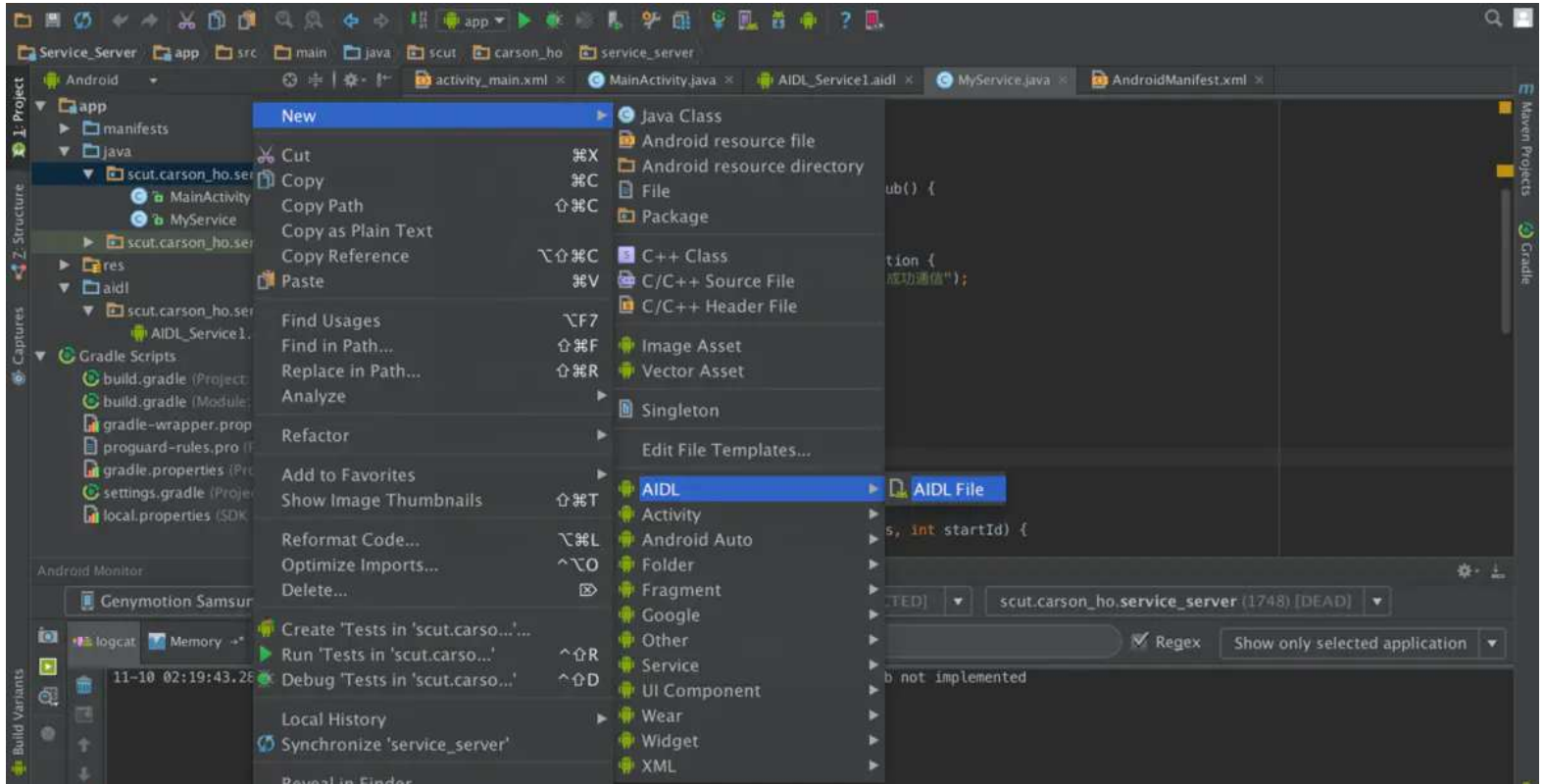
第 3 步：Service 端实现 + Activity 端调用



- 服务器端的实现

新建一个服务器端的工程： Service-server

- 步骤1. 新建一个AIDL文件（文件名为AIDL_Service1.aidl” ）



参考资料：
https://www.jianshu.com/p/34326751b2c6

- 步骤2. 在新建AIDL文件里定义Service需要与Activity进行通信方法（类似JAVA接口定义），并进行编译
  - 写 .aidl 文件
  - 编译自动生成 AIDL_Service1.java（含 Stub）

```
//在新建的AIDL_Service1.aidl里声明需要与Activity进行通信的方法
package scut.carson_ho.demo_service;
interface AIDL_Service1 {
    void AIDL_Service();
}
```

- 步骤3：在Service子类中实现AIDL中定义的接口方法，并定义生命周期的方法（onCreat、onBind()等）

```
//在MyService.java中实现
public class MyService extends Service {
    // 实例化AIDL的Stub类(Binder的子类)
    AIDL_Service1.Stub mBinder = new AIDL_Service1.Stub() {
        //重写接口里定义的方法
        @Override
        public void AIDL_Service() throws RemoteException {
            System.out.println("客户端通过AIDL与远程后台成功通信");
        }
    };
    //重写与Service生命周期的相关方法
    @Override
    public void onCreate() {
        super.onCreate();
        System.out.println("执行了onCreat()");
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
```

- 步骤4：在AndroidMainfest.xml中注册服务，同时声明为远程服务
  - 如有要在服务中注册Intent的action，则必须写成“服务器端包名.aidl文件名“

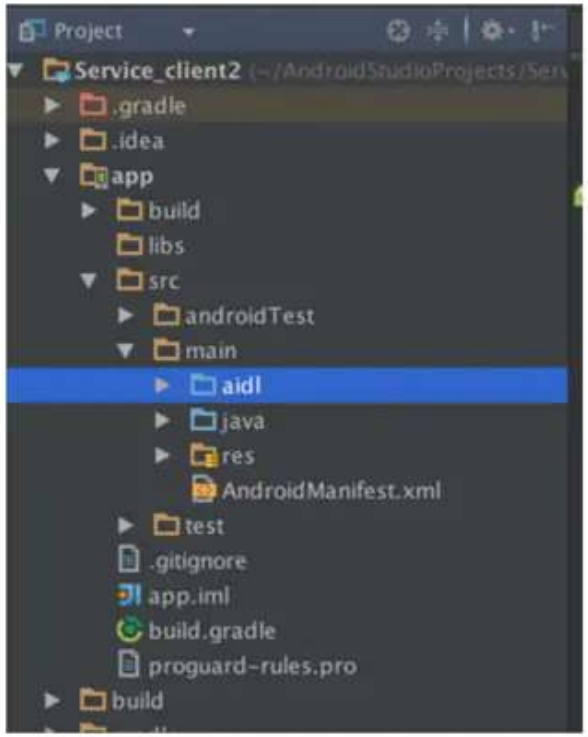
```
//在AndroidMainfest.xml中实现
<service android:name=".MyService"
    android:process=":remote" //将本地服务设置成远程服务
    android:exported="true" //设置可被其他进程调用
    //该Service可以响应带有scut.carson_ho.service_server.AIDL_Service1这个action的Intent。
    //此处Intent的action必须写成“服务器端包名.aidl文件名”
    <intent-filter>
        <action android:name="scut.carson_ho.service_server.AIDL_Service1"/>
    </intent-filter>
</service>
```

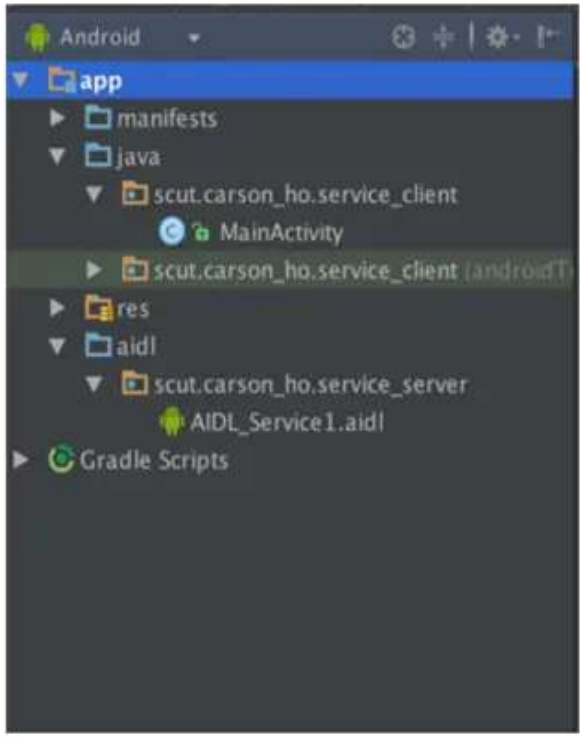
:remote	当前应用的私有进程，进程名为 包名:remote
com.example.remote	全局进程，其他应用也可以共享

- 客户器端的实现

  新建一个客户端的工程：Service-Client

- 步骤1：将服务端的AIDL文件所在的包复制到客户端目录下（Project/app/src/main），并进行编译（Make Project）





- 步骤2：在主布局文件定义“绑定服务”的按钮

```
//在MainActivity.xml中实现
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context="scut.carson_ho.service_client.MainActivity">

    <Button>
        android:layout_centerInParent="true" android:id="@+id/bind_service"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="绑定服务"
    <Buttion/>
</RelativeLayout>
```

- 步骤3：在MainActivity.java中实现如下内容
  - ✓ 使用Stub.asInterface接口获取服务器的Binder；
  - ✓ 通过Intent指定服务端的服务名称和所在包，进行Service绑定；
  - ✓ 根据需要调用服务提供的接口方法。

```
public class MainActivity extends AppCompatActivity {
    private Button bindService;
    private AIDL_Service1 mAIDL_Service; //定义aidl接口变量
    //创建ServiceConnection的匿名类
    private ServiceConnection connection = new ServiceConnection() {
        //重写onServiceConnected()方法和onServiceDisconnected()方法
        //在Activity与Service建立关联和解除关联的时候调用
        //在Activity与Service建立关联时调用
        @Override
        public void onServiceConnected(ComponentName name, IBinder service) {
            //使用AIDLService1.Stub.asInterface()方法获取服务器端返回的IBinder对象
            //将IBinder对象传换成了mAIDL_Service接口对象
            mAIDL_Service = AIDL_Service1.Stub.asInterface(service);
```

- 测试结果

```
11-10 03:51:57.410 1414-1414/scut.carson_ho.service_client I/System.out: 点击了[绑定服务]按钮
11-10 03:51:57.470 1695-1695/scut.carson_ho.service_server:remote I/System.out: 执行了onCreat()
11-10 03:51:57.470 1695-1695/scut.carson_ho.service_server:remote I/System.out: 执行了onBind()
11-10 03:51:57.470 1695-1706/scut.carson_ho.service_server:remote I/System.out: 客户端通过AIDL与远程后台成功通信
```

AIDL 流程

【步骤 1】写 .aidl 文件
【步骤 2】编译自动生成 AIDL_Service1.java（含 Stub）
【步骤 3】Service 里实例化 Stub + 生命周期 + onBind 返回 Stub
【步骤 4】Manifest 注册 + 声明远程进程 + 声明 action
【步骤 5】客户端 bindService 绑定并调用方法

- 跨App和同App之间的区别
- ①跨App开启服务是提供服务的App需要设置intent-filter过滤器，控制服务的App需要通过。setAction和setPackage方法进行设置action和包名，才能开启服务。而同App只需要指定启动的service就可。
- ②跨App的MyBinder实例要通过AIDL获取，两个应用定义同样的接口的方法，通过对应的AIDL名称.Stub.asInterface方法得到binder实例，然后就和同App的myBinder使用么有区别了。
- ③跨App的MyBinder对象的使用必须捕获异常，而同App不需要。

```
<!-- 服务端 App 的 Manifest -->
<service android:name=".MyService"
    android:process=":remote"
    android:exported="true">
    <intent-filter>
        <action android:name="com.example.service.AIDL_Service1"/>
    </intent-filter>
</service>
```

```
<!– 客户端 -->
Intent intent = new Intent();
intent.setAction("com.example.service.AIDL_Service1");     // 设置 action
intent.setPackage("com.example.service");                  // 设置服务端包名
bindService(intent, conn, BIND_AUTO_CREATE);
```

对比	跨 App	同 App
获取方式	`AIDL_Service1.Stub.asInterface(service)`	`(MyBinder) service`
返回类型	AIDL 接口	自定义 Binder
底层	系统帮你做了 Proxy 封装	直接强转

asInterface 做了什么?

- **如果是同进程：直接返回 Stub 本身**
- **如果是跨进程：返回一个 Proxy 对象（代理），自动把调用序列化、通过 Binder 传输**

为什么两个 App 要定义"同样的接口方法"?

- AIDL 要求**客户端和服务端的** .aidl **文件完全一致**
- 否则序列化/反序列化的顺序对不上，会崩溃
- 服务端和客户端通常复制同一份 .aidl 文件

- Stub

Stub类继承了Binder，因此Stub自己是一个Binder本地对象，然后实现了IMyAidlInterface接口，IMyAidlInterface本身是一个IInterface，因此他携带某种客户端需要的能力（这里是方法add)。此类有一个内部类Proxy，也就是Binder代理对象

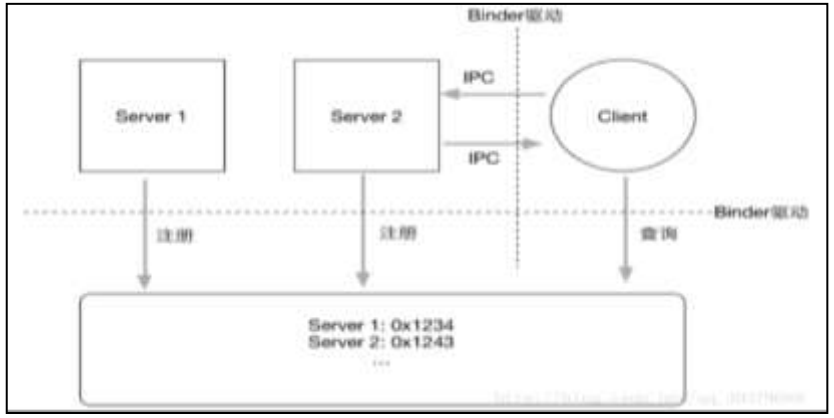
https://blog.csdn.net/qq_30379689/article/details/79451596

类别	特点	优点	缺点	应用场景
本地服务（LocalService）	1. 运行在主线程 2. 主进程被终止后，服务也会终止。	1. 节约资源； 2. 通信方便：由于在同一进程因此不需要IPC和AIDL。	限制性大：主进程被终止后，服务也会终止。	需要依附某个进程的服务，如音乐播放
远程服务（RemoteService）	1. 运行在独立进程； 2. 服务常驻在后台，不受其他Activity影响	灵活：服务常驻在后台，不受其他Activity影响	1. 消耗资源：单独进程 2. 使用AIDL进行IPC复杂	系统级别服务

- Binder通信模式

Binder通信采用的是client-server通信结构，client与server的通信由Binder驱动程序和Service Manager组件协助

- Client进程：跨进程通讯的客户端（运行在某个进程）
- Server进程：跨进程通讯的服务端（运行在某个进程）
- Binder驱动：跨进程通讯的介质
- ServiceManager：跨进程通讯中提供服务的注册和查询（运行在System进程）

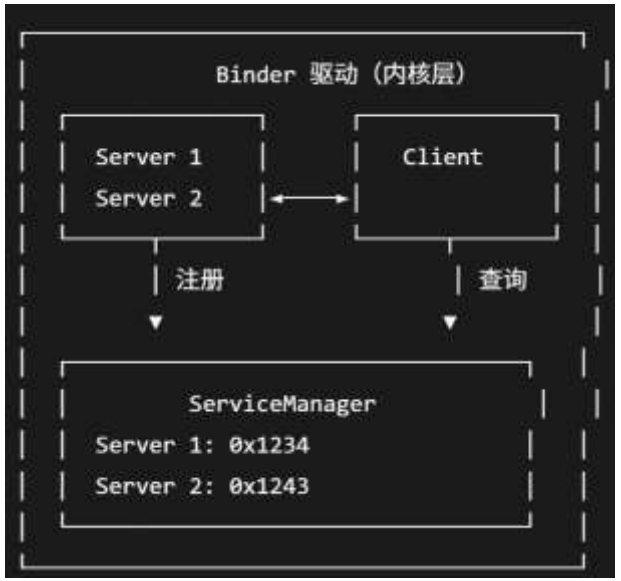


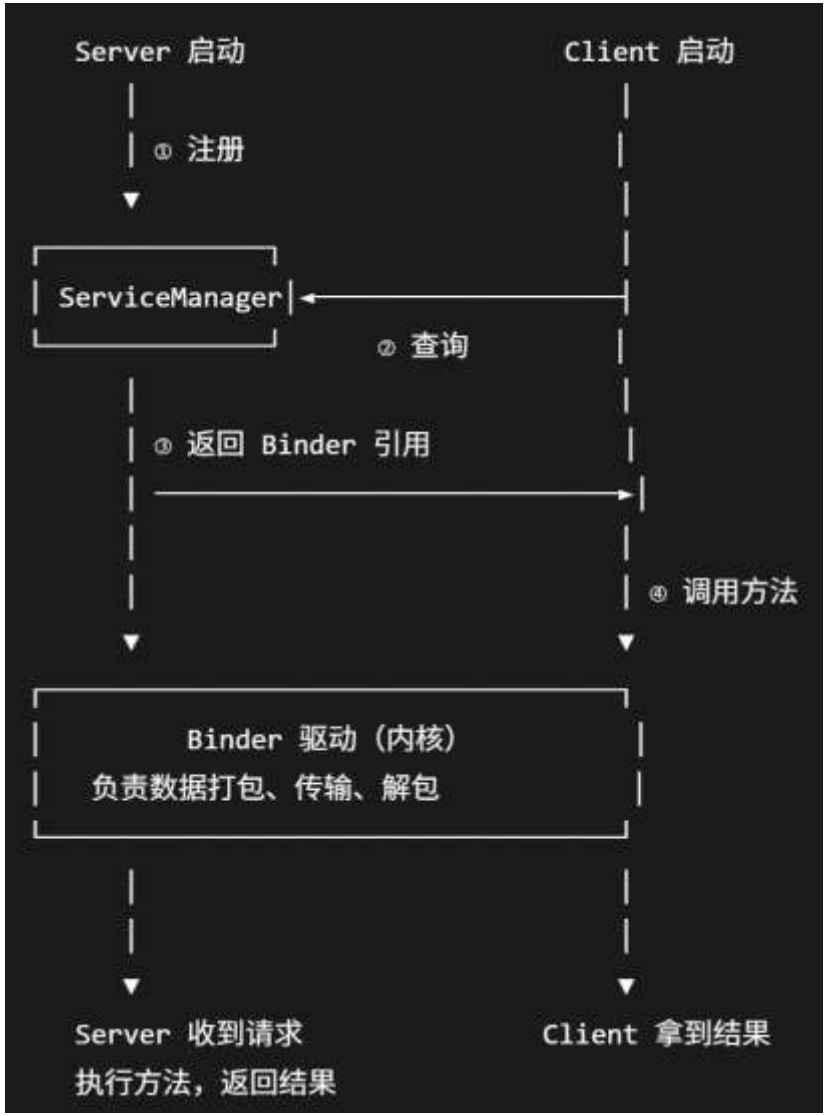
Binder模型通信流程

- Server端通过Binder驱动在ServiceManager中注册
- Client端通过Binder驱动获取ServiceManager中注册的Server端
- Client端通过Binder驱动和Server端进行通讯

Binder 通信采用 client-server 架构，四大角色分工

- Client 发起请求
- Server 提供服务
- ServiceManager 负责注册和查询（电话簿）
- Binder 驱动负责数据传输（内核通道）
- 流程三步：Server 注册 → Client 查询 → Client 调用。
- 整个通信建立在"一次拷贝"的 mmap 机制上，高效、安全。
- AIDL 就是 Binder 的上层封装。





- Binder通信过程

1.client通过获得一个server的代理接口，对server进行直接调用

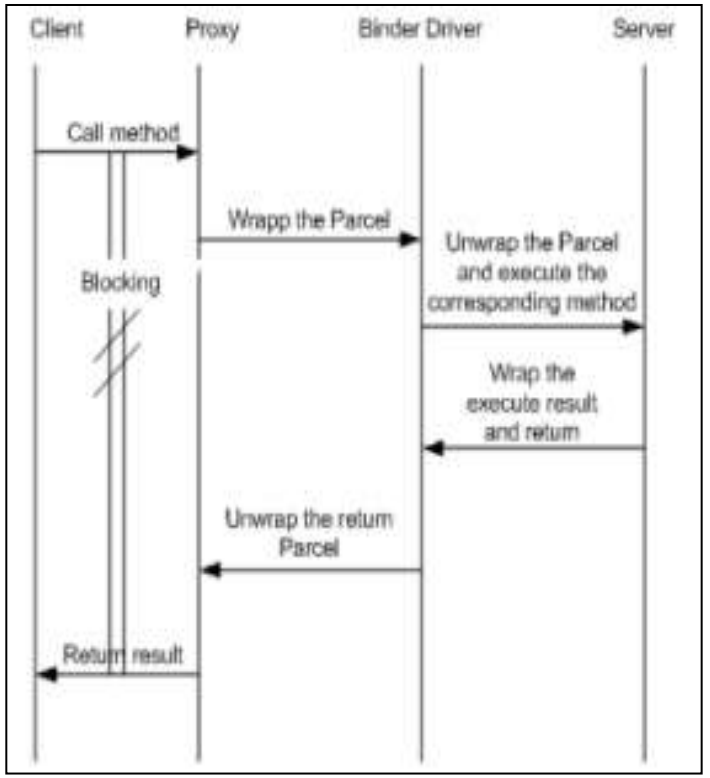
2.代理接口中定义的方法与server中定义的方法一一对应

3.client调用某个代理接口中的方法时，代理接口的方法会将client传递的参数打包成为Parcel对象

4.代理接口将该Parcel发送给内核中的binder driver

5.server会读取binder driver中的请求数据，如果是发送给自己的，解包Parcel对象，处理并返回结果

6.Binder调用过程是同步过程，当server处理时，client会被block

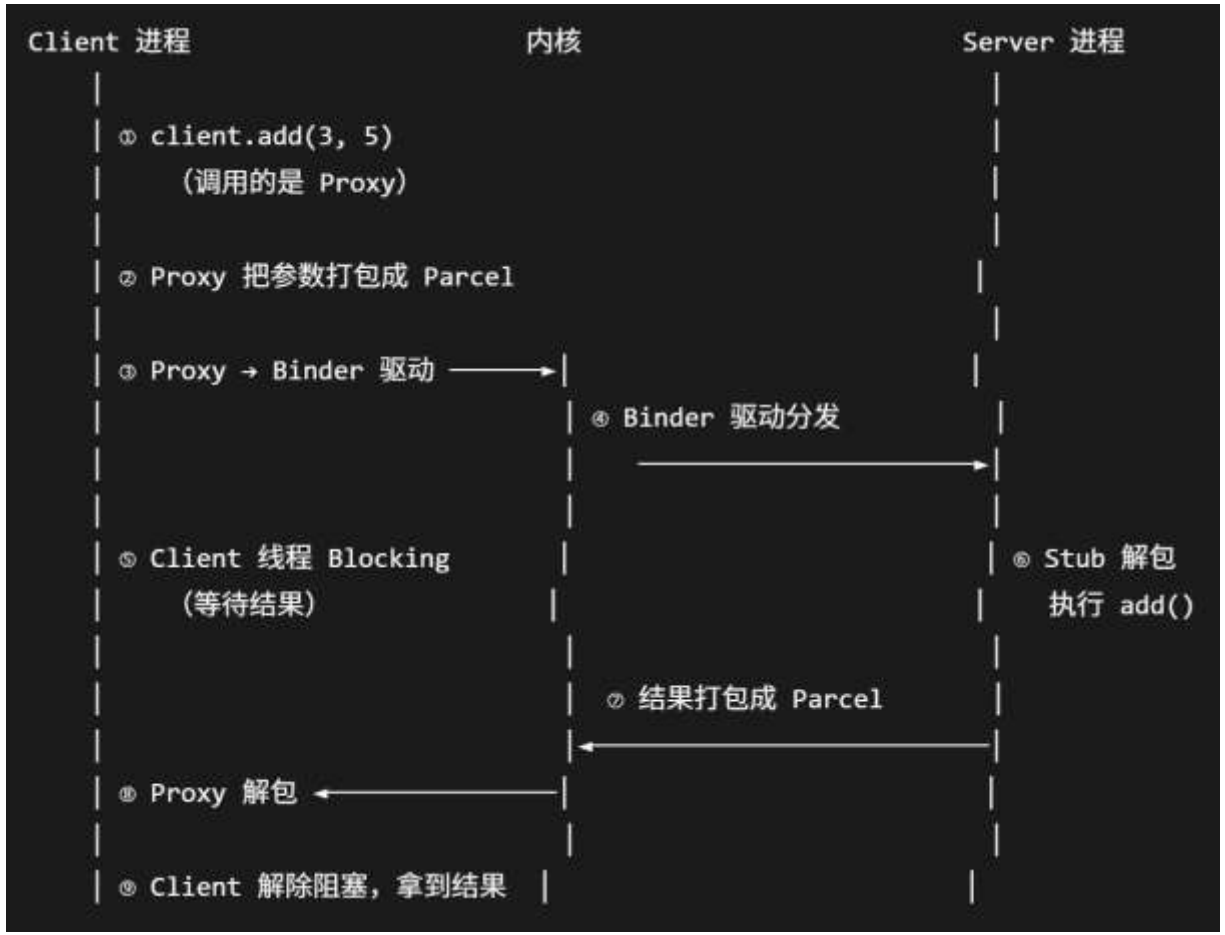


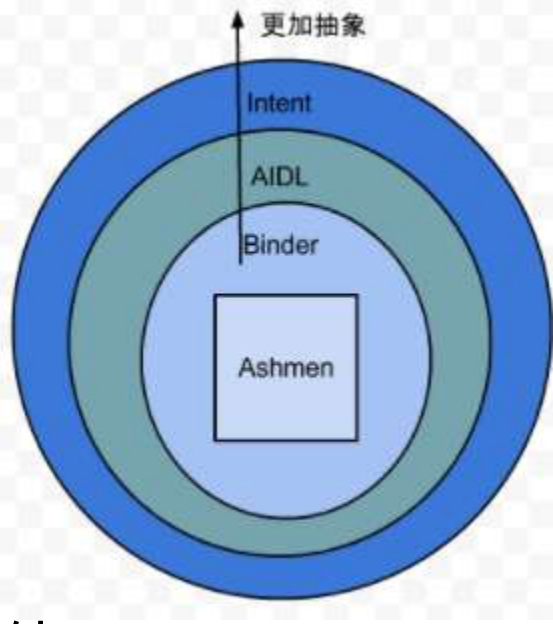
- Proxy 是谁创建的？（关键机制）

  AIDL 自动生成 + 系统动态返回。

  - 同进程：asInterface 返回 Stub 本身
  - 跨进程：asInterface 返回 Proxy 对象

① Service 端返回 Binder（Stub 对象）
↓
② 系统通过 Binder 驱动传到 Client 进程
↓
③ 到 Client 后，无法传递真实对象（跨进程）
↓
④ 系统生成一个 Proxy 对象（长得和接口一样）
↓
⑤ Proxy 持有 Binder 引用，负责打包/传输
↓
⑥ onServiceConnected 返回给 Client 的就是 Proxy





- Binder通信概念
  - IPC是一种概念，即进程间通信；
  - Binder是对IPC的具体实行，是IPC的一种具体实现；
  - AIDL是Binder机制向外提供的接口，目的是为了方便对Binder的使用；
  - Intent是最高层级的封装，实质是封装了对Binder的使用，当然Intent也常常在同一进程中调用，只是把两种方式封装在一起了。

参考资料：https://blog.csdn.net/jinxinliu1/article/details/70174591

- 为什么用Binder实现跨进程通信

Linux采用管道、消息队列、信号量、内存共享、套接字等实现跨进程通信，Android采用Binder实现跨进程通信。

- 传输性好
  - ✓ Socket：是一个通用接口，导致其传输效率低，开销大
  - ✓ 共享内存：虽然在传输时不需要拷贝数据，但其控制机制复杂
  - ✓ Binder：复杂数据类型传递可以复用内存
  - ✓ 管道和消息队列：采用存储转发方式，效率低
- 安全性高
  - ✓ 传统的进程：通信方式对于通信双方的身份并没有做出严格的验证，只有在上层协议上进行架设
  - ✓ Binder机制：从协议本身就支持对通信双方做身份校检，因而大大提升了安全性

参考资料：https://blog.csdn.net/qq_30379689/article/details/79451596

- AIDL使用方法

AIDL是为了编程方便而设计的，使得开发者尽可能关注代码实现。

- 只要按照规范写一个.aidl文件，AIDL插件会自动创建一个与interface同名的.java文件(.java文件提供标准化模板，编程者只需在自动生成的框架中填写必要实现)
- .aidl文件的写法和JAVA接口类似
- 自动生成的.java中有一个重要的内部类Stub，它继承了Binder类，需要实现预先定义的interface

```
//.aidl文件
interface MyAidl{
    int getId();
    void setId(int id);
  }
```

```
public static abstract class Stub extends android.os.Binder implements com.xxx.MyAidl {
     public static com.xxx.MyAidl asInterface(android.os.IBinder obj);
     public boolean onTransact(int code, android.os.Parcel data,android.os.Parcel reply,
int flags);

     private static class Proxy implements com.hsae.recorder.encoder.Mp4EncoderAidl {
            ......
      }
  }
```

Android数据存储

- 数据持久化

  数据持久化技术则提供一种机制让数据在瞬时状态（内存中的数据）和持久状态（外存中的数据）之间转换

  - 文件存储：原样保存
  - SharedPreferences存储
  - 数据库存储
  - SD卡存储

存储方式	具体描述	默认 存储路径	应用场景
SharedPreferences	以键值对的形式存储在xml中	/data/data/<PackageName>/shared_prefs	轻量级存储 (如 应用中的配置、参数属性)
ContentProvider	• Android的四大组件之一 • 仅作为传输数据的媒介，即 数据源可多样	通过 URI 对象	进程间 数据共享、交换
文件存储	通过 Java I/O 的读 / 写 方法 写入到文件中	/data/data/<PackageName>/files	量大的数据、文件缓存
SQLite数据库	嵌入式数据库、使用 SQL语言 (Android为此数据库提供了SQLiteDatabase类，封装了操作数据库的API)	/data/data/<PackageName>/databases	结构性 数据
网络存储	通过网络 获取 远程服务器 的数据	远程服务器	• 量非常大的数据（本地无法存储） • 与远程应用共享、交互 • 更新数据

文件存储

- 文件存储数据

  将数据保存到文件中

  - 特点：对数据不做任何改变
  - 适用：结构简单、数量较大的文本数据或二进制数据
  - 原理：将打开的文件构造成一个OutputStreamWriter对象，对该对象执行写入操作
  - 实现：调用Context类的openFileOutput()方法
    - 第一个参数：文件名（建议加扩展名），但不能包含路径（默认存储路径/data/data/< package name >/files/）
    - 第二个参数：文件操作模式
      - MODE_PRIVATE：默认模式，表示对同一文件操作时，后写入的内容将会覆盖原文件内容
      - MODE_APPEND：如果文件存在，则在文件末尾追加内容；如果文件不存在，则创建新文件
    - 返回值：一个FileOutputStream对象

- 文件存储数据（续）
  - 步骤：四步
    - 第一步：利用openFilieOutput()方法获得FileOutputStream对象
    - 第二步：借助FileOutputStream对象构建OutputStreamWriter对象
    - 第三步：通过OutputStreamWriter对象构建BufferedWriter对象
    - 第四步： 通过BufferedWriter对象将内容按行写入文件

```
public void save() {
    String data = "Data to save";
    FileOutputStream out = null;
    BufferedWriter writer = null;
    try {
        out = openFileOutput("data", Context.MODE_PRIVATE);
        writer = new BufferedWriter(new OutputStreamWriter(out));
```

扩展知识

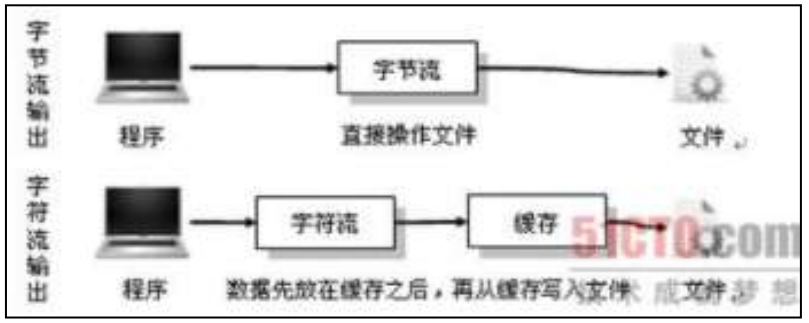
- 写文件的几点说明

说明1：FileOutputStream和OutputStreamWrite

- FileOutputStream：是字节流，它一个字节一个字节的向外传输数据（不能处理unicode编码）
- OutputStreamWrite：是字符流，它一个字符一个字符的向外传输数据（能处理unicde编码，如中文）

字节流和字符流

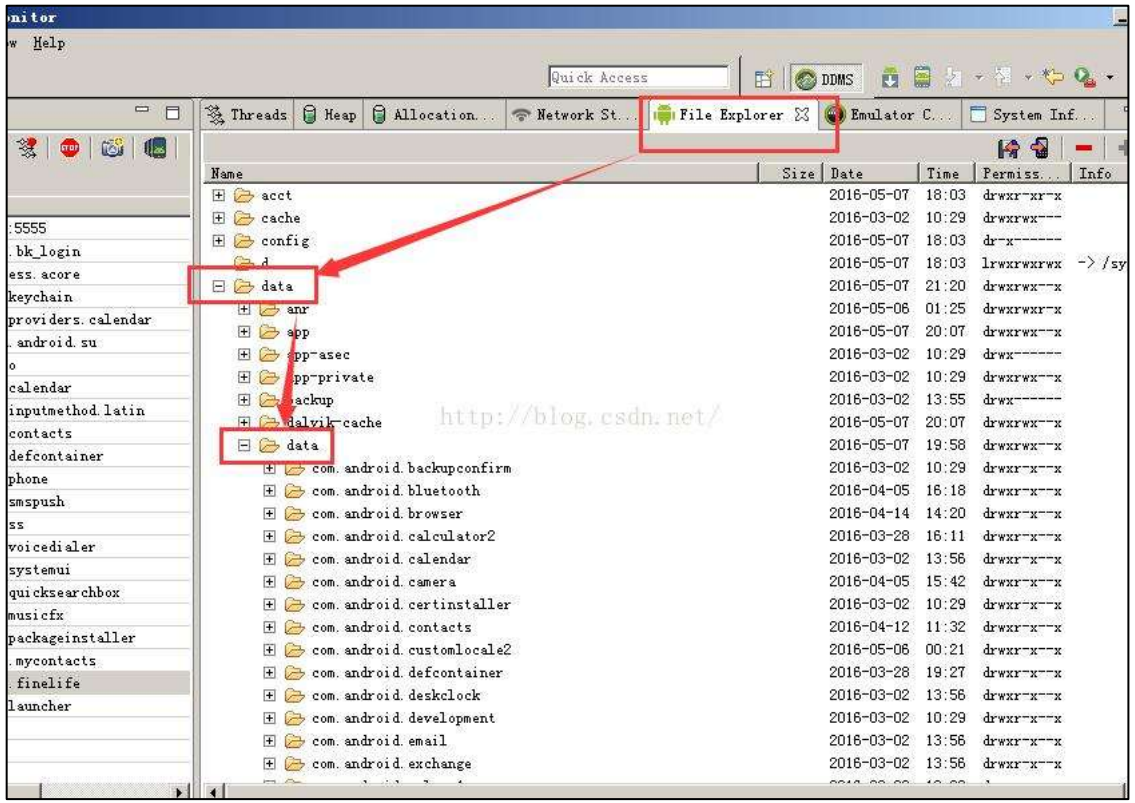
- 字节流是直接操作文件，不会用到缓冲区（内存）
- 字符流是通过缓冲区间接操作文件，会用到缓冲器



- 写文件的几点说明（续）
- 说明2：缓冲器的作用
  - 缓冲区是一段特殊的内存空间。将频繁操作一个资源（如文件或数据库）暂存在缓冲区，可以直接从内存操作数据，以提高程序性能。
  - 如果直接stream或write操作文件，则按一个字符或者一个字节的方式操作一次硬盘一次
  - 如果使用Buffer封装了stream或write的对象，则按一批数据的方式操作一次硬盘。

```
FileOutputStream out = openFileOutput("data", Context.MODE_PRIVATE);
out.write(data);
osw.close();
```

```
FileOutputStream out = openFileOutput("data", Context.MODE_PRIVATE);
BufferedWriter writer = new BufferedWriter(new OutputStreamWriter(out));
writer.write(data);
writer.close();
```



在Android Device Monitor中，查看生成的data文件（/data/data/< package name >/files/）

BufferedWriter类的使用

- 操作对象：文件流
- BufferedWriter.write()：将文件流写入缓冲区，并没有接写入目的文件
- BufferedWriter.flush()：刷新缓冲流，将文件流写入目的文件
- BufferedWriter.close()：在关闭文件流前刷新缓冲流，将文件流写入目的文件

openFileOutput(filename,mode)	打开文件输出流,就是往文件中写入数据,第二个参数是模式
openFileIntput(filename)	打开文件输入流,就是读取文件中的信息到程序中
getDir(name,mode)	在app的data目录下获取或创建name对应的子目录
getFileDir()	获得app的data目录的file目录的绝对路径
String[] fileList()	返回app的data目录下的全部文件
deleteFile(filename)	删除app的data目录下的指定文件

```
//完整代码
public class MainActivity extends AppCompatActivity {
    private EditText edit;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        edit = (EditText) findViewById(R.id.edit);
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        String inputText = edit.getText().toString();
        save(inputText);
    }
    public void save(String inputText) {
        FileOutputStream out = null;
        BufferedWriter writer = null;
        try {
        out = openFileOutput("data", Context.MODE_PRIVATE);
        writer = new BufferedWriter(new OutputStreamWriter(out));
```

- 文件读取数据

将数据从文件中读出

- 原理：将打开的文件构造成一个InputStreamReader对象，对该对象执行读取操作
- 实现：调用Context类的openFileInput()方法
  - 参数：要读取的文件名（建议加扩展名）
  - 返回值：一个FileInputStream对象

```
//按行读取文件
public String load() {
    FileInputStream in = null;
    BufferedReader reader = null;
    StringBuilder content = new StringBuilder();
    try {
        in = openFileInput("data");
        reader = new BufferedReader(new InputStreamReader(in));
```

- 文件读取数据（续）
  - 第一步：利用openFileInput()方法获得FileInputStream对象
  - 第二步：借助FileInputStream对象构建InputStreamReader对象
  - 第三步：通过InputStreamReader对象构建BufferedReader对象
  - 第四步： 通过BufferedReader对象将文件按行读出

```
//逐行读取文件内容
public String load() {
    import java.io.FileInputStream;
    void readFileOnLine(){
    String strFileName = "Filename.txt";
    FileInputStream fis = openFileInput(strFileName);
    StringBuffer sBuffer = new StringBuffer();
    DataInputStream dataIO = new DataInputStream(fis);
    String strLine = null;
    while((strLine =  dataIO.readLine()) != null) {
```

注意

- openFileInput()方法只能读取在目录/data/data/< package name >/files/下的文件
- openFileInput()方法的局限性较强，一般不常用

- SharedPreferences存储

SharedPreferences提供一种轻量级的数据存储方式。它通过用键值对把简单数据类型（boolean、int、float、long和String）存储在应用程序的私有目录下(data/data/包名/shared_prefs/)自己定义的*.xml文件中。

  - 应用场景：格式简单、数量较少（比如配置文件、登录信息、密码等）
  - 使用条件：需要先获取到SharedPreferences对象。具体有三种方法：
    - Context类的getSharedPreferences()方法
    - Activity类的getPreferences()方法
    - PreferenceManager类的getDefaultSharedPreferences()方法

```
//方法1：Context 类的 getSharedPreferences() 方法
SharedPreferences sp = getSharedPreferences("config", Context.MODE_PRIVATE);

//方法2：Activity 类的 getPreferences() 方法
SharedPreferences sp = getPreferences(Context.MODE_PRIVATE);

//方法 3：PreferenceManager 类的 getDefaultSharedPreferences() 方法
SharedPreferences sp = PreferenceManager.getDefaultSharedPreferences(this);
```

方法	文件名	适用范围
Context.getSharedPreferences()	自定义	任意 Context
Activity.getPreferences()	Activity 类名	只能在 Activity 内
PreferenceManager.getDefaultSharedPreferences()	包名_preferences	任意 Context

- Context类的getSharedPreferences()方法
  - 创建文件对象：通过调用Context类中的getSharedPreferences(String name,int mode)方法创建SharePeferences文件对象（简称SF对象）
    - 第一个参数：指定SharedPreferences文件的名称（*.xml），如果指定的文件存在，则自动在/data/data/<package name>/shared_prefs/目录下创建
    - 第二个参数：指定操作模式，即有MODE_PRIVATE（或0）表示只有当前程序可以读写该SharedPreferences文件
  - 创建Editor对象：调用SF对象的edit()方法创建SharePeferences.Editor对象（简称SE对象）

- Context类的getSharedPreferences()方法
  - 读取数据：通过SE对象的键key可以获取到对应key的键值，如
    getString(String key, String defValue)，不同类型的键值有不同的函数
  - 存储数据：第一步，向SE对象添加数据，如putString()，putInt()等
    第二步，通过SE对象调用commit()提交新值，并写入xml文件

put*写操作说明**

- 执行过程分为两个步骤：

1）首先，将数据先写入内存的map集合中，

2）然后，将数据写入硬盘文件，保证数据的完整性

- 写操作有两种实现方法

1）commit()：同步提交，线程安全，但性能慢，通常在当前线程完成写文件操作（有返回值）

2）apply()：异步提交，线程不安全，但性能高，通常在当前进程完成写文件操作（无返回值）

扩展知识

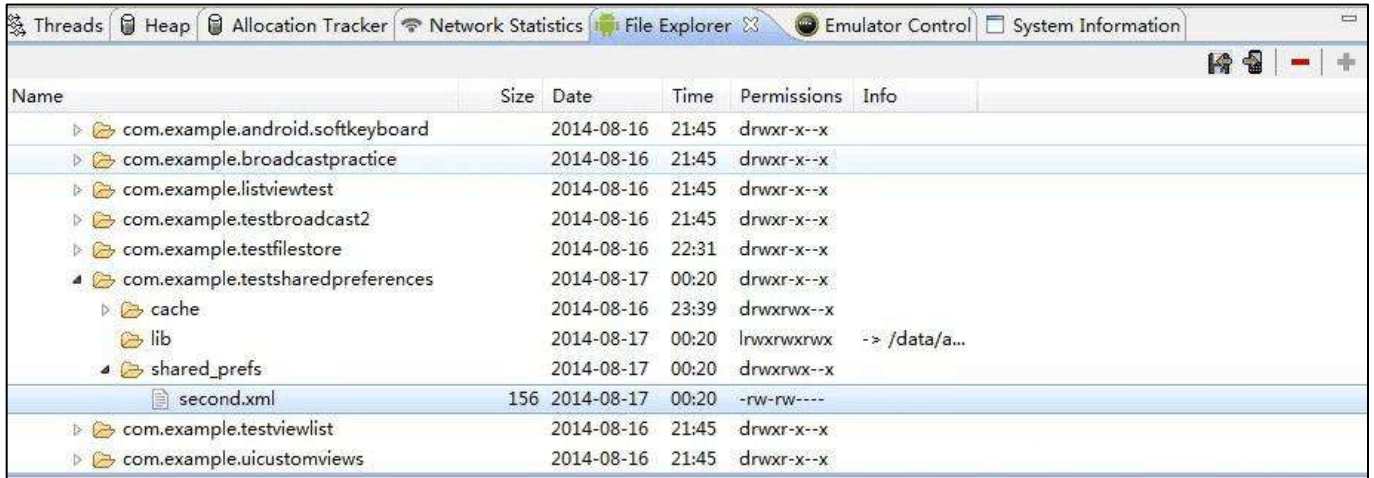
- 线程的安全性
  - 线程安全：指多个线程在执行同一段代码的时候采用加锁机制，使每次的执行结果和单线程执行的结果都是一样的，不存在执行结果的二义性。
  - 线程不安全：指不提供加锁机制保护，有可能出现多个线程先后更改数据造成所得到的数据是脏数据。
  - 什么情况下会导致线程不安全：多个线程同时写全局变量或静态变量。
  - 解决线程安全性的方法：锁机制、同步代码块、同步方法

```
//第 1 步：获取 SharedPreferences 对象
SharedPreferences sp = getSharedPreferences("config", Context.MODE_PRIVATE);

//第 2 步：写入数据（通过 Editor）
SharedPreferences.Editor editor = sp.edit();
editor.putString("username", "张三");
editor.putInt("age", 25);
editor.putBoolean("isVip", true);
editor.apply();  // 或 commit()

//第 3 步：读取数据（关键：读取需要提供默认值（防 null））
String username = sp.getString("username", "");  // 第二个参数是默认值
int age = sp.getInt("age", 0);
boolean isVip = sp.getBoolean("isVip", false);
```

```
public class TestSharedPreferences extends Activity {
    @Override
  public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main);
    SharedPreferences mSharedPreferences = getSharedPreferences("TestSharedPreferences", 0);
    int counter = mSharedPreferences.getInt("counter", 0);
    SharedPreferences.Editor mEditor = mSharedPreferences.edit();
    mEditor.putInt("counter", ++counter);
    mEditor.commit();
    TextView mTextView = (TextView)findViewById(R.id.textview);
    mTextView.setText("This app has been started " + counter + " times.");
  }
}
```




```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="name">lois</string>
    <int name="age" value="21" />
</map>
```

```
//保存用户信息
private void saveUserInfo(){
    SharedPreferences userInfo = getSharedPreferences(PREFS_NAME, MODE_PRIVATE);
    SharedPreferences.Editor editor = userInfo.edit();//获取Editor
    //得到Editor后，写入需要保存的数据
    editor.putString(“username”, “小红");
    editor.putInt("age", 20); editor.commit();//提交修改
    Log.i(TAG, "保存用户信息成功");
}
//读取用户信息
private void getUserInfo(){
    SharedPreferences userInfo = getSharedPreferences(PREFS_NAME, MODE_PRIVATE);
    String username = userInfo.getString("username", null);//读取username
    int age = userInfo.getInt("age", 0);//读取age
    Log.i(TAG, "读取用户信息");
    Log.i(TAG, "username:" + username + ",  age:" + age);
}
//移除年龄信数据
private void removeUserInfo(){
    SharedPreferences userInfo = getSharedPreferences(PREFS_NAME, MODE_PRIVATE);
    SharedPreferences.Editor editor = userInfo.edit();//获取Editor
```

- SharePeferences.Editor对象（简称SE对象）常用方法
  - boolean commit()：同步提交新值，替换当前SE对象中的任何内容。当新值被成功写入本地数据库时，返回 true。（apply()先将改变提交到内存的SharedPreferences中，然后再异步写入本地数据库）
  - void apply()：异步提交新值，替换当前SE对象中的任何内容。没有返回值。
  - Editor clear()：删除SE对象中所有值。返回SE对象。
  - Editor remove(String key)：删除指定key的单个值。返回SE对象。

说明：如果有两个SharePeferences.Editor对象同时修改同一个SharePeferences文件对象，最后一个调用commit()方法或apply()方法的 SE起作用

SE中多个方法连用：mEditor.putString("firstAdd", "first add by jere\n").clear().putString("secondAdd", "second add by jere !").commit();

底层结构图

```
App 私有目录
└── /data/data/<PackageName>/
    └── shared_prefs/
        ├── config.xml          ← getSharedPreferences("config", ...)
        ├── MainActivity.xml    ← getPreferences(...)
        └── 包名_preferences.xml ← getDefaultSharedPreferences(...)
```

//每个文件的内容（XML）：

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="username">张三</string>
    <int name="age" value="25" />
    <boolean name="isVip" value="true" />
</map>
```

- 读写SD卡数据
  - 调用Environment的getExternalStorageState()方法，同时判断是否插入SD卡以及是否具有SD卡读写权限，

    Environment.getExternalStorageState().equals(Environment.MEDIA_MOUNTED)
  - 调用Environment的getExternalStorageDirectory()方法获取SD卡的文件目录
  - 使用FileInputStream，FileOutputStream等内置函数读写SD卡的文件

```
//在SD卡创建和删除文件权限
<uses-permission android:name="android.permission.MOUNT_UNMOUNT_FILESYSTEMS"></uses-permission>
//向SD卡中写入数据权限
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"></uses-permission>
```

```
//向sdcard写文件
private void onWrite(String pathName, String fileName, String writeContent) {
    try {
        if (Environment.getExternalStorageState().equals(Environment.MEDIA_MOUNTED)) {
            File path = new File(Environment.getExternalStorageDirectory().getAbsolutePath() + "/" + pathNa
            if (!path.exists()) {
                path.mkdirs();//创建目录
            }
            File f = new File(Environment.getExternalStorageDirectory().getAbsolutePath() + "/" + pathName
"/" + fileName);
            if (!f.exists()) {
                f.createNewFile();//创建文件
            }
            OutputStreamWriter osw = new OutputStreamWriter(new FileOutputStream(f));
            //写文件，从EditView获得文本值
            osw.write(writeContent);
            osw.close();
            Toast.makeText(MainActivity.this,"创建成功",Toast.LENGTH_SHORT).show();
        }
    } catch (Exception e) {
        Toast.makeText(MainActivity.this,"创建失败",Toast.LENGTH_SHORT).show();
```

```
//读取文件夹/文件中的内容
private String read(String fileName) {
    String str = "内容为空";
    if (Environment.getExternalStorageState().equals(Environment.MEDIA_MOUNTED)) {
        File f = new File(Environment.getExternalStorageDirectory().getAbsolutePath() + "/" + fileName);
        Reader read;
        char[] b = null;
        int len = 0;
        if (f.exists()) {
            try {
                read = new InputStreamReader(new FileInputStream(f));
                b = new char[1024];
                len = read.read(b);
                read.close();
                str = new String(b, 0, len);
            } catch (FileNotFoundException e) {
                e.printStackTrace();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
    return str;
}
```

```
//删除自定义下文件夹下中的文件
public void deleteCacheFile(Filefile){
    if(Environment.getExternalStorageState().equals(Environment.MEDIA_MOUNTED)){
        if(file.exists()){
            if(file.isFile()){//文件直接删除
                file.delete();
            }
            if(file.isDirectory()){//文件夹
                File[] childfFiles = file.listFiles();
                if(childfFiles == null||childfFiles.length == 0){//文件夹不存在子文件（文件夹）
                    file.delete();
                }else {//文件夹中存在文件（子文件）
                    for (File chilFile : childfFiles){
                        deleteCacheFile(chilFile);
                    }
                    file.delete();
                }
            }
            Toast.makeText(MainActivity.this,"删除文件成功",Toast.LENGTH_SHORT).show();
        }
    }
}
```

- 数据库SQLite

SQLite是一个轻量级的关系型数据库，运算速度快，占用资源少。

- 应用场景：多线程并发访问、复杂数据结构
- 支持类型：NULL、INTEGER、REAL(浮点数)、TEXT(字符串文本)、BLOB(二进制对象)等五类。但可以用任何字段保存各种数据类型，不用考虑类型匹配（除主键必须是64位整数）
- 组成结构：一个文件（数据库）包含多个表格；一个表格包含 多条记录；一条记录包含多个字段；一个字段包含对应的值（每个值可以指定类型，也可以不指定类型，主键除外）

对比项	SQLite	MySQL	Oracle
部署	嵌入式	独立服务器	独立服务器
使用场景	移动端	Web 后端	大型企业
占用资源	极少	中	大
跨进程	单机	网络	网络

- 创建SQLite数据库（续）

  创建数据库主要分两步：

  - 第一步：自定义一个继承SQLiteOpenHelper的数据库子类
    - S1-1：自定义一个类继承SQLiteOpenHelper类
    - S1-2：在该类的构造方法的super中设置好要创建的数据库名，版本号
    - S1-3：重写onCreate( )方法创建表结构
    - S1-4：重写onUpgrade()方法定义版本号发生改变后执行的操作
  - 第二步：利用自定义的数据库子类实例化一个数据库对象
    - S2-1：利用数据库对象调用getWritableDatabase()或getReadableDatabase()方法

说明：创建的数据库文件默认在/data/data/<package name>/databases/目录下.

Cursor的理解

游标，类似于JDBC里的resultset，是一个结果集，可以将其理解为是指向数据库中某 一个记录的指针!

- 创建SQLite数据库（续）

继承SQLiteOpenHelper抽象类

- 第一个参数：Context类型，上下文对象
- 第二个参数：String类型，数据库的名称
- 第三个参数：CursorFactory类型，如果该参数值为null，则使用标准的SQLiteCursor行为。否则，可以通过扩展Cursor类实现一个自定义的Cursor(对查询结果做额外操作)。
- 第四个参数：int类型，数据库版本

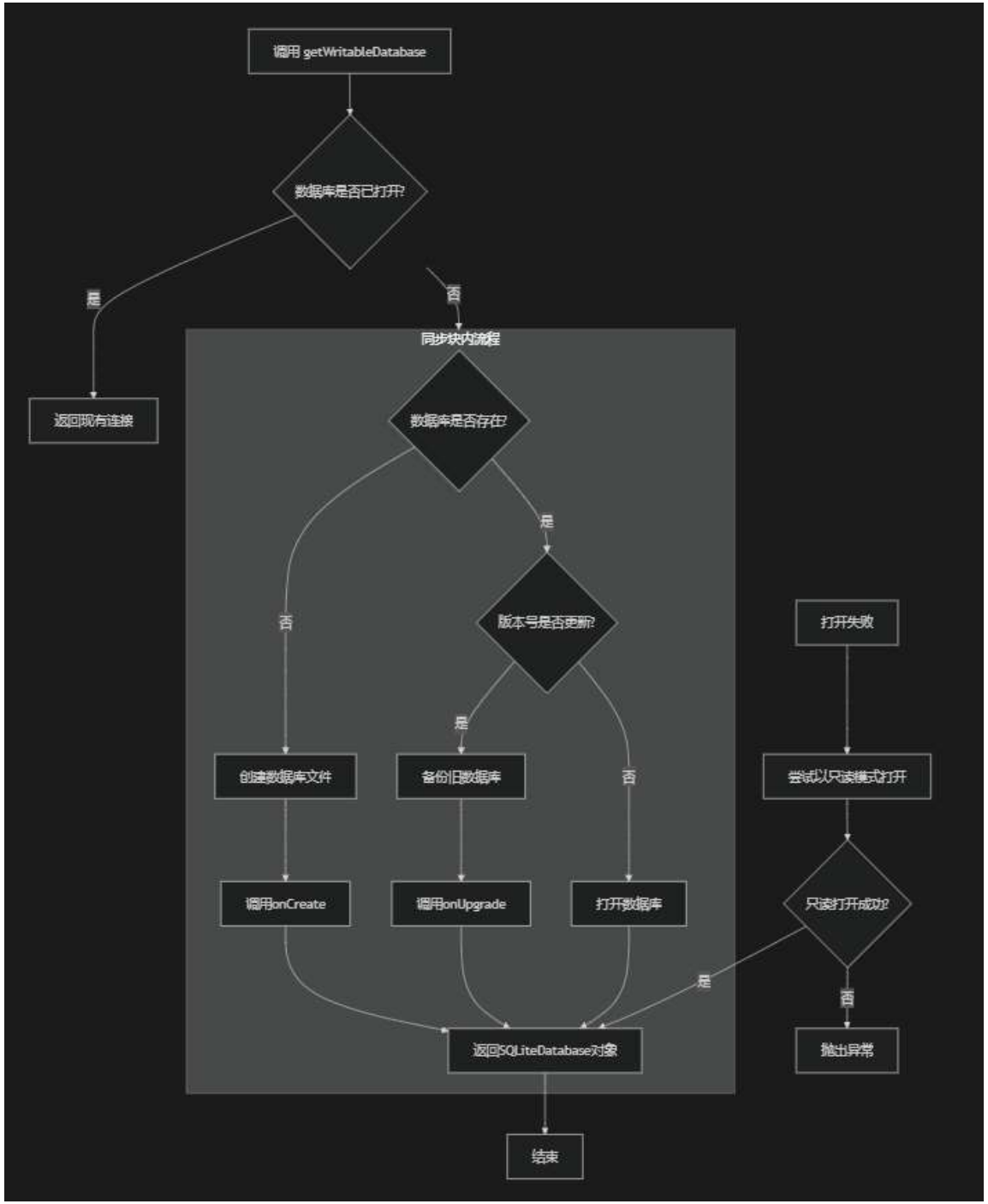
```
public SQLiteOpenHelper(Context context,String name,CursorFactory factory,int version);
```

- 创建SQLite数据库（续）

SQLiteOpenHelper中的常用方法：

- onCreate()：创建数据库，首次创建数据库时自动执行（只被调用一次）
- onUpgrade()：升级数据库，输入版本号高于当前版本号时被自动执行
- getReadableDatabase()：创建或打开一个现有数据库，返回一个可对数据库读写操作的对象。当数据库不可写入时，以只读方式打开
- getWritableDatabase()：创建或打开一个现有数据库，返回一个可对数据库读写操作的对象。当数据库不可写入时，出现异常。

操作数据库前，一定要通过数据库子类调用getWritableDatabase()或getReadableDatabase()，以获得该数据库的对象。


调用 getWritableDatabase
数据库是否已打开?
是
否
返回现有连接
同步块内流程
数据库是否存在?
是
否
版本号是否更新?
是
否
创建数据库文件
备份旧数据库
调用onCreate
调用onUpgrade
打开数据库
返回SQLiteDatabase对象
打开失败
尝试以只读模式打开
只读打开成功?
是
否
抛出异常
结束

```
//自定义SQLite数据库字类
public class MyDatabaseHelper extends SQLiteOpenHelper {
    public static final String CREATE_BOOK = "create table Book ("
        + "id integer primary key autoincrement, "
        + "author text, "
        + "price real, "
        + "pages integer, "
        + "name text)";
    private Context mContext;
    public MyDatabaseHelper(Context context, String name,
                            SQLiteDatabase.CursorFactory factory, int version) {
        super(context, name, factory, version);
        mContext = context;
        }//构造方法，  调用父类四参数构造函数
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(CREATE_BOOK);
        Toast.makeText(mContext, "Create succeeded", Toast.LENGTH_SHORT).show();
    } // 数据库第一次创建时调用
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    } // 版本号变化时调用，根据 oldVersion 和 newVersion 决定如何升级
}
```

```
public class MainActivity extends AppCompatActivity{
    private MyDBOpenHelper dbHelper;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

- SQLite数据库中的SQL语句
  - execSQL(SQL,Object[])：使用带占位符的SQL语句，它是执行修改数据库内容的sql语句用的
  - rawQuery(SQL,Object[])：使用带占位符的SQL查询操作
  - move(offset)：指定向上或者向下移动的行数，正数表示向下移动，负数表示向上移动
  - moveToFirst()：指针移动到第一行。成功返回true，表示数据库中含有数据
  - moveToLast():指针移动到最后一样。成功返回true
  - moveToNext()：指针移动到下一行。成功返回true，表示数据库中含有数据
  - moveToPrevious()：移动到上一条记录
  - getCount()：获得数据表中的记录总条数
  - isFirst()：是否是第一条记录
  - isLast()：是否是最后一条记录
  - moveToPosition(int)：移动到指定行

- 更新数据库

```
public class MyDatabaseHelper extends SQLiteOpenHelper {
    public static final String CREATE_BOOK = "create table Book ("
        + "id integer primary key autoincrement, "
        + "author text, "
        + "price real, "
        + "pages integer, "
        + "name text)";
    private Context mContext;
    public MyDatabaseHelper(Context context, String name,
                            SQLiteDatabase.CursorFactory factory, int version) {
        super(context, name, factory, version);
        mContext = context;
        }//构造方法
    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL(CREATE_BOOK);
        db.execSQL(CREATE_CATEGORY);
        Toast.makeText(mContext, "Create succeeded", Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    }
}
```

能更新成功吗？

- 更新数据库（续）
  - 自动调用onCreate()方法：如果数据库已经存在，必须先删除数据库（清空所有表）
  - 自动调用onUpgrade()方法：新版本号必须高于当前版本号

```
//onUpgrade 必须处理升级逻辑
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    if (oldVersion < 2) {
        db.execSQL("CREATE TABLE Category ("
            + "id integer primary key autoincrement, "
            + "name text)");
    }
    // 以后升级到 version 3 再加
    // if (oldVersion < 3) { ... }
}
```

```
//暴力升级
public class MyDatabaseHelper extends SQLiteOpenHelper {
    ...
    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int
    newVersion) {
        // 删除旧表
        db.execSQL("DROP TABLE IF EXISTS Book");
        db.execSQL("DROP TABLE IF EXISTS Category");
        // 重新创建
        onCreate(db);
    }
}
```

- 添加数据（insert()方法）

```
public class MainActivity extends AppCompatActivity {
    private MyDatabaseHelper dbHelper;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        dbHelper = new MyDatabaseHelper(this, "BookStore.db", null, 2);
        Button createDatabase = (Button) findViewById(R.id.create_database);
        createDatabase.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                SQLiteDatabase db = dbHelper.getWritableDatabase();
                ContentValues values = new ContentValues();
                // 第一条数据
                values.put("name", "The Da Vinci Code");
                values.put("author", "Dan Brown");
                values.put("pages", 454);
                values.put("price", 16.96);
                db.insert(“Book”, null, values); // 插入第一条数据，values
                有数据，第二个参数用不上
                values.clear();
            }
        });
```

inset()方法

- 第一个参数：操作的数据库表名
- 第二个参数：在未指定添加数据的情况下给某些可为空的列自动赋值为NULL，一般不用设置，直接设置为null
- 第三个参数：是一个ContentValues对象（类似于键值对存储方式），提供一系列put()方法重载，用于向ContentValues中添加数据

insert(String table, String nullColumnHack, ContentValues values)

- 更新数据（update()方法）

```
public class MainActivity extends AppCompatActivity {
    private MyDatabaseHelper dbHelper;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        dbHelper = new MyDatabaseHelper(this, "BookStore.db", null, 2);
        Button updateData = (Button) findViewById(R.id.update_data);
        updateData.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                SQLiteDatabase db = dbHelper.getWritableDatabase();
                ContentValues values = new ContentValues();
                values.put("price", 10.99);
                db.update("Book", values, "name = ?", new String[] { "The Code" });
            }
        });
    }
}
```

update()方法

- 第一个参数：操作的数据库表名
- 第二个参数：是一个ContentValues对象，存入要更新的数据
- 第三个和第四个参数：设置约束条件，更新某一行或某几行的数据，默认就是更新所有行。（相当于sql语句中的where ）

update(String table, ContentValues values, String whereClause, String[] whereArgs)

- 删除数据（delete()方法）

```
public class MainActivity extends AppCompatActivity {
    private MyDatabaseHelper dbHelper;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        dbHelper = new MyDatabaseHelper(this, "BookStore.db", null, 2);
        Button deleteButton = (Button) findViewById(R.id.update_data);
        deleteButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                SQLiteDatabase db = dbHelper.getWritableDatabase();
                db.delete("Book", "pages > ?", new String[] { "500" });
            }
        });
    }
}
```

delete()方法

- 第一个参数：操作的数据库表名
- 第二个和第三个参数：设置约束条件，更新某一行或某几行的数据，默认就是更新所有行。（相当于sql语句中的where ）

- 查询数据（query()方法）

query()方法返回一个Cursor对象，查询到的所有数据都从该对象中取出，query()方法的主要参数如表所示。

query()方法参数	对应SQL部分	描述	如果不指定该参数
table	from table_name	指定查询的表名	
columns	select column1, column2	指定查询的列名	默认查询所有列
selection	where column = value	指定where 的约束条件	默认查询所有行
selectionArgs	-	为where 中的占位符提供具体的值	
groupBy	group by column	指定需要group by 的列	默认不执行groupby操作
having	having column = value	对group by 后的结果进一步约束	默认不执行过滤操作
orderBy	order by column1, column2	指定查询结果的排序方式	默认不执行排序操作

```
public class MainActivity extends AppCompatActivity {
    private MyDatabaseHelper dbHelper;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        dbHelper = new MyDatabaseHelper(this, "BookStore.db", null, 2);
        Button queryButton = (Button) findViewById(R.id.query_data);
        queryButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                SQLiteDatabase db = dbHelper.getWritableDatabase();
                // 查询Book表中所有的数据
                Cursor cursor = db.query("Book", null, null, null, null, null, null);
                if (cursor.moveToFirst()) {
                    do {
                        // 遍历Cursor对象，取出数据并打印
                        String name = cursor.getString(cursor.getColumnIndex("name"));
                        String author = cursor.getString(cursor.getColumnIndex("author"));
                        int pages = cursor.getInt(cursor.getColumnIndex("pages"));
                        double price = cursor.getDouble(cursor.getColumnIndex("price"));
                        Log.d("MainActivity", "book name is " + name);
                        Log.d("MainActivity", "book author is " + author);
                        Log.d("MainActivity", "book pages is " + pages);
                        Log.d("MainActivity", "book price is " + price);
                    } while (cursor.moveToNext());
                }
```

- 数据库SQLite数据库创建过程

1）创建数据库子类，继承自定义的SQLiteOpenHelper数据库子类：重构onCreat()、onUpgrade()方法

2）创建数据库对象，利用自定义的数据库子类，调用getWritableDatabase()或getReadableDatabase()，获得数据库操作对象

3）操作数据库：调用各种内置方法，实现增、删、改、查等操作

- 事务在数据库中的使用
  - 应用场景：多线程并发操作数据库，导致数据不一致问题；或者同时操作大量数据，无法保证所有数据都被执行正确
  - 解决方法：采用事务
  - 事务定义：是应用程序中一系列严密的操作，所有操作必须成功完成，否则在每个操作中所作的所有更改都会被撤消。也就是事务具有原子性，一个事务中的一系列的操作要么全部成功，要么一个都不做。
  - 事务结果：提交或回滚。当事务中的所有操作被全部成功执行时，事务提交。如果其中一个操作失败，将发生回滚，撤消之前到事务开始时的所有操作。

```
public void payment() {
    SQLiteDatabase db = dbOpenHelper.getWritableDatabase();
    //开启事务
    db.beginTransaction();
    try {
        db.execSQL("update person set amount=amount-10 where personid=?", new Object[]{1});
        db.execSQL("update person set amount=amount+10 where personid=?", new Object[]{2});
        //设置事务标志为成功，当结束事务时就会提交事务
        db.setTransactionSuccessful();//设置事务正确结束，如果此方法不执行，则所有数据库改操
作都会回滚
    }
   catch (Exception e) {
        throw(e);
    }
    finally
    {
        //结束事务
        db.endTransaction();
    }
}
```

- 数据库SQLite读写图片

  两种实现方法：

  - 方法1：保存图片所在路径，实现简单，但备份效率较低
  - 方法2：将图片以二进制的形式存储（BLOB数据类型），读写数据库效率低，但便于迁移

```
//bitmap保存到SQLite 中 数据格式:
db.execSQL("Create table express ( _id INTEGER PRIMARY KEY AUTOINCREMENT,express_no varchar(100),
express_name TEXT,express_img BLOB );");
```

```
//将图片bitmap转换为Blob类型
ContentValues values = new ContentValues();
final ByteArrayOutputStream os = new ByteArrayOutputStream();
bmp.compress(Bitmap.CompressFormat.PNG, 100, os); //将图片压缩为png格式，100表示按原图大小压缩
values.put("express_img", os.toByteArray());
values.put("express_name","zf");
values.put("express_no","zf");
getContentResolver().insert("express", values);
```

```
//从SQLite中读取Bitmap
byte[] in=cur.getBlob(cur.getColumnIndex("express_img"));
bmpout=BitmapFactory.decodeByteArray(in,0,in.length);
```

广播机制

- 广播

  本质是实现数据通信。

  - Android数据通信
    - Activity方式：startActivity(Intent intent)
    - Service方式：startService()和bindService()
    - Broadcast方式：广播方式
    - Content provider方式：数据共享

为什么需要Broadcast方式?

- 高内聚低耦合性思想
- 不需要知道发送者和接受者

```
Intent intent = new Intent(this, SecondActivity.class);  // 必须知道目标类名
startActivity(intent);
```

```
Intent intent = new Intent(this, MyService.class);      // 必须知道目标类名
startService(intent);
```

```
Intent intent = new Intent("com.example.MY_ACTION");
// 只声明 action
sendBroadcast(intent);
```

广播机制

- 广播和binder
  - 都可以实现进程之间的通讯。

维度	广播	Binder
通信模型	发布-订阅	客户端-服务器 (C/S)
通信方向	单向（发送者 -> 接收者）	双向（客户端可调用，服务器可返回）
通信关系	多对多（一个广播可被多个接收者处理）	一对一（一个连接对应一个服务端）
耦合性	松耦合：发送者和接收者互不知晓对方的存在。	紧耦合：客户端需要知道服务端的接口（AIDL）。
性能开销	大：涉及Intent的创建、序列化、系统匹配和多个接收者的启动。	小：直接的内存映射，是Android最高效的IPC机制。
主要用途	系统事件通知、应用内组件间解耦通信、应用间简单的信息传递。	跨进程服务调用、应用向系统服务请求、需要获取返回值的复杂操作。
典型代表	Intent， BroadcastReceiver	Activity/Service的启动， AIDL， 系统服务（如LocationManager）

- 网络中的广播
  - 同一个IP网络范围的设备都能收到广播消息
  - 网段中最大的地址是广播地址

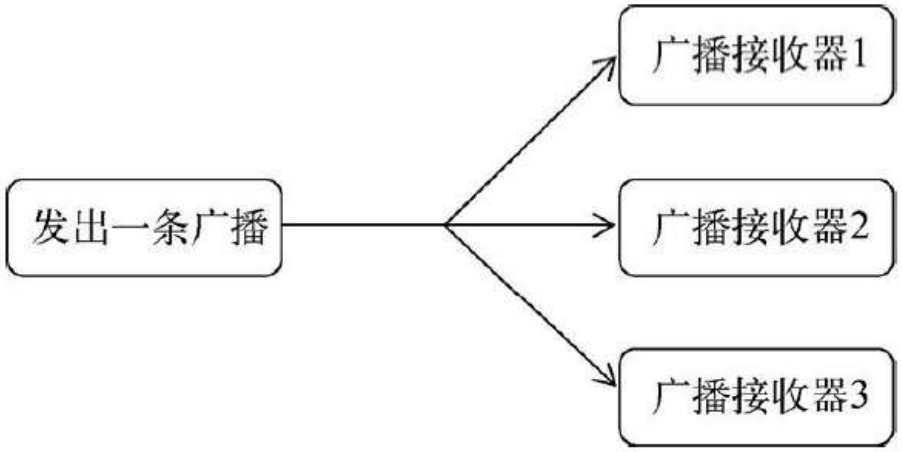
IP地址：192.168.0.XXX

子网掩码：255.255.255.0

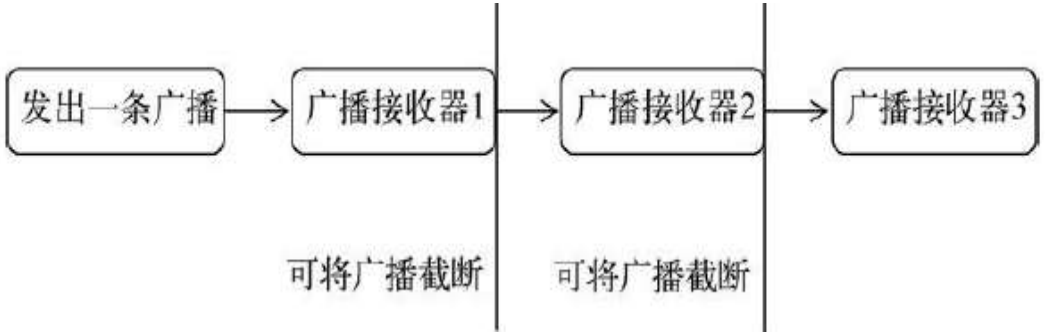
广播地址：192.168.0.255

广播机制

- Android中的广播
  - 广播发送器：Intent
  - 广播接收器：Broadcast Receiver（借助IntentFilter）
  - 标准广播：异步执行，同时接收，无优先级，不可截断
  - 有序广播：同步执行，按序接收，有优先级，可以截断



标准广播



有序广播

接收系统广播

- 广播接收器

  广播接收器是一种基于消息驱动模型的消息订阅/发布机制

  - 创建广播接收器：在活动类中定义接收广播的内部类
    - 首先，继承BroadcastReceiver类
    - 然后，重写父类的onReceive()方法（触发广播时，系统会自动调用并执行该方法）
  - 注册广播接收器
    - 动态注册：直接在活动类中调用registerReceiver()方法，注册广播接收器
    - 静态注册：在AndroidManifest.xml中注册广播接收器
  - 销毁广播接收器
    - 重写onDestroy()方法，调用unregisterReceiver()方法

接收系统广播

```
//通过广播监听网络变化
public class MainActivity extends AppCompatActivity {
    private IntentFilter intentFilter;
    private NetworkChangeReceiver networkChangeReceiver;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        intentFilter = new IntentFilter();
        intentFilter.addAction("android.net.conn.CONNECTIVITY_CHANGE");
        networkChangeReceiver = new NetworkChangeReceiver();
        registerReceiver(networkChangeReceiver, intentFilter);
    }
    @Override
        protected void onDestroy() {
        super.onDestroy();
        unregisterReceiver(networkChangeReceiver);
    }
    class NetworkChangeReceiver extends BroadcastReceiver {
        @Override
        public void onReceive(Context context, Intent intent) {
            Toast.makeText(context, "network changes",
                           Toast.LENGTH_SHORT).show();
        }
    }
}
```

广播接收器过程分析

- 系统发出某个广播：网络变化的广播名称为android.net.conn.CONNECTIVITY_CHANGE
- 广播接收器添加需要监听的广播动作：intentFilter.addAction()
- 动态注册广播接收器：调用registerReceiver()方法将广播接收器注册到ActivityManagerService中（它负责系统中所有广播的注册和发布操作）
- 动态取消注册广播接收器：调用unregisterReceiver()方法（Home键有效，Back键无效）

阶段一：注册监听器（订阅事件） - onCreate()

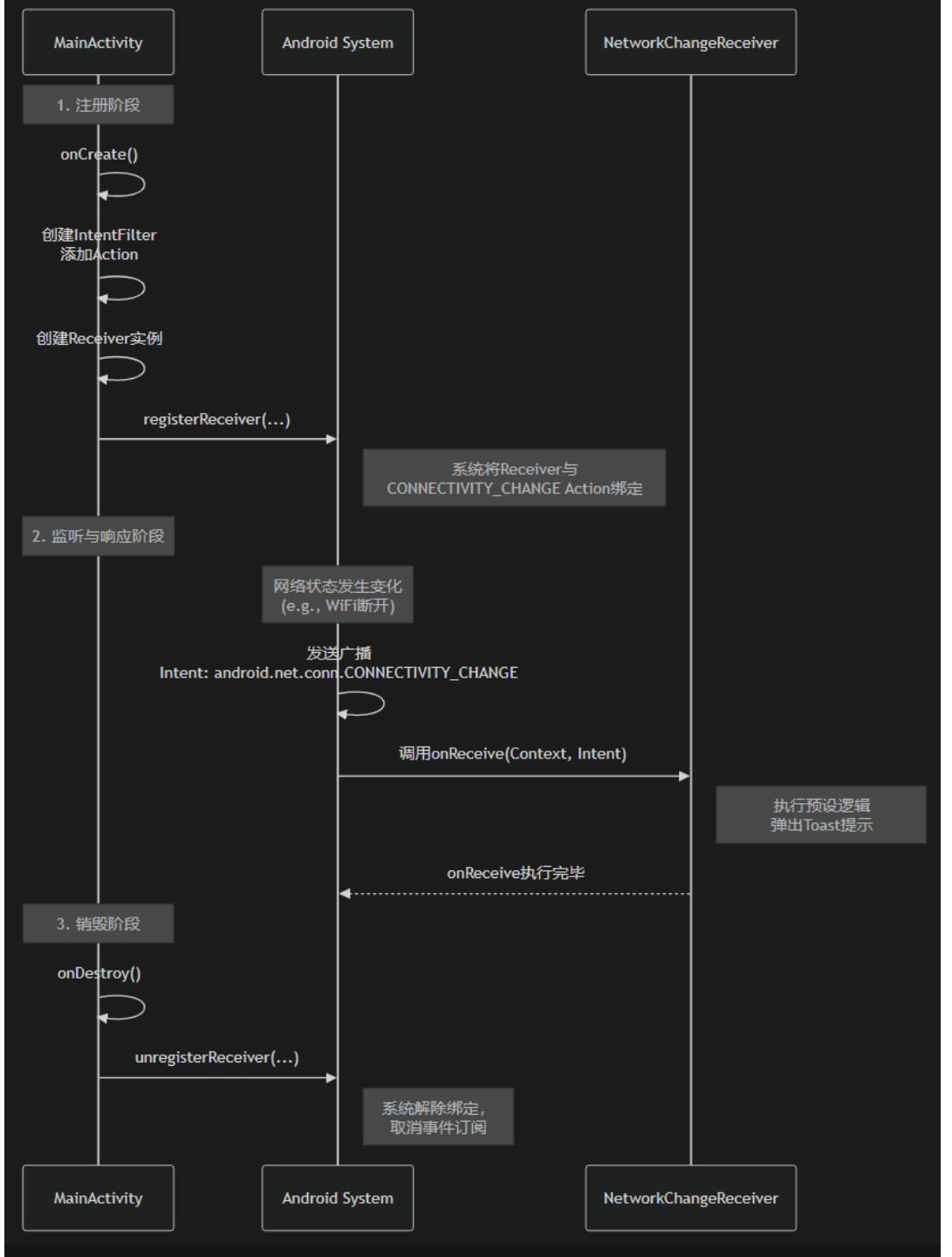
- 当 Activity 启动时，在 onCreate 方法中完成广播接收器的初始化与注册

阶段二：接收并处理广播（响应事件） - onReceive()

- 当网络状态真的发生变化时（例如用户打开或关闭了飞行模式，Wi-Fi 断开或连接）

阶段三：取消注册（退订事件） - onDestroy()

- 当 Activity 被销毁时（用户按下返回键或屏幕旋转等配置变化）

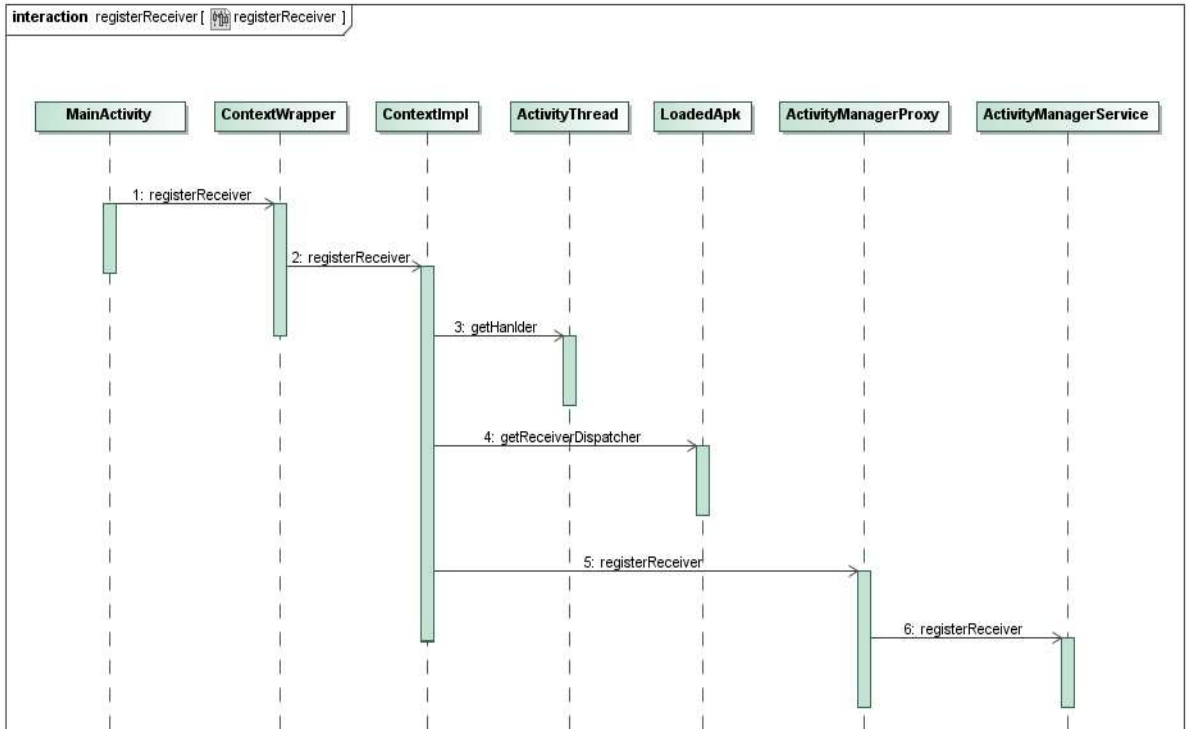


接收系统广播

- 动态注册广播接收器

  广播接收器可以订阅感兴趣的广播

  - 订阅机制
    - 系统通过IntentFilter设置订阅的广播类型，实现监听特定广播
  - 注册机制
    - 系统调用registerReceiver()将创建的广播接收器注册到ActivityManagerService
    - ActivityManagerService负责系统中所有广播的注册和发布



接收系统广播

```
//通过广播监听网络状态，是否连接
public class MainActivity extends AppCompatActivity {
...
    class NetworkChangeReceiver extends BroadcastReceiver {
        @Override
        public void onReceive(Context context, Intent intent) {
            ConnectivityManager connectionManager = (ConnectivityManager)
                getSystemService(Context.CONNECTIVITY_SERVICE);//网络连接服务
            NetworkInfo networkInfo = connectionManager.getActiveNetworkInfo();
            if (networkInfo != null && networkInfo.isAvailable()) {
                Toast.makeText(context, "network is available",
                               Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(context, "network is unavailable",
                               Toast.LENGTH_SHORT).show();
        }
        }
    }
}
```

广播接收器onReceive()方法中判断网络状态的逻辑

- 调用getSystemService()方法：获得指定的系统服务类对象ConnectivityManager
- ConnectivityManager：专用于负责管理网络连接，可以判断当前网络状态
- 调用getActiveNetworkInfo()方法：获取当前可用网络

说明：getSystemService()方法是Android很重要的一个API，它是Activity的一个方法，根据传入的NAME来取得对应的Object，然后转换成相应的服务对象。（引自：https://www.cnblogs.com/gaopeng527/p/4595901.html）

补充知识

- ConnectivityManager类

主要负责查询网络连接状态以及在连接状态有变化的时候发出通知。

- 监视网络状态，包括（Wi-Fi、GPRS、UMTS等）
- 当网络状态发生变化时发送广播通知
- 当网络连接失败后会尝试连接其他网络
- 为App提供API，用于获取网络状态信息等

公有方法	
NetworkInfo	getActiveNetworkInfo ()
NetworkInfo[]	getAllNetworkInfo ()
boolean	getBackgroundDataSetting () Returns the value of the setting for background data usage.
NetworkInfo	getNetworkInfo (int networkType)
int	getNetworkPreference ()
static boolean	isNetworkTypeValid (int networkType)
boolean	requestRouteToHost (int networkType, int hostAddress) Ensure that a network route exists to deliver traffic to the specified host via the specified network interface.
void	setNetworkPreference (int preference)
int	startUsingNetworkFeature (int networkType, String feature) Tells the underlying networking system that the caller wants to begin using the named feature.
int	stopUsingNetworkFeature (int networkType, String feature) Tells the underlying networking system that the caller is finished using the named feature.

接收系统广播

- 注意访问敏感操作问题

  必须在配置文件中声明权限，否则程序直接崩溃

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.broadcasttest">
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    ...
</manifest>
```

补充知识

广播和Intent

发送者（系统）
需要创建和发送Intent

```
// 系统内部代码（你不需要写）
public class AndroidSystem {
    public void onNetworkChanged() {
        // 系统创建Intent对象
        Intent broadcastIntent = new Intent("android.net.conn.CONNECTIVITY_CHANGE");
        broadcastIntent.putExtra("networkInfo", getCurrentNetworkInfo());

        // 系统发送广播
        sendBroadcast(broadcastIntent);
    }
}
```

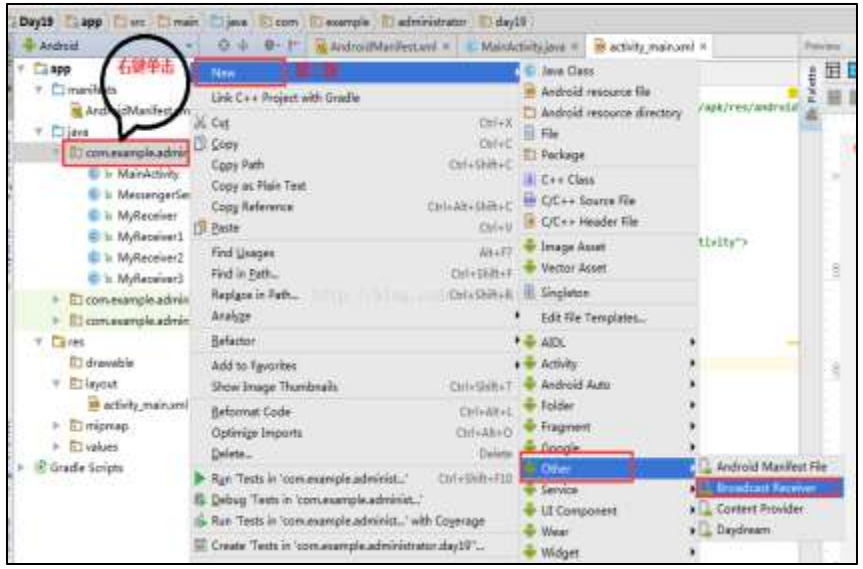
接收者（你的App）
需要创建IntentFilter，
接收Intent

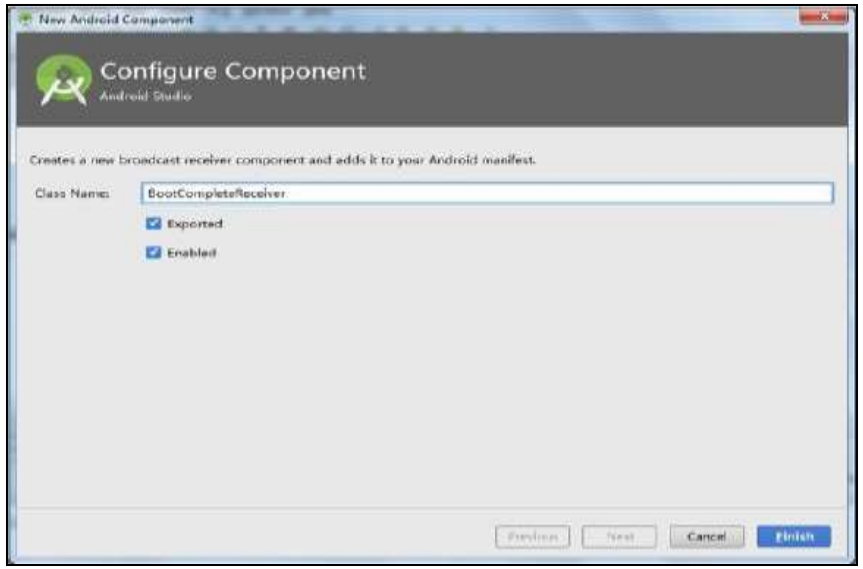
```
//通过广播监听网络变化
public class MainActivity extends AppCompatActivity {
    private IntentFilter intentFilter;
    private NetworkChangeReceiver networkChangeReceiver;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        intentFilter = new IntentFilter();
        intentFilter.addAction("android.net.conn.CONNECTIVITY_CHANGE");
        networkChangeReceiver = new NetworkChangeReceiver();
        registerReceiver(networkChangeReceiver, intentFilter);
    }
}
```

静态注册

- 静/动态注册广播接收器
  - 动态注册广播接收器：
    - 原理：在Activity文件的onCreat()方法中注册广播接收器
    - 条件：敏感操作要在配置文件中声明权限
    - 缺点：必须程序启动后才能接收广播
  - 静态注册广播接收器
    - 原理：在继承BroadcastReceiver类的配置文件AndroidManifest.xml中注册广播接收器
    - 条件：所有操作都要在配置文件中声明权限
    - 优点：实现开机广播

- 创建广播接收器（快捷方式创建）
  - 右击包→New→Other→Broadcast Receiver
  - 勾选Exported和Enabled
    - Exported 属性表示是否允许该广播接收器接收本程序以外的广播
    - Enabled 属性表示是否启用该广播接收器





- 创建广播接收器（代码直接创建）

```
public class BootCompleteReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Boot Complete", Toast.LENGTH_LONG).show();
    }
}
```

注意：不要在广播文件的onReceive()方法中添加过多逻辑功能或耗时操作。由于onReceive()不允许开启线程，这使得它运行较长时间而不结束时将会报错（ANR，Application Not responding）。

方法：利用intent将复杂的处理逻辑移交给服务，转到后台去做。

- 静态注册广播接收器

  静态广播接收器一定要在AndroidManifest.xml文件中注册才可以使用

  - Android Studio快捷方式创建广播接收器：自动实现静态注册
  - 代码直接创建广播接收器：手动完成静态注册
  - 注册步骤
    - 添加<receiver>标签：注册指定的广播接收器名
    - 添加<intent-filter>标签：设置需要监听的广播名
    - 添加<uses-permission>标签：声明访问该广播的权限

```
//静态注册广播接收器
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.broadcasttest">
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        ...
        <receiver
            android:name=".BootCompleteReceiver"
            android:enabled="true"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
            </intent-filter>
        </receiver>
    </application>
</manifest>
```

- 自定义广播

Android不仅可以处理系统广播，还提供自定义广播。

- 类型：标准广播和有序广播
- 发送：采用Intent方法发送指定的广播名，通过调用sendBroadcast()方法发送intent实例
- 接收：创建广播接收器，静态注册或动态注册等两种方法
- 条件：和系统广播在配置文件中的注册方法不同（主要区别是不需要额外授权）
  - 动态注册的自定义广播不需要在配置文件中注册
  - 静态注册的自定义广播需要在配置文件中添加<receiver>标签和<intent-filter>标签，但不需要添加<uses-permission>标签

- 发送自定义的标准广播

```
//发送自定义的标准广播
public class MainActivity extends AppCompatActivity {
    ...
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button button = (Button) findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent("com.example.broadcasttest.MY_BROADCAST");
                sendBroadcast(intent);
            }
        });
        ...
    }
    ...
}
```

- 接收自定义的标准广播

```
//创建自定义的标准广播接收器
public class MyBroadcastReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "received in MyBroadcastReceiver",
                       Toast.LENGTH_SHORT).show();
    }
}
```

```
//静态注册自定义的标准广播接收器，设置要监听的广播名
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.broadcasttest">
    ...
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        ...
        <receiver
```

- 发送自定义的有序广播

```
//发送自定义的有序广播
public class MainActivity extends AppCompatActivity {
    ...
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button button = (Button) findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent("com.example.broadcasttest.MY_BROADCAST");
                sendOrderedBroadcast(intent, null);//发送有序广播
            }
        });
        ...
    }
    ...
}
```

- 接收自定义的有序广播
  - 特征：广播有优先级，传播有先后顺序，可以被拦截
  - 设置接收广播优先级：在配置文件的<intent-filter>标签设置android:priority属性值
    优先级越高，则该广播解释器优先接收到它指定的广播
  - 设置广播被拦截：在继承的BroadcastReceiver子类中调用abortBroadcast()方法

```
//静态注册自定义的有序广播接收器，设置优先级
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.broadcasttest">
    ...
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        ...
        <receiver
```

```
public class MyBroadcastR
    @Override
    public void onReceive
        Toast.makeText(co
                       To
        abortBroadcast();
    }
}
```

- 本地广播
  - 系统广播：全局的，不安全，效率较低
    - 任何应用程序之间都能互相接收和发送广播
    - “我”的广播接收器可以接收或拦截其他任何应用程序的广播
    - “我”的广播可以被其他任何应用程序接收或拦截
  - 本地广播：局部的，安全，效率较高
    - “我”的广播接收器只接收或拦截“我”的应用程序的广播
    - “我”的广播可以只能被“我”的应用程序接收或拦截

注意：本地广播只能在应用程序内部传递，并且无法通过静态注册方式接收

- 本地广播（续）
  - 原理：采用LocalBroadcastManager类管理、发送和注册本地广播
    - 创建本地广播接收器：在活动中创建接收本地广播的内部类
      - 首先，继承BroadcastReceiver类
      - 然后，重写onReceive()方法
    - 发送本地广播：采用Intent方法发送指定的广播名，通过调用LocalBroadcastManager.sendBroadcast()发送intent实例
    - 接收本地广播：动态注册本地广播接收器
      - 首先，调用intentFilter.addAction()添加指定接收的本地广播；
      - 然后，调用LocalBroadcastManager.registerReceiver()注册该本地广播

```
public class MainActivity extends AppCompatActivity {
    private IntentFilter intentFilter;
    private LocalReceiver localReceiver;
    private LocalBroadcastManager localBroadcastManager;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        localBroadcastManager = LocalBroadcastManager.getInstance(this); // 获取实例
        Button button = (Button) findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
            Intent intent = new Intent("com.example.broadcasttest.LOCAL_BROADCAST");
            localBroadcastManager.sendBroadcast(intent); // 发送本地广播
            }
        });
        intentFilter = new IntentFilter();
        intentFilter.addAction("com.example.broadcasttest.LOCAL_BROADCAST");
        localReceiver = new LocalReceiver();
        localBroadcastManager.registerReceiver(localReceiver, intentFilter); // 注册本地广播监听器
    }
    @Override
    protected void onDestroy() {
        super.onDestroy();
        localBroadcastManager.unregisterReceiver(localReceiver);
    }
```

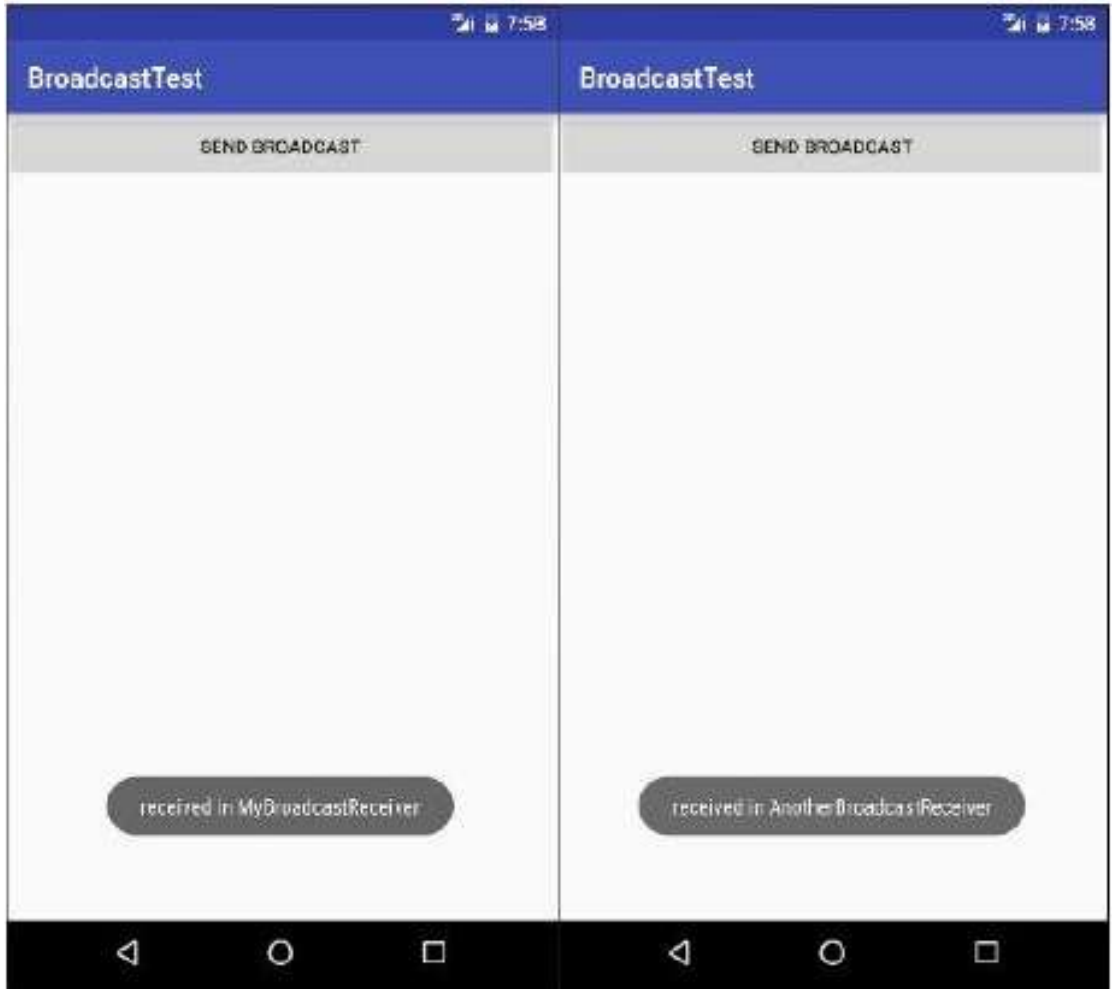
```
//在项目2中测试是否能接收到项目1发送的本地广播
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.broadcasttest2">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        ...
        <receiver
            android:name=".AnotherBroadcastReceiver"
            android:enabled="true"
            android:exported="true">
            <intent-filter>
                <action android:name="com.example.broadcasttest.LOCAL_BROADCAST" />
            </intent-filter>
        </receiver>
    </application>
</manifest>
```

- 广播特点
  - 特点1：跨进程通信

```
//新建一个项目
public class AnotherBroadcastReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "received in AnotherBroadcastReceiver",
        Toast.LENGTH_SHORT).show();
    }
}
```

```
//两个广播接收器监听同一个广播
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.broadcasttest2">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        ...
        <receiver
```

广播是跨进程的，某个应用程序发送的广播，其他应用程序都可以接收



两个项目同时接受到广播

com.example.broadcasttest.MY_BROADCAST

- 广播特点
  - 特点2：有序性

```
//发送自定义广播
public class MainActivity extends AppCompatActivity {
    ...
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button button = (Button) findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent("com.example.broadcasttest.MY_BROADCAST");
                sendOrderedBroadcast(intent, null);
            }
        });
        ...
    }
    ...
}
```

sendOrderedBroadcast(参数1，参数2)方法

- 参数1：要发送的Intent
- 参数2：与权限相关的字符串

- 广播的权限配置
  - 发送者权限：
    - 设置配置文件中<receiver>标签的属性android:permission
    - 要有该权限的广播接收器才能接收“我”发出的广播
  - 接收者权限：
    - 设置sendBroadcast()或sendOrderedBroadcast()中的第二个参数
    - 要有该权限的发送者发出的广播“我”才会接收

```
endBroadCastButton.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View arg0) {
        //sendOrderedBroadcast(new
        Intent("guangbo"),android.Manifest.permission.BLUETOOTH);
        sendBroadcast(new Intent("guangbo"), Manifest.permission.myper);
    }
```

```
<receiver android:name="com.wild.w
    android:permission="com.wild.w
    <intent-filter android:priorit
    <action android:name="guangbo"
</intent-filter>
</receiver>
```

例：强制下线

需求分析：关闭所有活动，回到登录界面

具体实现：创建所有活动的父类

发送强制下线的广播

在活动父类中创建广播接收器

- 静态注册：没办法显示对话框
- 动态注册：没办法在每个活动中添加广播接收器

内容提供器

- Content Provider

  实现不同程序之间的数据共享，并且保证被访数据的安全性。

  - 特点：可以选择需要共享的数据范围

几种共享数据的实现方法

- 文件存储：大量数据且结构简单，不能选择数据共享的范围
- SharedPreferences：少量数据且结构简单，不能选择数据共享的范围
- 数据库：大量数据且结构复杂，不能选择数据共享的范围
- 内容提供器：通常与数据库配合使用，可以选择数据共享的范围

- 运行时权限

可以在应用程序运行时动态管理权限（授权/禁止）。Android中的权限分为两类：

- 普通权限：不会直接危险用户安全和隐私的权限，系统自动授权
- 危险权限：会直接危险用户安全和隐私的权限，用户手动授权（定位、联系人信息），Android共有9组24个危险权限
- 核心机制：必须由用户授权执行危险操作，程序不能擅自执行
- 保护方式：让用户可见、可知、可控
  - 版本<6.0系统时，在安装界面弹出提示信息
  - 随时查看任意程序的权限申请情况





- 运行时权限优点
  - 传统的权限：一次性授权、不认可则无法使用整个软件
  - 运行时权限：动态、灵活，允许使用中临时授权、允许选择性授权



- 运行时权限——危险权限
  - 危险权限需要单独处理
  - 同组危险权限共享授权：某个危险权限被授权，同组的其他危险权限也会被同时授权

表 Android中的危险权限（共9组24个）

权限组名	权限名
CALENDAR	READ_CALENDAR WRITE_CALENDAR
CAMERA	CAMERA
CONTACTS	READ_CONTACTS WRITE_CONTACTS GET_ACCOUNTS
LOCATION	ACCESS_FINE_LOCATION ACCESS_COARSE_LOCATION
MICROPHONE	RECORD_AUDIO
PHONE	READ_PHONE_STATE CALL_PHONE READ_CALL_LOG WRITE_CALL_LOG ADD_VOICEMAIL USE_SIP PROCESS_OUTGOING_CALLS
SENSORS	BODY_SENSORS
SMS	SEND_SMS RECEIVE_SMS READ_SMS RECEIVE_WAP_PUSH RECEIVE_MMS
STORAGE	READ_EXTERNAL_STORAGE WRITE_EXTERNAL_STORAGE

- 申请运行时权限

分为申请普通权限和申请危险权限：

- 申请普通权限：
  - 系统会默认应用程序取得该权限
  - 直接在配置文件的<uses-permission>标签中添加权限声明
- 申请危险权限：
  - 使用前必须得到用户授权
  - 首先，在配置文件的<uses-permission>标签中添加权限声明；然后，在活动类中申请运行时权限

危险权限几点说明

- >6.0系统使用危险权限：不仅要在配置文件中声明权限，还必须在活动类中进行运行时处理
- <6.0系统是否危险权限：只需要在在配置文件中声明权限

- 申请运行时权限——危险权限（续）

  申请危险权限主要包括判断是否已授权和申请指定权限，其中：

  - ContextCompat.checkSelfPermission()：判断用户是否已经给指定的危险权限授权
    - 第一个参数：Context
    - 第二个参数：权限名
    - 返回值：一个常量，PackageManager.PERMISSION_GRANTED（有权限），PackageManager.PERMISSION_DENIED（无权限）

- 申请运行时权限——危险权限（续）
  - ActivityCompat.requestPermissions()：向用户申请授权
    - 第一个参数：Activity的实例
    - 第二个参数：一个String数组，用于设置要申请的权限名，一次可以请求多个权限
    - 第三个参数：一个int类型，是请求码，要求唯一值

权限申请方法requestPermissions()的几点说明

- 权限参数传入的是数组，可以调用该方法一次请求多个权限；
- 传入权限数组参数以单个具体权限为单位，但弹框询问用户授权时，属于同一权限组的权限将自动合并询问授权一次
- 请求的权限必须事先在 AndroidManifest.xml中有声明，否则调用此方法请求时，将不弹框，而是直接返回“拒绝”的结果；
- 第一次请求权限时，用户点击了“拒绝”，第二次再请求该权限时，对话框将出现“不再询问”复选框，如果用户勾选了“不再询问”并点击了“拒绝”，则之后再请求此权限组时将不弹框，而是直接返回“拒绝”的结果。

```
    makeCall.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v){
            if(ContextCompat.checkSelfPermission(MainActivity.this, Manifest.permission.CALL_PHONE)!=
            PackageManager.PERMISSION_GRANTED){
                ActivityCompat.requestPermissions(MainActivity.this,new String[]{Manifest.permission.CA
            }else{
                call();
            }
        }
    });
}
private void call(){
    try {
        Intent intent = new Intent(Intent.ACTION_CALL);
        intent.setData(Uri.parse("tel:10086"));
        startActivity(intent);
    }catch (SecurityException e){
        e.printStackTrace();
    }
}
@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] g
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    switch (requestCode) {
        case 1:
            if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED){
                call();
            } else {
                Toast.makeText(this, "You denied the permission", Toast.LENGTH_SHORT).show();
```

申请危险权限的执行过程

- 首先，调用requestPermissions()，系统会弹出权限申请的对话框
- 然后，用户可以选择同意或者拒绝权限申请
- 最后，不论用户选择哪种结果，系统都会调用onRequestPermissionResult()，授权结果会通过grantResults参数

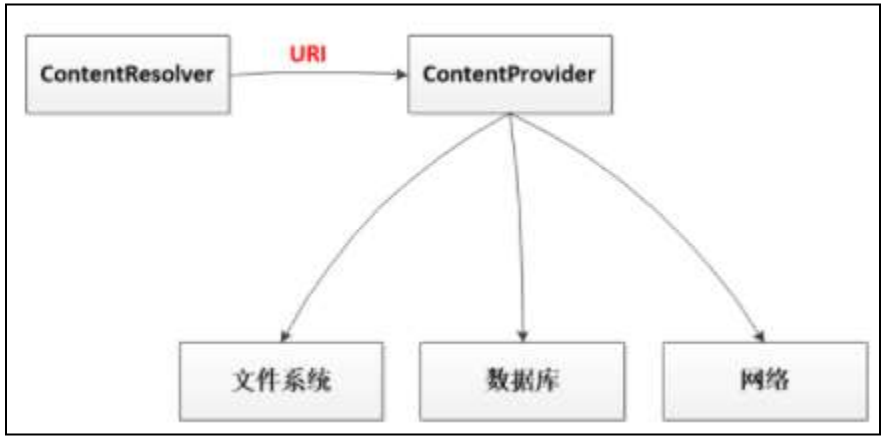


提示：在“权限管理”界面可以随时关闭已经授权的危险权限

- 访问其他应用程序的数据

采用内容提供器实现不同应用程序间的数据共享。

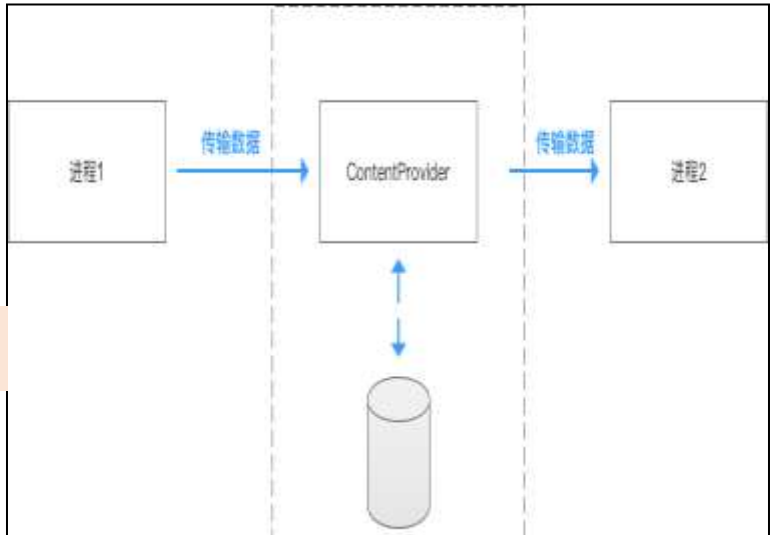
- ContentProvider：内容提供者，提供数据
- ContentResolver：内容解决者，使用数据



ContentProvider底层是采用Android中的Binder机制

理解内容提供器

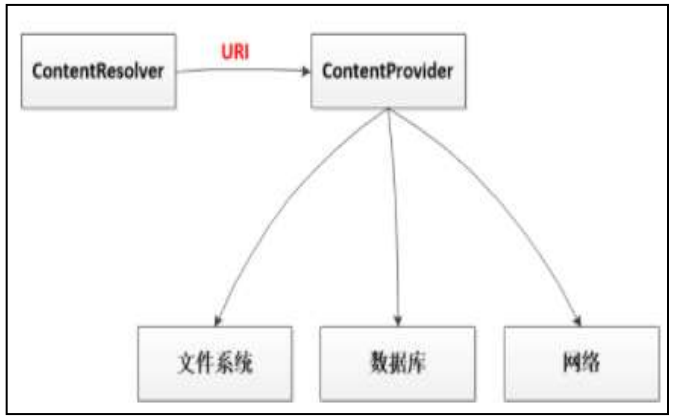
- 内容提供器：是数据暂存的仓库，真正存储和操作数据的数据源还是原来的存储方式
- 数据源：支持数据库、文件、XML、网络等

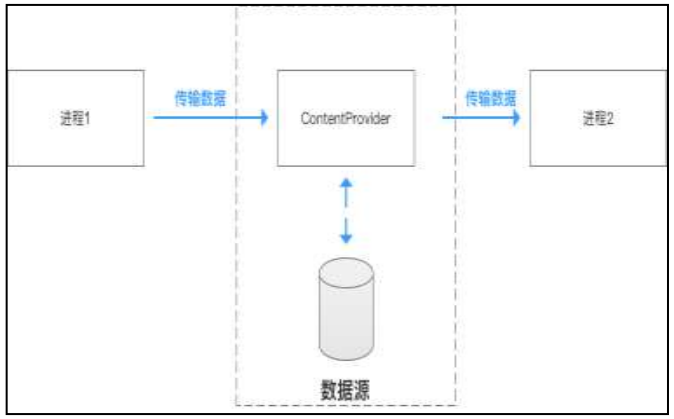


- 访问其他应用程序的数据

采用内容提供器实现不同应用程序（进程间）的数据共享。

- ContentProvider：内容提供者（进程A），提供数据，发送URI
- ContentResolver：内容解决者（进程B），使用数据，接收URI
- URI (Uniform Resource Identifier，统一资源标识符)：内容提供者以URI形式向外提供操作数据的接口
- MIME：ContentProvider根据URI返回MIME类型，指定某类文件(不同扩展名)采用某种应用程序打开





理解内容提供器

- 内容提供器：中转站，只负责转发URI
- 存储和操作数据的数据源：原来的存储方式
- 数据源：支持数据库、文件、XML、网络等

- URI的使用

系统为所有资源分配了唯一的URI，以“content://”为前缀标志

- 组成：authority + path [+ id]
  - authority：用于区分不同应用程序，一般是“content://+包名”
  - path：用于区分同一应用程序中的不同表，一般是数据库表名
  - id：数据库表中的字段id
- 格式：URI支持匹配通配符，*和 #
  - *：匹配任意长度的任何有效字符的字符串
  - #：匹配任意长度的数字字符的字符串
- 解析：调用Uri.parse()将URI解析为Uri对象
- 操作：利用Uri对象操作数据库



- 使用内容提供器组织数据

ContentProvider主要以类似数据库表形式组织数据（也支持文件数据，但应用较少）

```
<-- ContentProvider的6个主要方法 -->
// 外部进程向 ContentProvider 中添加数据
public Uri insert(Uri uri, ContentValues values)
// 外部进程 删除 ContentProvider 中的数据
public int delete(Uri uri, String selection, String[] selectionArgs)
// 外部进程更新 ContentProvider 中的数据
public int update(Uri uri, ContentValues values, String selection, String[] selectionArgs)
// 外部应用 获取 ContentProvider 中的数据
public Cursor query(Uri uri, String[] projection, String selection, String[] selectionArgs,  String sortOrder)
// ContentProvider创建后 或 打开系统后其它进程第一次访问该ContentProvider时 由系统进行调用
// 注：运行在ContentProvider进程的主线程，故不能做耗时操作
public boolean onCreate()
// 得到数据类型，即返回当前 Url 所代表数据的MIME类型
public String getType(Uri uri)
```

- 创建内容提供器的步骤
  - S1：新建类继承ContentProvider，重写6个方法
  - S2：采用UriMatcher类，匹配内容URI的功能
    - 添加URI：调用UriMatcher.addURI()，将进程A对外提供的URI添加到内容提供器
      - 第一个参数：authority
      - 第二个参数：path
      - 第三个参数：自定义代码
    - 匹配URI：调用UriMatcher.match()，尝试将进程A提供的URI与进程B期望访问的URI匹配
      - 参数：期望访问的URI对象
      - 返回值：如果匹配成功，则返回该URI中的自定义代码
  - S3：在配置文件注册内容提供器，并允许外部访问
    - 在<application>标签中添加<provider>标签，并设置权限

```
//第一步：新建类继承ContentProvider
public class MyProvider extends ContentProvider {
    @Override
    public boolean onCreate() {
        return false;
    }
     @Override
    public Cursor query(@NonNull Uri uri, @Nullable String[] projection, @Nullable String
selection, @Nullable String[] selectionArgs, @Nullable String sortOrder) {
        return null;
    }
    @Override
    public String getType(@NonNull Uri uri) {
        return null;
    }
     @Override
    public Uri insert(@NonNull Uri uri, @Nullable ContentValues values) {
        return null;
    }
     @Override
    public int delete(@NonNull Uri uri, @Nullable String selection, @Nullable String[]
selectionArgs) {
        return 0;
    }
     @Override
    public int update(@NonNull Uri uri, @Nullable ContentValues values, @Nullable String selection
```

```
//第二步：采用UriMatcher类，匹配内容URI的功能
public class MyProvider extends ContentProvider {
    public static final int TABLE1_DIR = 0;
    public static final int TABLE1_ITEM = 1;
    public static final int TABLE2_DIR = 2;
    public static final int TABLE2_ITEM = 3;
    private static UriMatcher uriMatcher; static {
        uriMatcher = new UriMatcher(UriMatcher.NO_MATCH);//常量UriMatcher.NO_MATCH表示不匹配任何路径的返回

        uriMatcher.addURI("com.example.providertest.provider", "table1", TABLE1_DIR);
        uriMatcher.addURI("com.example.providertest.provider", "table1/#", TABLE1_ITEM);
        uriMatcher.addURI("com.example.providertest.provider", "table2", TABLE2_DIR);
        uriMatcher.addURI("com.example.providertest.provider", "table2/#", TABLE2_ITEM);
    @Override
    public boolean onCreate() {
        return false;
    }
    @Override
    public Cursor query(@NonNull Uri uri, @Nullable String[] projection, @Nullable String selection, @Null
    String[] selectionArgs, @Nullable String sortOrder) {
        switch (uriMatcher.match(uri)) {
            case TABLE1_DIR:
                // 查询 table1 表中所有数据
                break;
            case TABLE1_ITEM:
                // 查询 table1 表中单条数据
                break;
            case TABLE2_DIR:
```

```
//第三步：在配置文件中注册内容提供器
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.contentprovider">
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        ...
        <provider
            android:name=".DatabaseProvider"
            android:authorities="com.example.databasetest.provider"
            android:enabled="true"
            android:exported="true">
        </provider>
    </application>
</manifest>
```

- MIME类型

调用内容提供器的getType()方法，返回Uri对象所对应的MIME类型。

- 必须以 vnd 开头；
- 如果URI以path结尾，则后接android.cursor.dir/
- 如果URI以id结尾，则后接android.cursor.item/；
- 最后接上vnd.<authority>.<path>

```
uriMatcher.addURI("com.example.providertest.provider", "table1", TABLE1_DIR);
uriMatcher.addURI("com.example.providertest.provider", "table1/#", TABLE1_ITEM);
uriMatcher.addURI("com.example.providertest.provider", "table2", TABLE2_DIR);
uriMatcher.addURI("com.example.providertest.provider", "table2/#", TABLE2_ITEM);
```

```
public String getType(@NonNull Uri uri) {    switch (uriMatcher.match(uri)){
        case TABLE1_DIR
:           return"vnd.android.cursor.dir/vnd.com.example.providertest.provider.table1";
        case TABLE1_ITEM:
            return "vnd.android.cursor.item/vnd.com.example.providertest.provider.table1";
```

```
ArrayAdapter<String> adapter;
List<String > contactsList = new ArrayList<>();
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    ListView contactsView = (ListView) findViewById(R.id.contacts_view);
    adapter = new ArrayAdapter<>(this,android.R.layout.simple_list_item_1,contactsList);
    contactsView.setAdapter(adapter);
    if(ContextCompat.checkSelfPermission(this, Manifest.permission.READ_CONTACTS) != PackageManager.PERMIS
        ActivityCompat.requestPermissions(this,new String[]{Manifest.permission.READ_CONTACTS},1);
    }else{
        readContacts();
    }
}
private void readContacts() {
    Cursor cursor = null;
    try{
        cursor = getContentResolver().query(ContactsContract.CommonDataKinds.Phone.CONTENT_URI,null,null,n
        if(cursor != null){
            while (cursor.moveToNext()){
                @SuppressLint("Range") String displayName = cursor.getString(cursor.getColumnIndex(Contact
                @SuppressLint("Range") String number = cursor.getString(cursor.getColumnIndex(ContactsCont
                contactsList.add(displayName+"\n"+number);
            }
            adapter.notifyDataSetChanged();
        }
    }catch (Exception e){
        e.printStackTrace();
    }finally{
        if(cursor != null){
```

- Activity：Android组件中最基本也是最为常见用的四大组件之一，Activity中所有操作都与用户密切相关，是一个负责与用户交互的组件，可以通过setContentView(View)来显示指定控件。

  Intent：中文意思是“意图，意向”，在Android中提供了Intent机制来协助应用间的交互与通讯，Intent负责对应用中一次操作的动作，动作涉及数据、附加数据进行描述，Android则根据此Intent的描述，负责找到对应的组件，将 Intent传递给调用的组件，并完成组件的调用。Intent不仅可用于应用程序之间，也可用于应用程序内部的Activity/Service之间的交互。因此，可以将Intent理解为不同组件之间通信的“媒介”专门提供组件互相调用的相关信息。

  ContentProvider：为程序提供了在应用程序之前共享数据的一种机制，而我们知道每一个应用程序都是运行在不同的应用程序的，数据和文件在不同应用程序之间达到数据的共享不是没有可能，而是显得比较复杂，而正好Android中的ContentProvider则达到了这一需求，比如有时候需要操作手机里的联系人，手机里的多媒体等一些信息，都可以用到这个ContentProvider来达到我们所需。

  Service：通常称之为“后台服务”，其中“后台”一词是相对于前台而言的，具体是指其本身的运行并不依赖于用户可视的UI界面，因此，从实际业务需求上来理解，Service的适用场景应该具备以下条件：1.并不依赖于用户可视的UI界面，2.具有较长时间的运行特性。